מבוא לטרנספורם פורייה
Fourier Transform Introduction

4

תוכן

- ייצוג אותות לפי בסיסים שונים -דוגמאות
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

תוכן

- ייצוג אותות לפי בסיסים שונים -דוגמאות
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

ייצוג אותות לפי בסיסים שונים

ייצוג לפי הבסיס הסטנדרטי:

$$(f(0), f(1), \ldots, f(N-1)) =$$
$$f(0)(1,0,0,\ldots,0,0) + f(1)(0,1,0,\ldots,0,0) + \cdots + f(N-1)(0,0,0,\ldots 0,1) =$$
$$\sum_{u=0}^{N-1} f(u) e_u$$

דוגמא ייצוג לפי בסיס אחר

$$(f(0), f(1), \ldots, f(N-1)) =$$
$$c(0)(1,-1,0,\ldots,0,0) + c(1)(0,1,-1,\ldots,0,0) + \cdots + c(N-1)(-1,0,0,\ldots 0,1) =$$
$$\sum_{u=0}^{N-1} c(u) d_u$$

(חסר משמעות, רק לצורך הדוגמא):

7

תוכן

- ייצוג אותות לפי בסיסים שונים -דוגמאות
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

ייצוג לפי הבסיס הסטנדרטי:

- דוגמא

$$(2,3,4,4)$$

$$\begin{aligned}(2,3,4,4) = \quad & 2\cdot(1,0,0,0) \\ & +3\cdot(0,1,0,0) \\ & +4\cdot(0,0,1,0) \\ & +4\cdot(0,0,0,1)\end{aligned}$$

9

תוכן

- ייצוג אותות לפי בסיסים שונים -דוגמאות
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

Lagrange Interpolation

רוצים להעביר פולינום דרך זוג הנקודות הבאות:

(1,3), (2,4), (7,11).

$$P(x) = \frac{(x-2)(x-7)}{(1-2)(1-7)} \times 3 + \frac{(x-1)(x-7)}{(2-1)(2-7)} \times 4 + \frac{(x-1)(x-2)}{(7-1)(7-2)} \times 11$$

$$P(1) = 3$$
$$P(2) = 4$$
$$P(7) = 11$$

Lagrange Interpolation

נוסחה כללית

$$P(x) = \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}y_1 + \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}y_2 + \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}y_3$$

$$P(x) = \sum_{1}^{3} P_i(x) y_i$$

https://brilliant.org/wiki/lagrange-interpolation/

תוכן

- ייצוג אותות לפי בסיסים שונים -דוגמאות
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

מספרים מרוכבים ושורשי היחידה

בסיס פורייה

אחד ספציפי ומאוד חשוב!

בסיס פורייה למרחב האותות בגודל N:

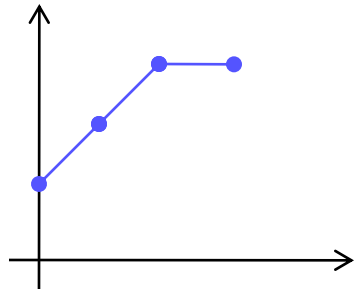
$$g_u(x) = e^{\frac{2\pi i}{N}xu} = \cos(\frac{2\pi}{N}xu) + i\sin(\frac{2\pi}{N}xu)$$

$$u = 0,\ldots,N-1 \quad x = 0,\ldots N-1$$

ייצוג לפי הבסיס הסטנדרטי:

- בסיס פורייה

$f(x) = (2,3,4,4)$
$(0,2), (1,3), (2,4), (3,4)$



$f(x) = (2,3,4,4)$

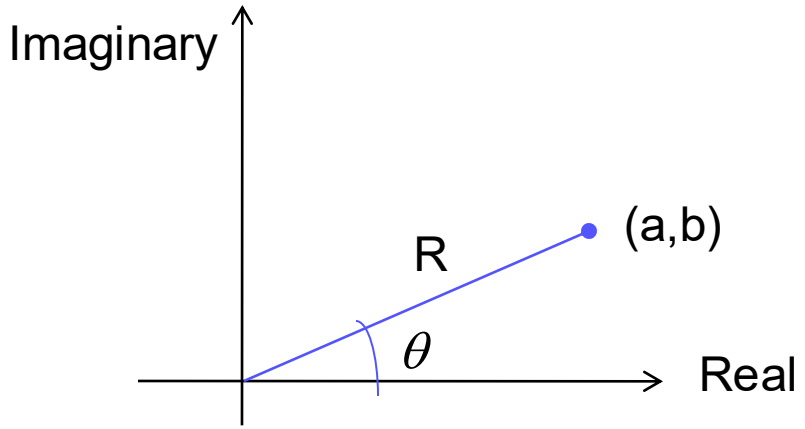
$$= ??e^{\frac{2\pi i}{N}x\cdot 0} + ??e^{\frac{2\pi i}{N}x\cdot 1} + ??e^{\frac{2\pi i}{N}x\cdot 2} + ??e^{\frac{2\pi i}{N}x\cdot 3} + \ldots.$$

17

כבר נחזור למציאת המקדמים...

שלוש שקפים, תזכורת בנושא מספרים מרוכבים....

מספרים מרוכבים



ישנם 2 יצוגים:

ייצוג קרטזי: $a+ib$

ייצוג פולארי: $R\cdot e^{i\theta}$

מעבר מפולרי לקרטזי: $R\cos\theta+iR\sin\theta$

מעבר מקרטזי לפולרי: $\sqrt{a^2+b^2}\cdot e^{i\tan^{-1}(b/a)}$

שורשי יחידה

$$Z^n = 1$$

ניתן להראות שלמשוואה ליעיל, ישנם n פתרונות:
הפתרונות הם n ששורשי יחידה מסדר N:

$$e^{\frac{2\pi i}{N}\cdot 0}, e^{\frac{2\pi i}{N}\cdot 1}, e^{\frac{2\pi i}{N}\cdot 2}, \ldots, e^{\frac{2\pi i}{N}\cdot (N-1)}$$

<u>תזכורת בנושא מספרים מרוכבים: שקף 2 מתוך 3</u>

$$Z^n = 1$$

$$w_n = e^{\frac{2\pi i}{n}}$$

$$Z_k = \cos\left(\frac{2\pi k}{n}\right) + i\sin\left(\frac{2\pi k}{n}\right) = e^{\frac{2\pi k i}{n}} = (e^{\frac{2\pi i}{n}})^k = (w_n)^k$$

על מנת למצוא את n הפתרונות נגדיר משתנה עזר z_k

דוגמא: $Z^4 = 1$

ראשית נמצא את w_n

$$w_4 = e^{\frac{2\pi i}{4}} = \cos\left(\frac{2\pi}{4}\right) + i\sin\left(\frac{2\pi}{4}\right) = 0 + 1i = i$$

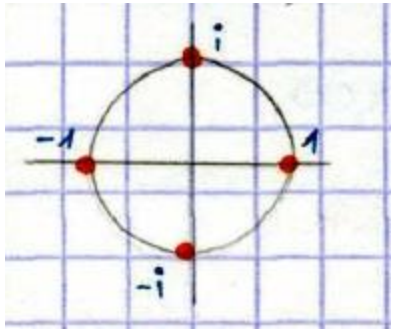
אלו 4 הפתרונות...

$$z_0 = (w_4)^0 = i^0 = 1$$

$$z_1 = (w_4)^1 = i^1 = i$$

$$z_2 = (w_4)\text{^}2 = i\text{^}2 = -1$$

$$z_3 = (w_4)\text{^}3 = i\text{^}3 = -i$$



תוכן

- ייצוג אותות לפי בסיסים שונים
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

נחזור לבסיס פוריה...

בסיס פורייה

אחד ספציפי ומאוד חשוב!

בסיס פורייה למרחב האותות בגודל N:

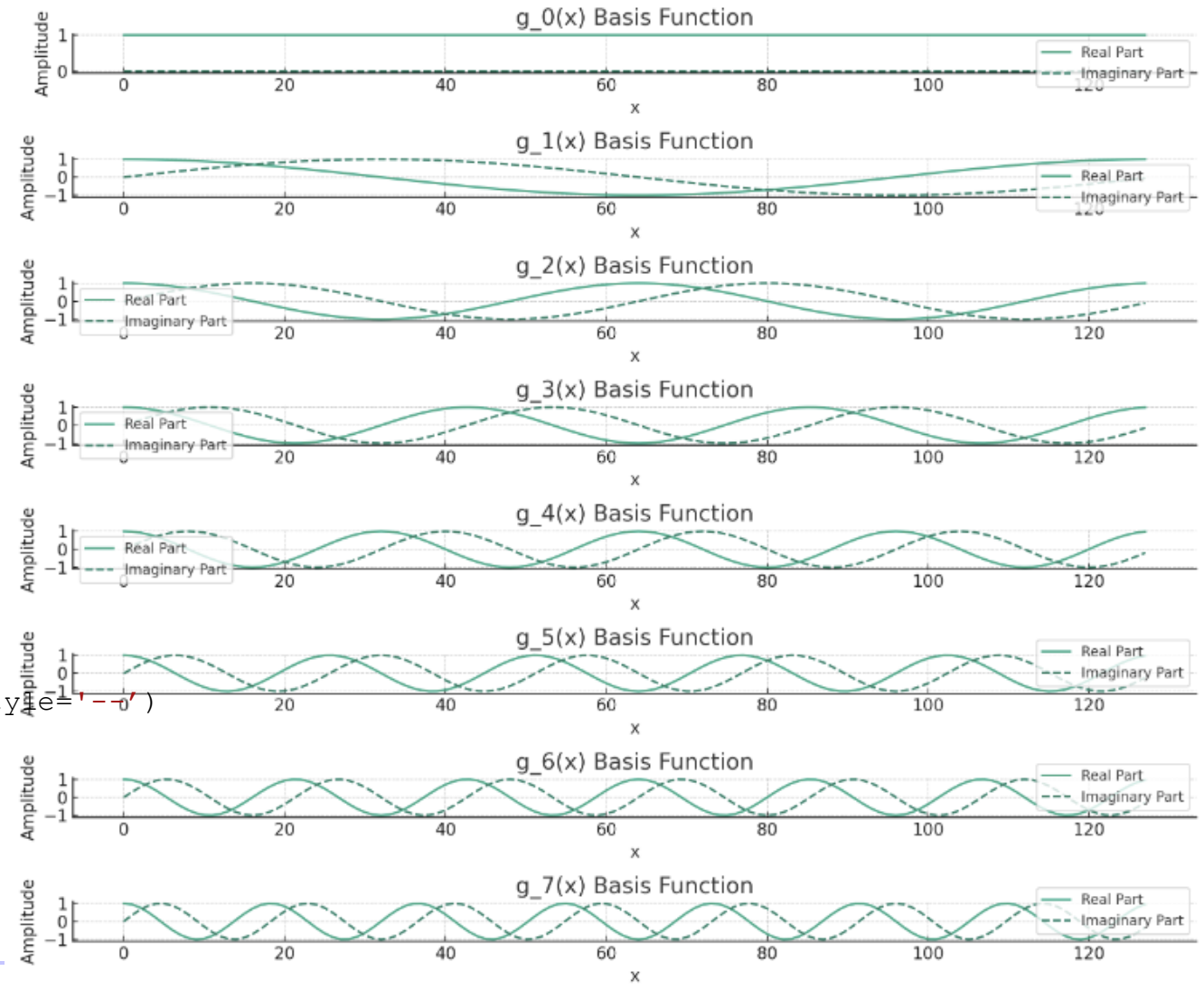
$$g_u(x) = e^{\frac{2\pi i}{N}xu} = \cos(\frac{2\pi}{N}xu) + i\sin(\frac{2\pi}{N}xu)$$

$$u = 0,\ldots,N-1 \quad x = 0,\ldots N-1$$

$$g_u(x) = e^{\frac{2\pi i}{N}xu} = \cos(\frac{2\pi}{N}xu) + i\sin(\frac{2\pi}{N}xu)$$
$$u = 0,\ldots,N-1 \quad x = 0,\ldots N-1$$

איך נראה בסיס פורייה ?

```
import numpy as np
import matplotlib.pyplot as plt

# Define the parameters for the plot
N = 128 # Number of points
components = 8 # Number of Fourier components to plot

# Create a figure with subplots
fig, axs = plt.subplots(components, figsize=(12, 10))

# Calculate and plot the first 8 Fourier basis functions
for u in range(components):
    # Compute the basis function g_u(x)
    x = np.arange(N)
    g_u = np.exp(2j * np.pi * u * x / N)

    # Plot the real part (cosine component)
    axs[u].plot(x, g_u.real, label='Real Part')

    # Plot the imaginary part (sine component)
    axs[u].plot(x, g_u.imag, label='Imaginary Part', linestyle='--')
    # Set title and labels
     axs[u].set_title(f'g_{u}(x) Basis Function')
     axs[u].set_xlabel('x')
     axs[u].set_ylabel('Amplitude')
     axs[u].grid(True)
     axs[u].legend()

# Adjust the layout
plt.tight_layout()
plt.show()
```



28

ייצוג לפי הבסיס הסטנדרטי:

- בסיס פורייה

$$(2,3,4,4)$$

$$= ??e^{\frac{2\pi i}{N}x\cdot 0} + ??e^{\frac{2\pi i}{N}x\cdot 1} + ??e^{\frac{2\pi i}{N}x\cdot 2} + ??e^{\frac{2\pi i}{N}x\cdot 3} + \ldots.$$

31

הגדרת טרנספורם פורייה

- הפונקציה $f(x)$ ניתנת לפירוק לפי בסיס פורייה באופן הבא:

$$f(x)=\sum_{u=0}^{N-1}F(u)e^{\frac{2\pi i}{N}xu} \qquad x=0,1,\ldots,N-1$$

- המקדמים $F(u)$ נתונים על ידי:

$$F(u)=\frac{1}{N}\sum_{x=0}^{N-1}f(x)e^{-\frac{2\pi i}{N}xu} \qquad u=0,1,\ldots,N-1$$

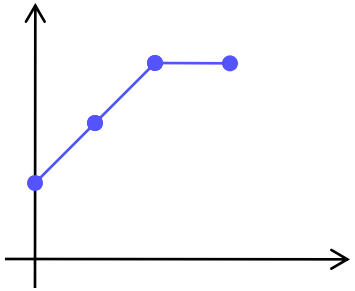
- בהינתן f ניתן לחשב עבורו את F. F נקרא **טרנספורם פורייה** של f.
- בהינתן F ניתן לחשב עבורו את f. f נקרא **טרנספורם פורייה הפוך** של F.

32

טרנספורם פורייה - דוגמא

$$f(x) = (2,3,4,4)$$

$$(0,2),(1,3),(2,4),(3,4)$$

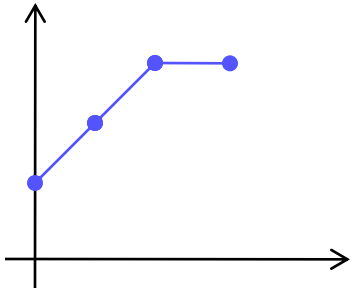


$$F(u) = \frac{1}{N}\sum_{x=0}^{N-1} f(x)e^{-\frac{2\pi i}{N}xu} \qquad u = 0,1,\ldots,N-1$$

$$= ??e^{\frac{2\pi i}{N}x\cdot 0} + ??e^{\frac{2\pi i}{N}x\cdot 1} + ??e^{\frac{2\pi i}{N}x\cdot 2} + ??e^{\frac{2\pi i}{N}x\cdot 3} + \ldots.$$

טרנספורם פורייה - דוגמא

$$f(x) = (2,3,4,4)$$
$$(0,2),(1,3),(2,4),(3,4)$$



$$F(u) = \frac{1}{N}\sum_{x=0}^{N-1} f(x)e^{-\frac{2\pi i}{N}xu} \qquad u = 0,1,\ldots,N-1$$

$$F(0) = \frac{1}{4}(2+3+4+4) = 3.25$$

$$F(1) = \frac{1}{4}\left(2e^{-\frac{2\pi i}{4}1\cdot 0} + 3e^{-\frac{2\pi i}{4}1\cdot 1} + 4e^{-\frac{2\pi i}{4}1\cdot 2} + 4e^{-\frac{2\pi i}{4}1\cdot 3}\right) = -\frac{1}{2} + \frac{1}{4}i$$

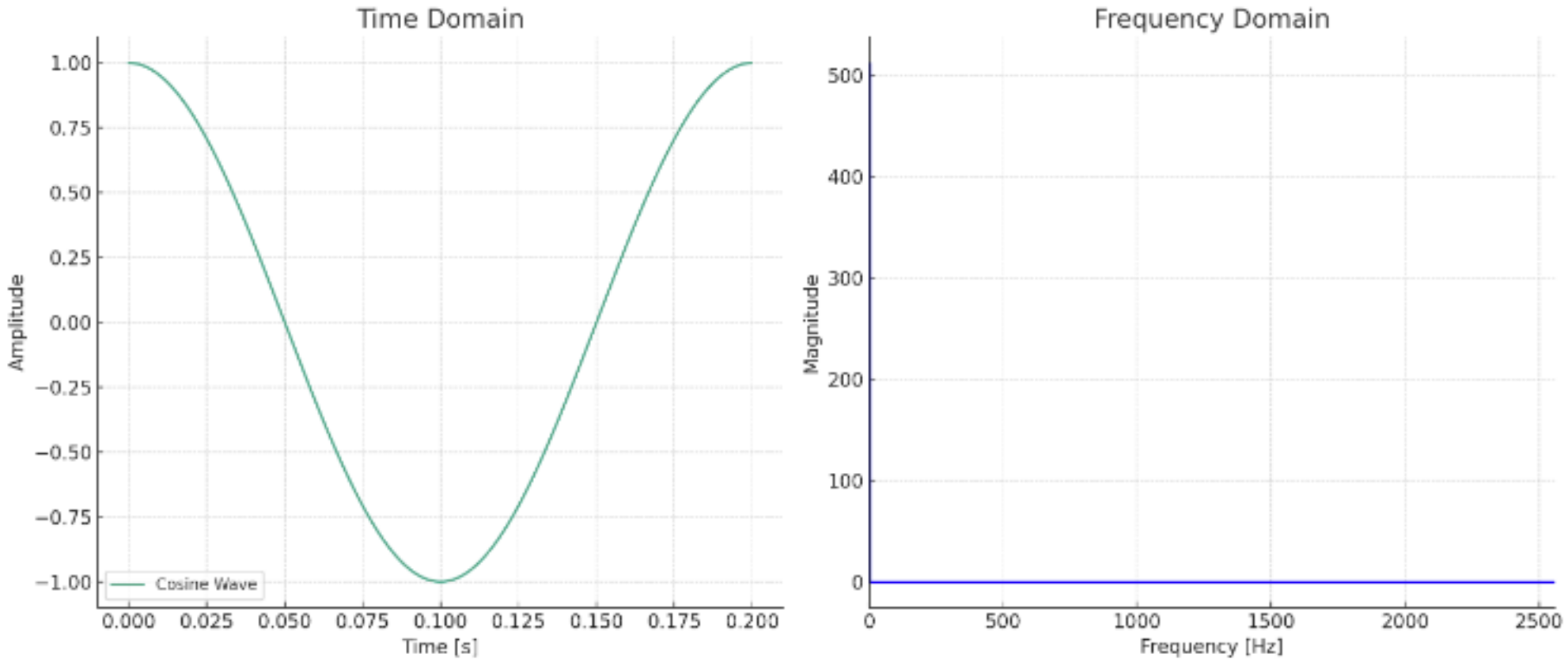
$$F(2) = \frac{1}{4}\left(2e^{-\frac{2\pi i}{4}2\cdot 0} + 3e^{-\frac{2\pi i}{4}2\cdot 1} + 4e^{-\frac{2\pi i}{4}2\cdot 2} + 4e^{-\frac{2\pi i}{4}2\cdot 3}\right) = -\frac{1}{4}$$

$$F(3) = \frac{1}{4}\left(2e^{-\frac{2\pi i}{4}3\cdot 0} + 3e^{-\frac{2\pi i}{4}3\cdot 1} + 4e^{-\frac{2\pi i}{4}3\cdot 2} + 4e^{-\frac{2\pi i}{4}3\cdot 3}\right) = -\frac{1}{2} - \frac{1}{4}i$$

35

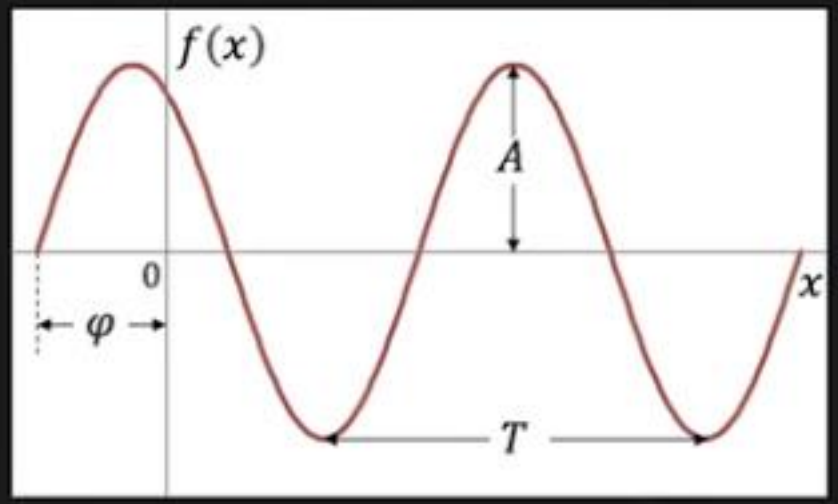
תוכן

- ייצוג אותות לפי בסיסים שונים
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד


Time Domain
Amplitude
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Cosine Wave
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
Time [s]
Frequency Domain
Magnitude
500
400
300
200
100
0
0
500
1000
1500
2000
2500
Frequency [Hz]

Sinusoid

$$f(x) = A\sin(2\pi ux + \varphi)$$

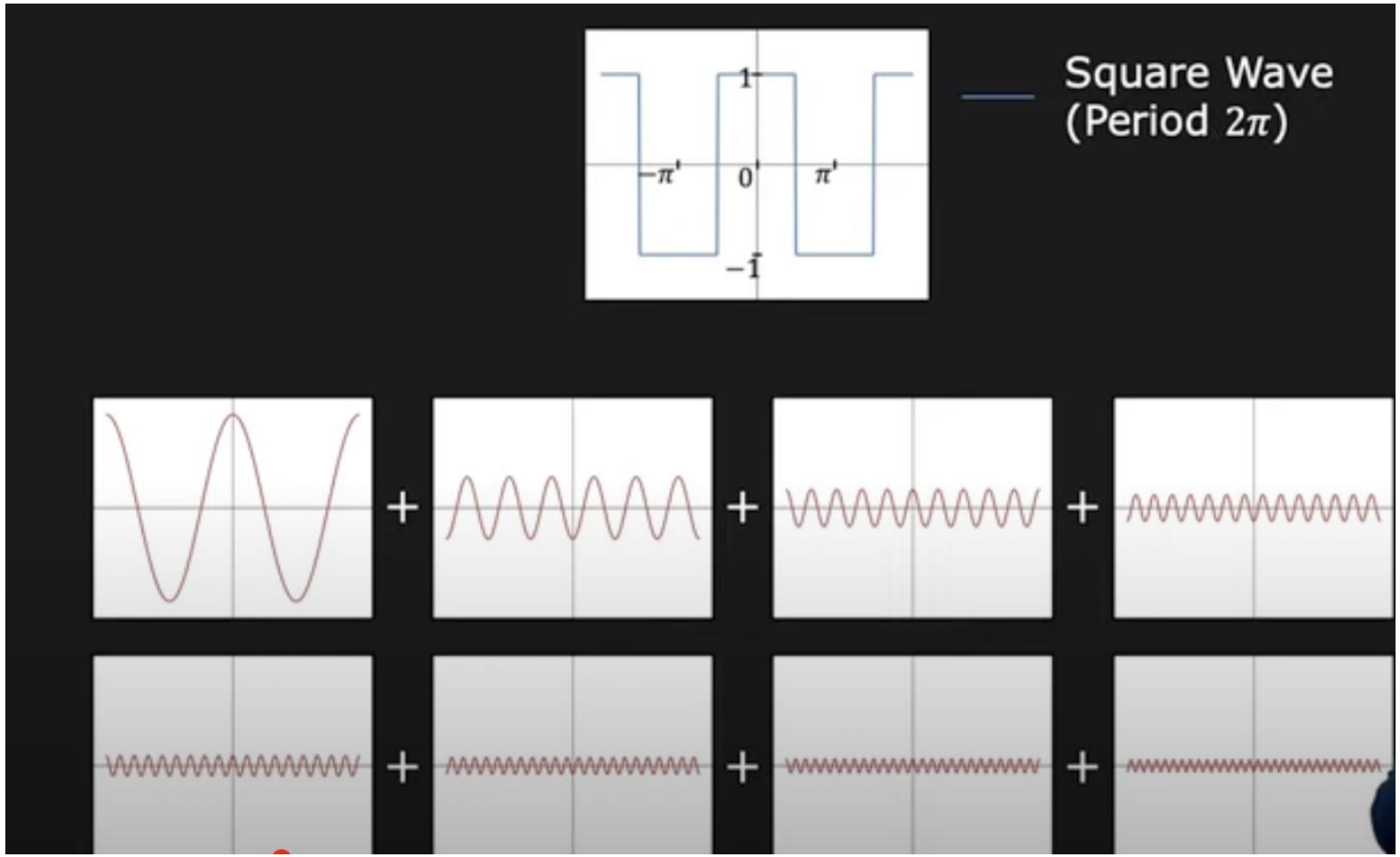


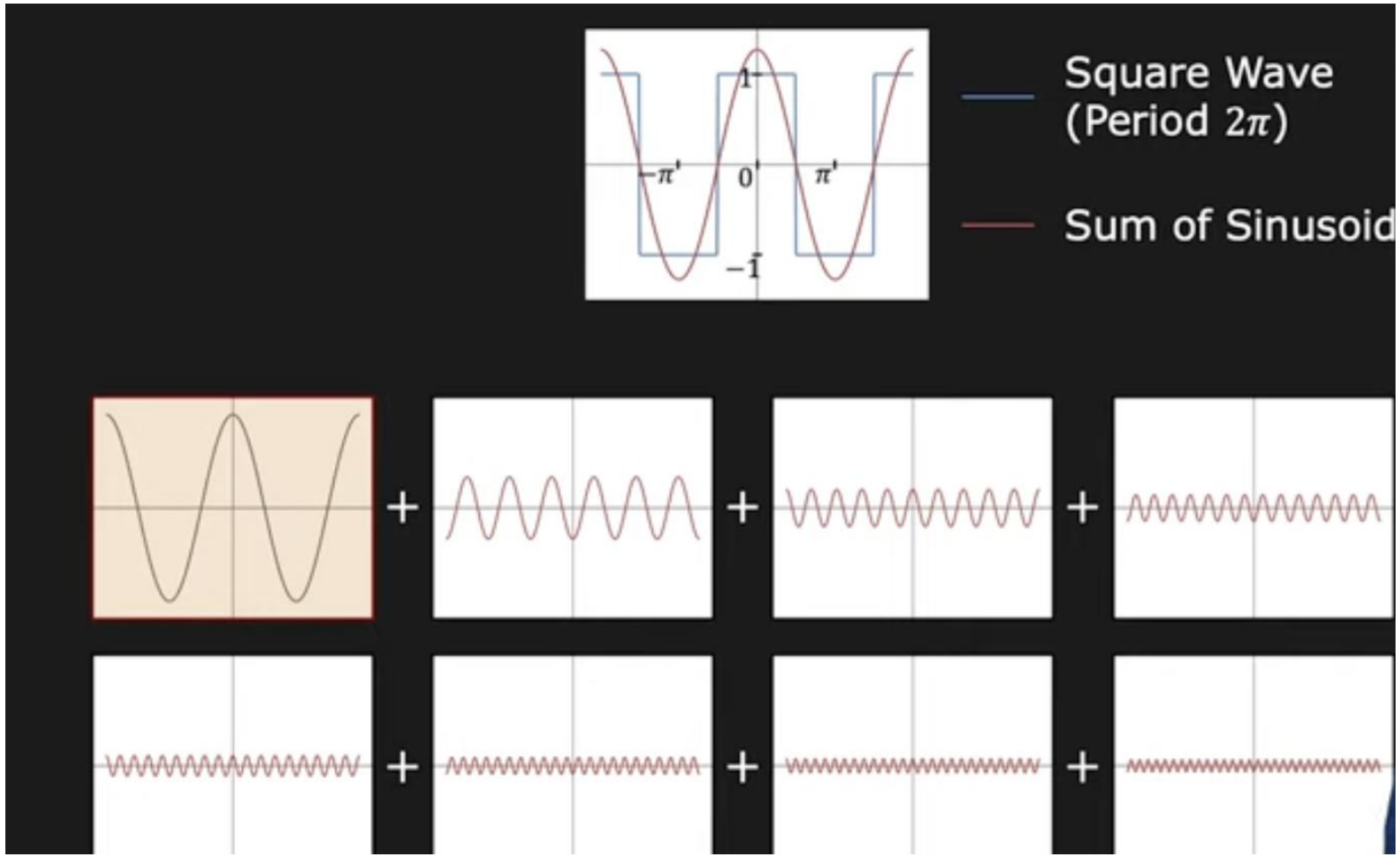
A: Amplitude

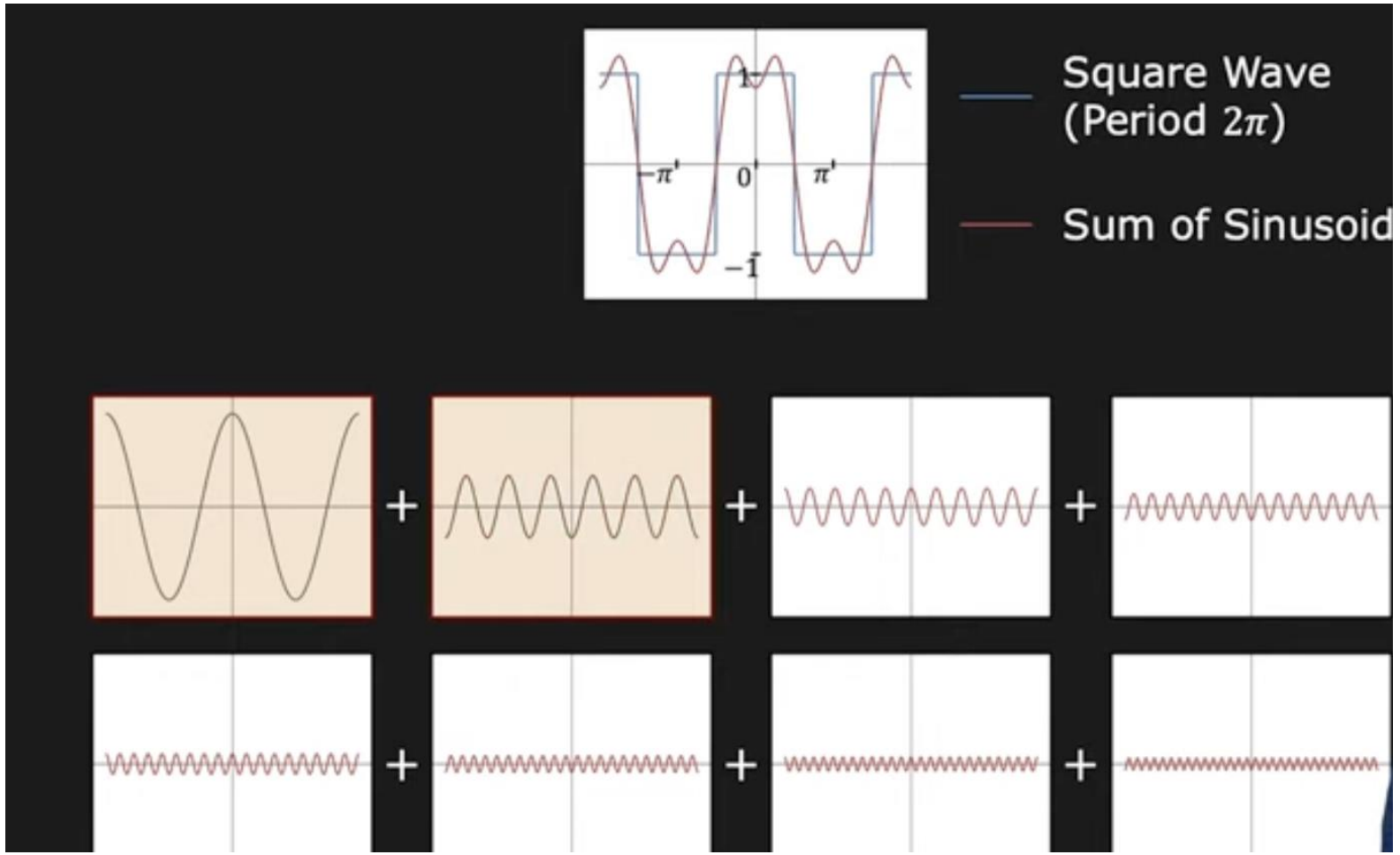
T: Period

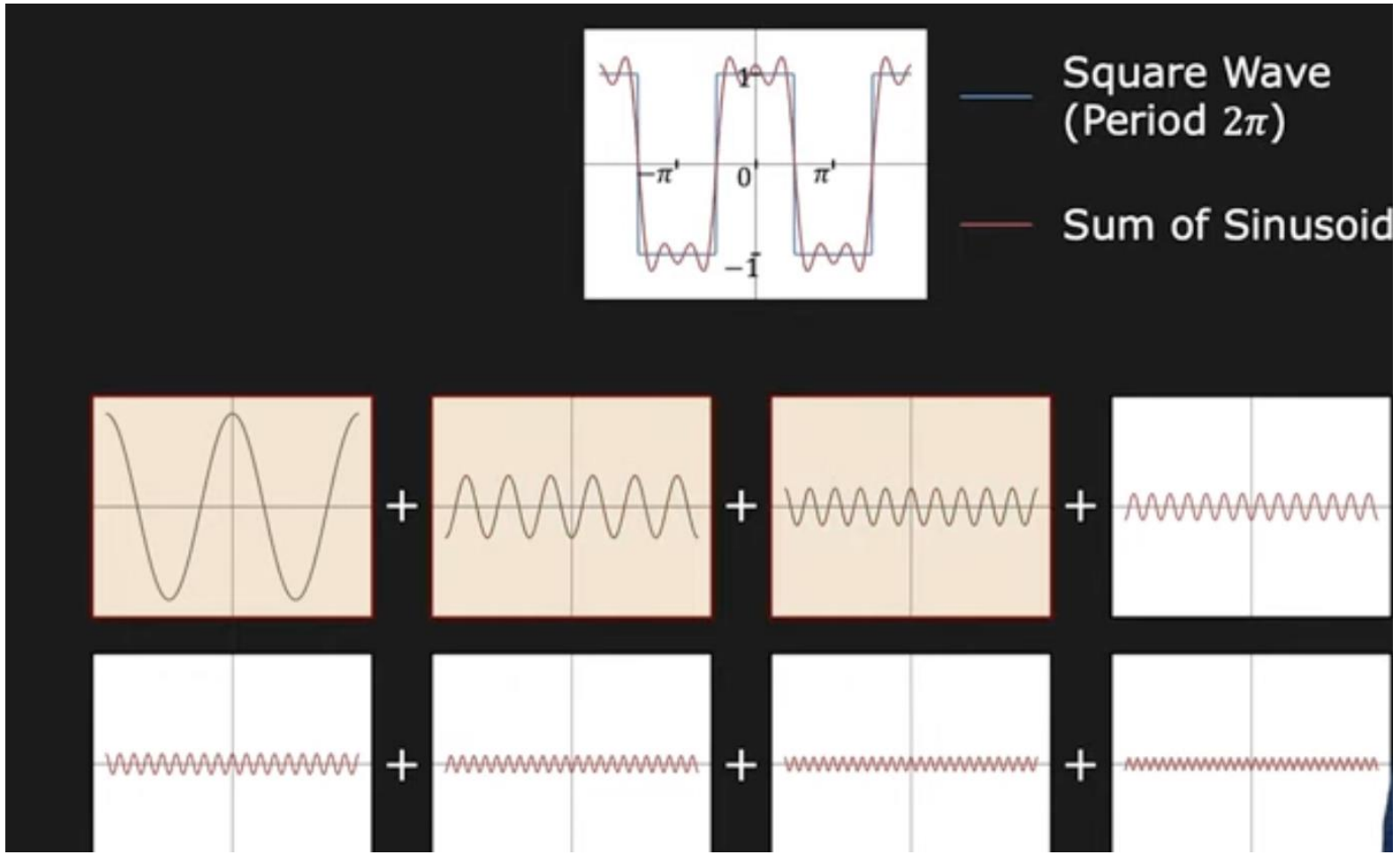
φ: Phase

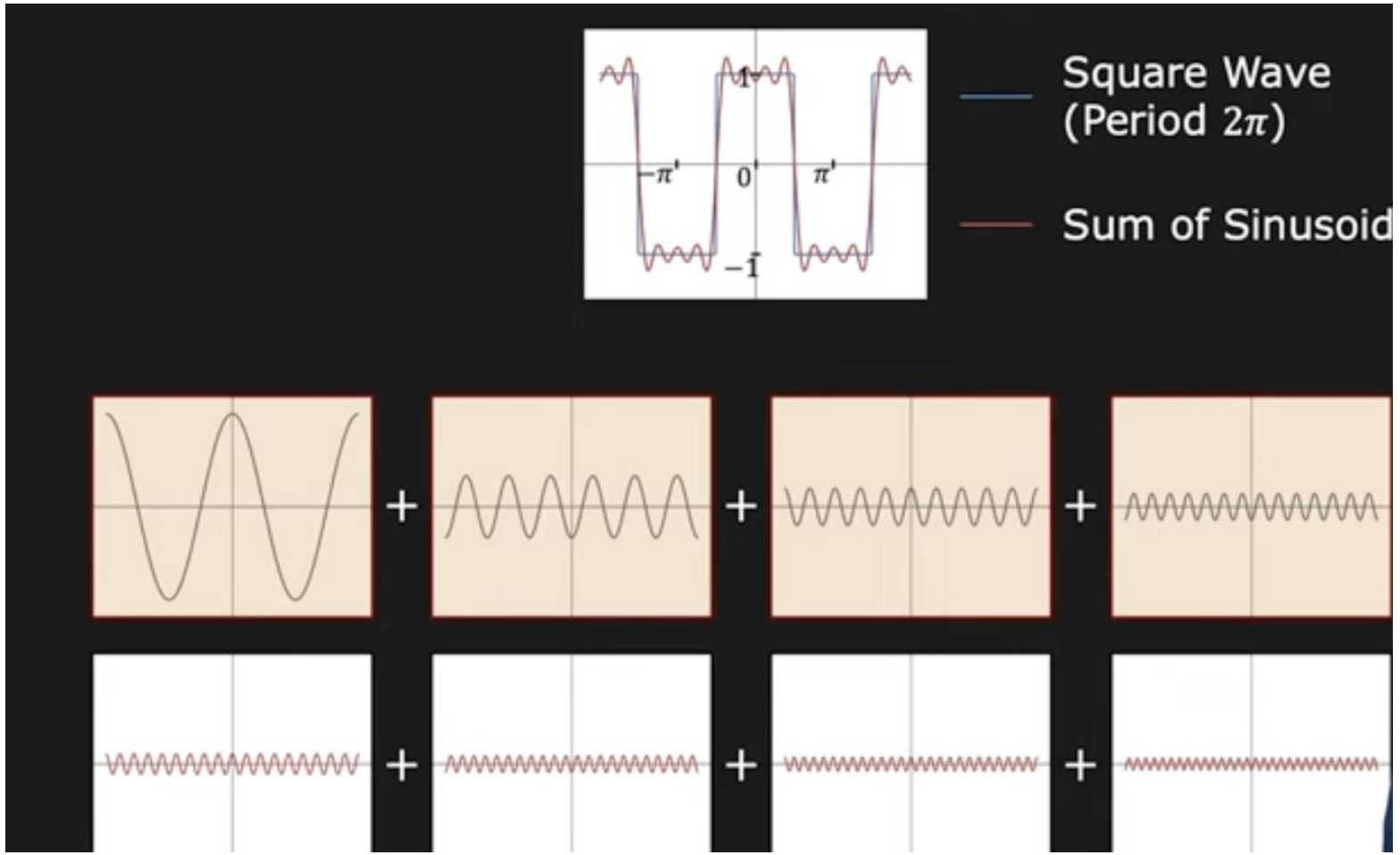
u: Frequency $(1/T)$

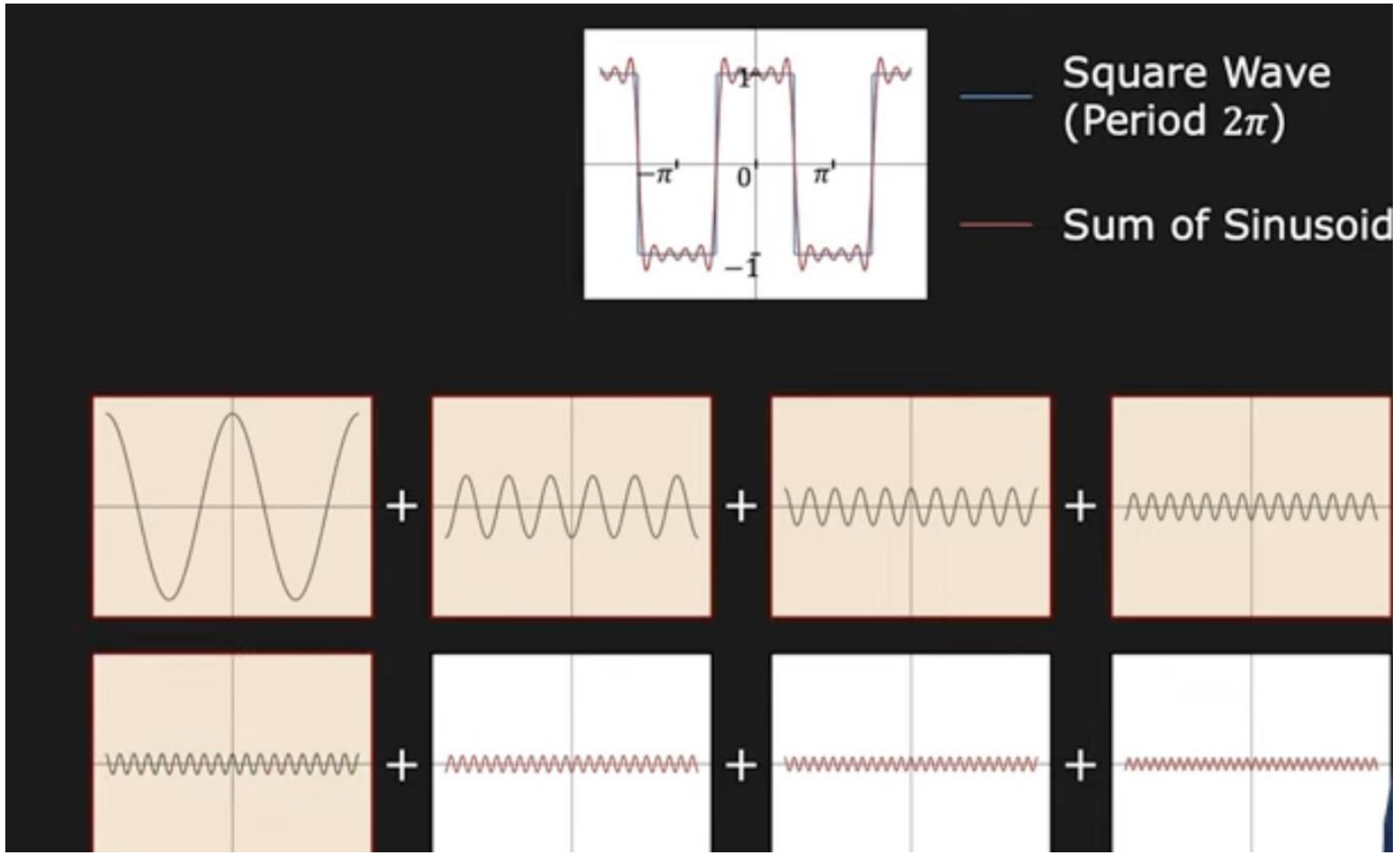

Square Wave
(Period 2π)
1
−1
−π'
0'
π'

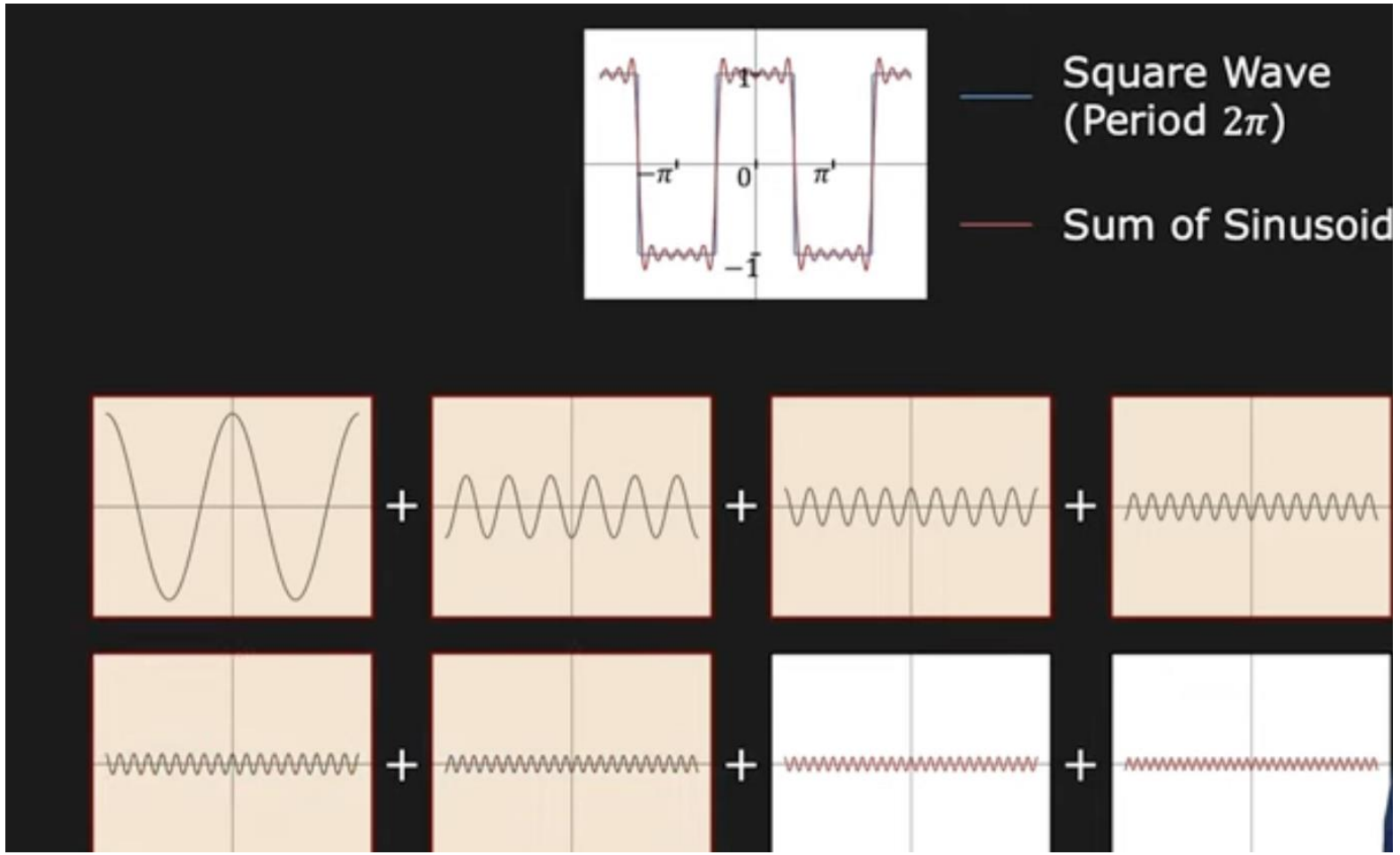

Square Wave
(Period 2π)
Sum of Sinusoid
1
−1
−π'
0
π'

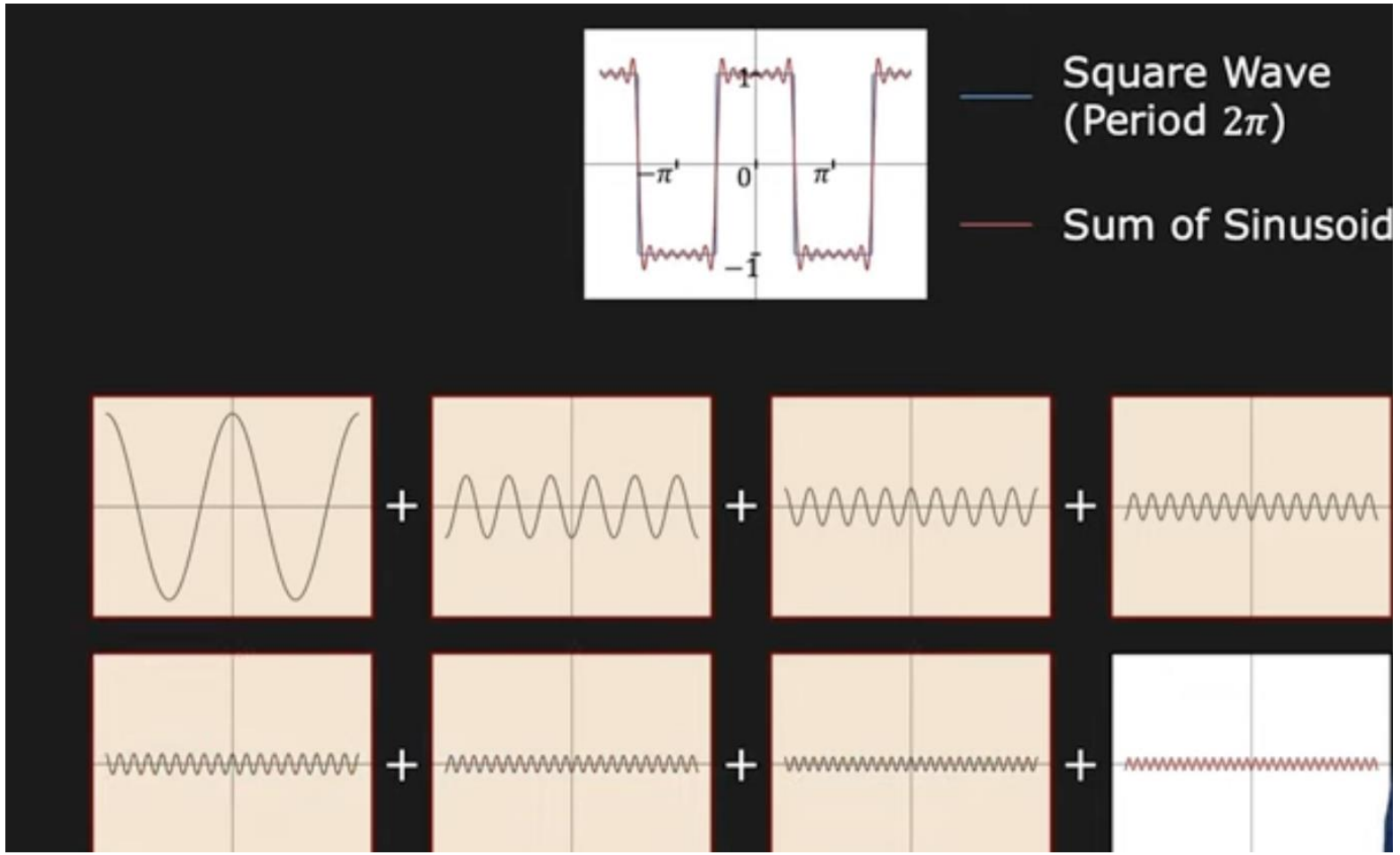

Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0'
π'
−1

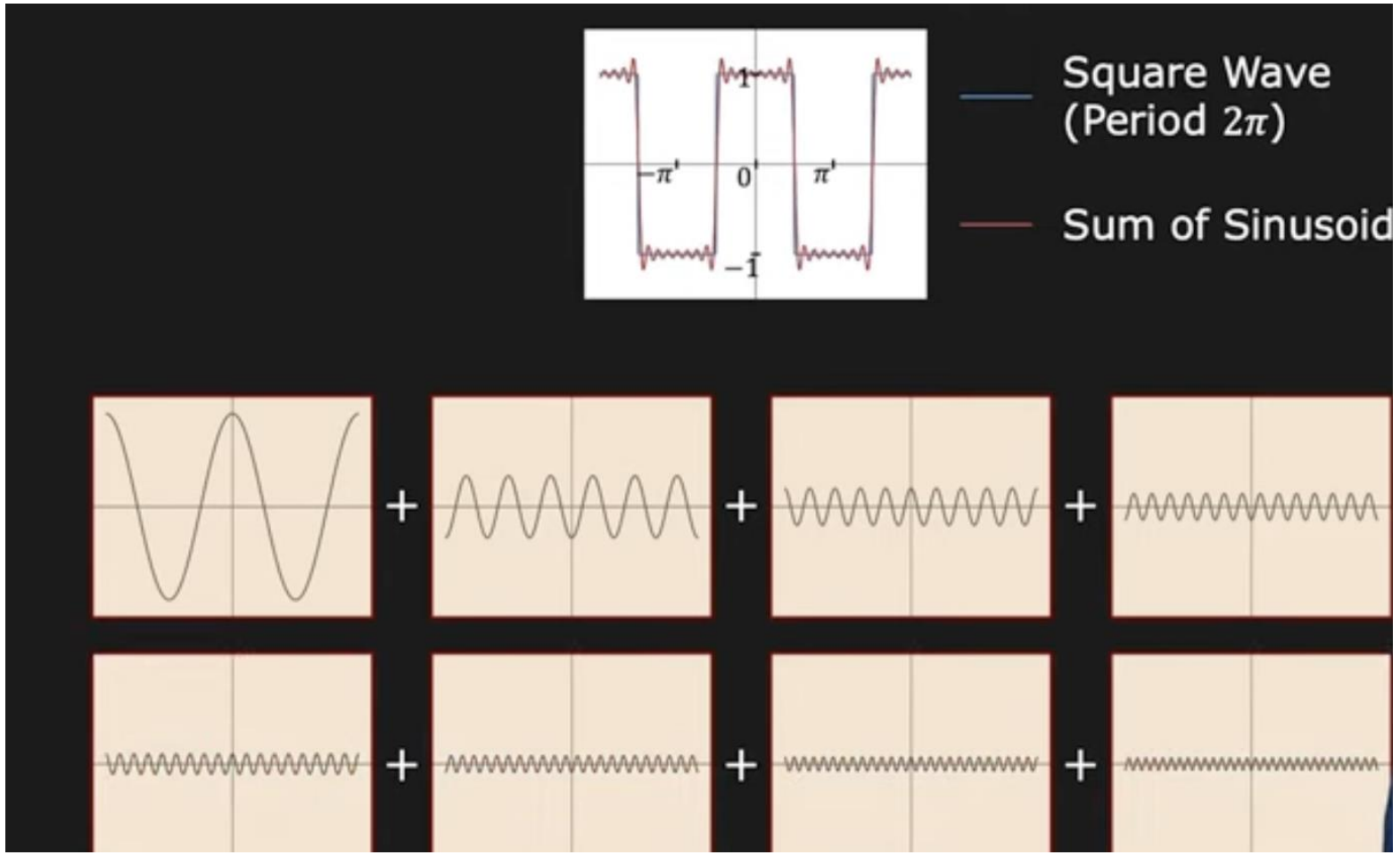

Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0'
π'
−1


Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0'
π'
−1


Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0'
π'
−1


Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0
π'
−1


Square Wave
(Period 2π)
Sum of Sinusoid
1
−π'
0'
π'
−1


Square Wave
(Period 2π)
Sum of Sinusoid
1
−1
−π
0
π

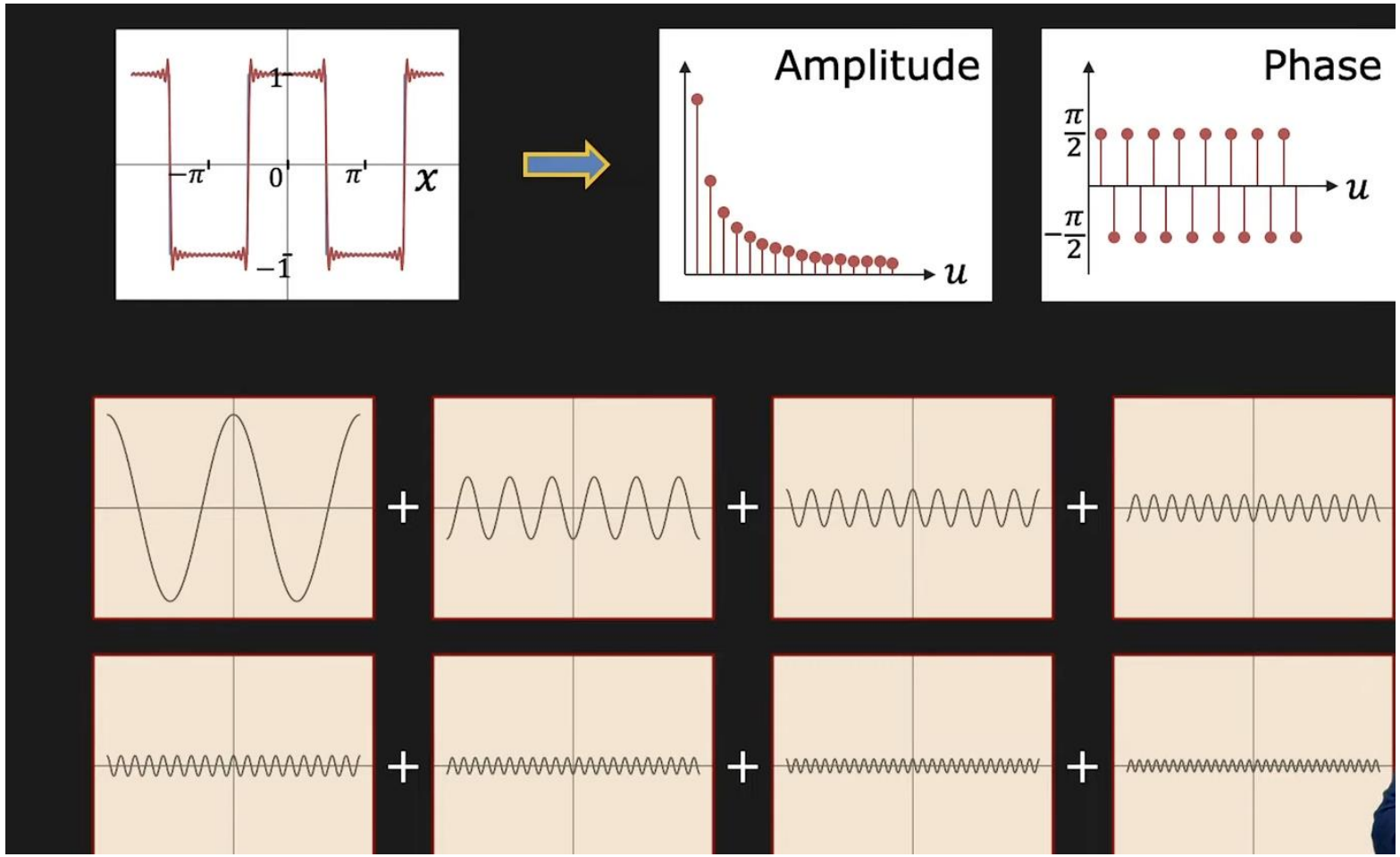
(Euler Formula)

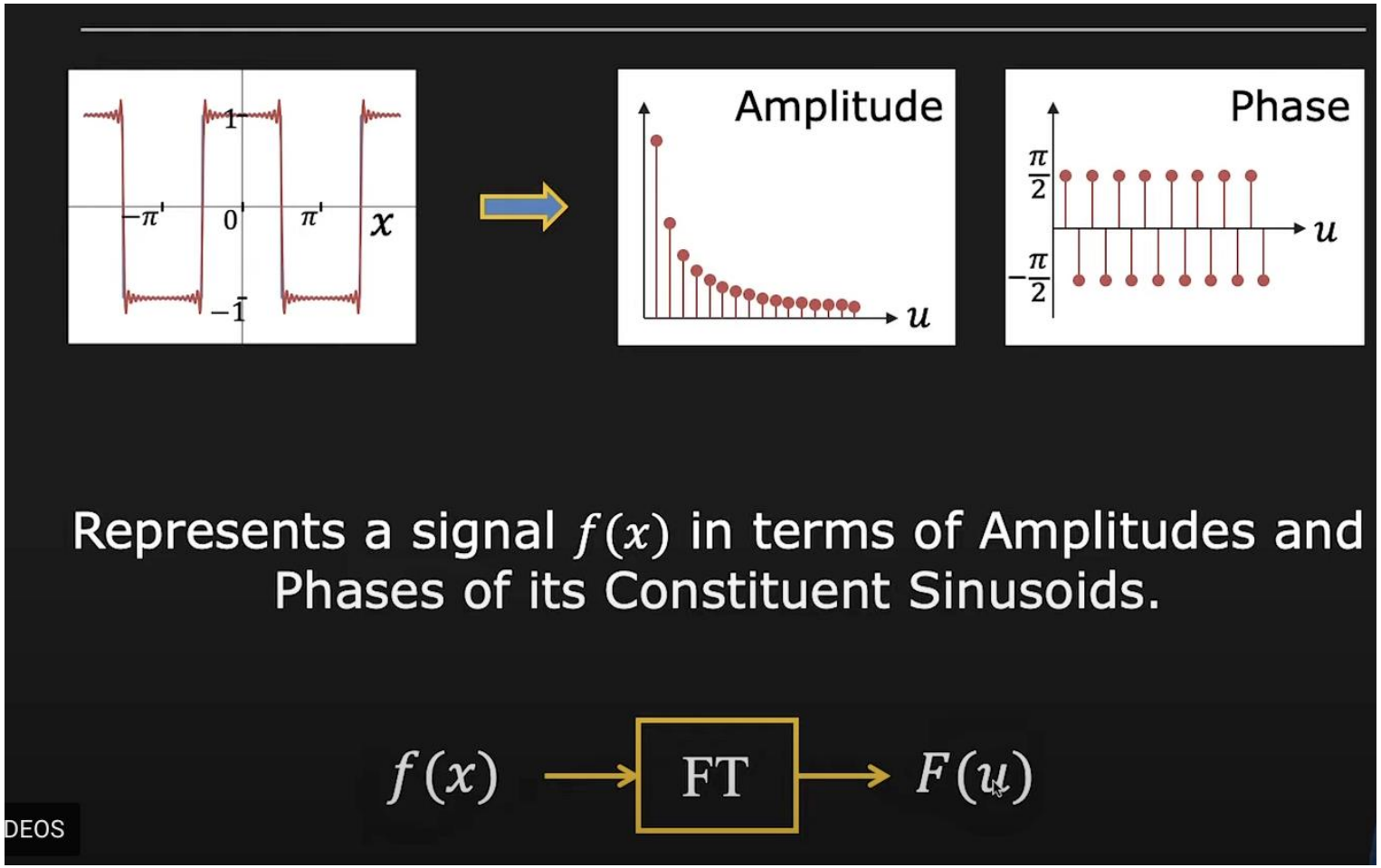
$$\boxed{e^{i\theta} = \cos\theta + i\sin\theta} \qquad i = \sqrt{-1}$$

Expand $e^{i\theta}$ using Taylor Series:

$$e^{i\theta} = 1 + i\theta + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \frac{(i\theta)^4}{4!} + \frac{(i\theta)^5}{5!} + \frac{(i\theta)^6}{6!} + \ldots$$

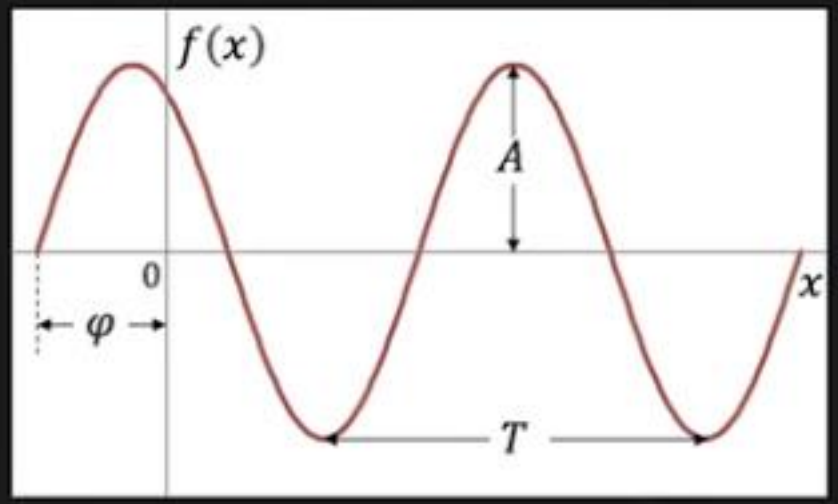
$$e^{i\theta} = \underbrace{\left(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \frac{\theta^6}{6!} + \cdots\right)}_{\cos\theta} + i\underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \frac{\theta^7}{7!} + \cdots\right)}_{\sin\theta}$$


1
$-\pi$
0
π
x
-1
Amplitude
u
Phase
$\frac{\pi}{2}$
u
$-\frac{\pi}{2}$


1
$-\pi$
0
π
x
-1
Amplitude
u
Phase
$\frac{\pi}{2}$
u
$-\frac{\pi}{2}$
Represents a signal $f(x)$ in terms of Amplitudes and Phases of its Constituent Sinusoids.
$f(x)$
FT
$F(u)$
DEOS

Sinusoid

$$f(x) = A\sin(2\pi ux + \varphi)$$



A: Amplitude

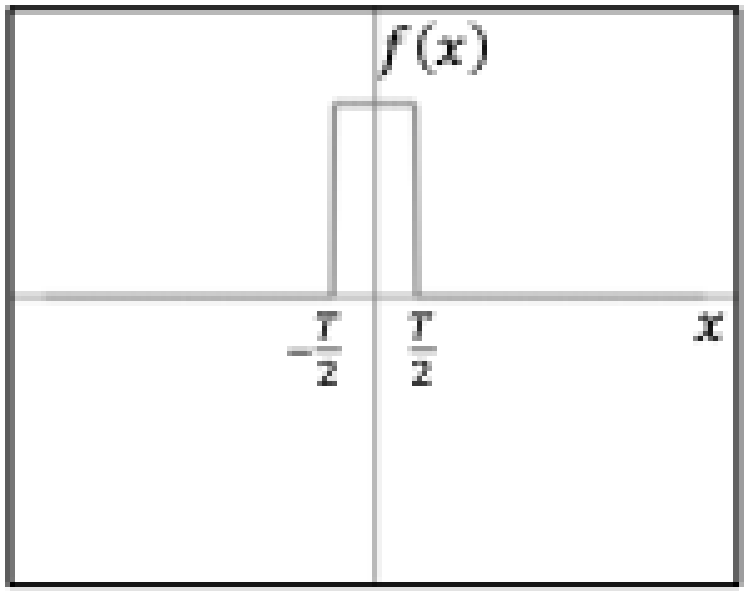
T: Period

φ: Phase

u: Frequency $(1/T)$

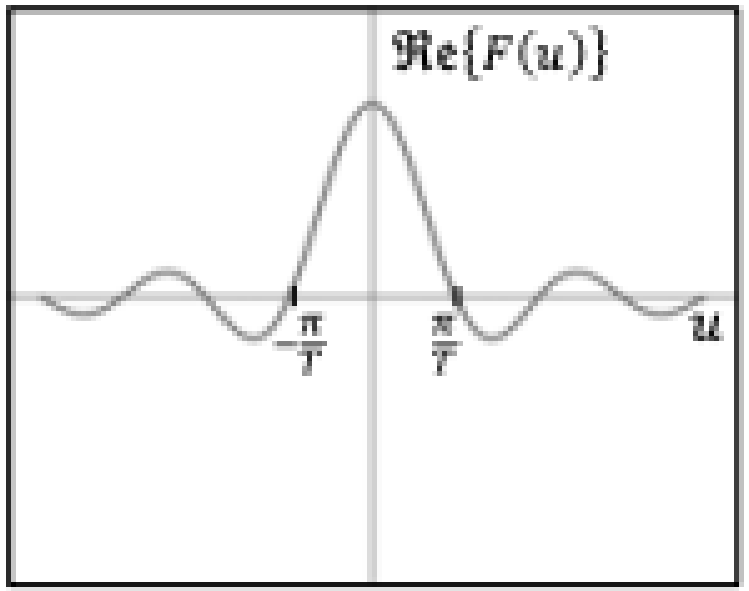
Fourier Transform Examples

Signal $f(x)$



$$f(x) = \text{Rect}\left(\frac{x}{T}\right)$$

Fourier Transform $F(u)$

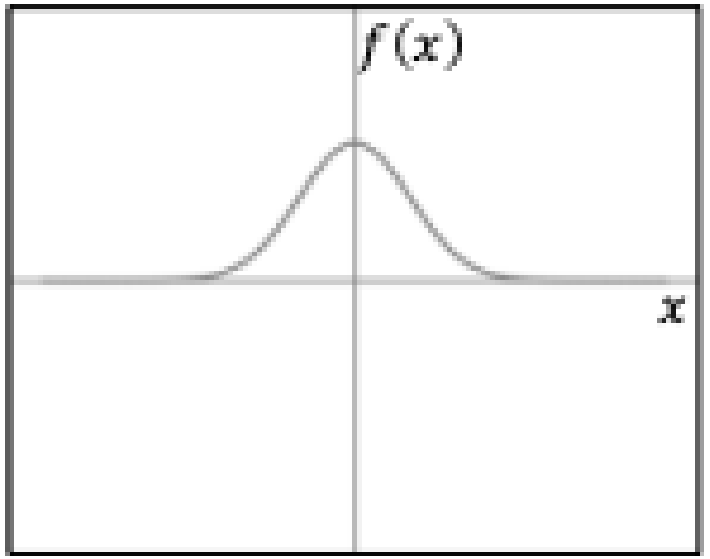


$$F(u) = T\,\text{sinc}\,Tu$$

עיבוד סיפרתי של תמונות: מבוא לטרנספורם פורייה

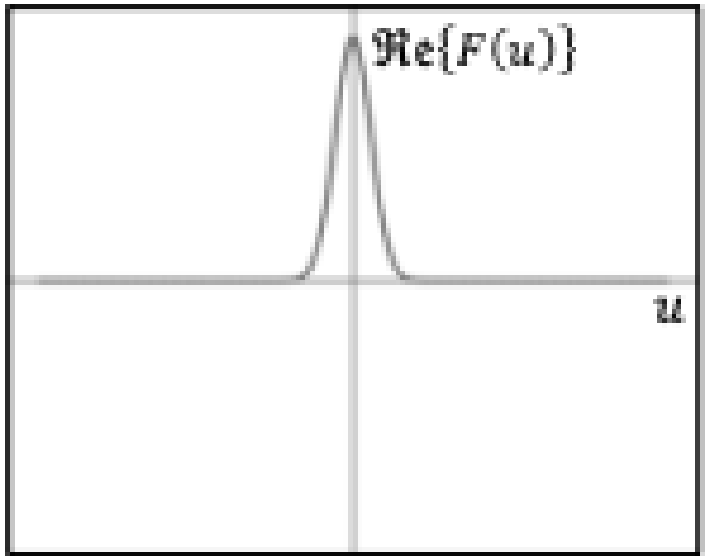
Fourier Transform Examples

Signal $f(x)$



$$f(x) = e^{-ax^2}$$

Fourier Transform $F(u)$



$$F(u) = \sqrt{\pi/a}\ e^{-\pi^2 u^2/a}$$

תוכן

- ייצוג אותות לפי בסיסים שונים
  - בסיס הסטנדרטי
  - פולינום -- Lagrange Interpolation
- בסיס פוריה - מספרים מורכבים \ שורשי היחידה
- בסיס פוריה
- טרנספורם פוריה – דוגמאות
- טרנספורם פוריה - דו-ממד

בדו-ממד

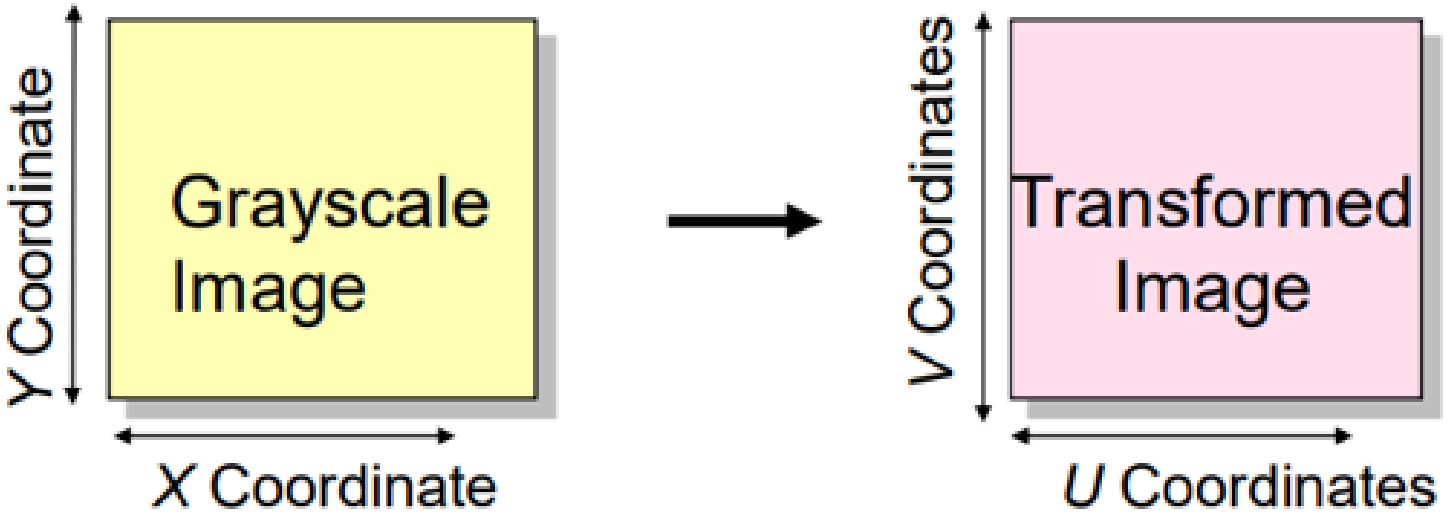
הגדרת טרנספורם פורייה דו ממד

- אברי הבסיס:

$$g_{u,v}(x,y) = e^{2\pi i(\frac{ux}{N}+\frac{vy}{M})} \qquad u = 0,1,\ldots,N-1 \quad v = 0,1,\ldots,M-1$$

הגדלים של u, v קובעים את התדר

היחס u/v קובע את הכיוון



הגדרת טרנספורם פורייה דו ממד

- אברי הבסיס:

$$g_{u,v}(x,y) = e^{2\pi i(\frac{ux}{N}+\frac{vy}{M})} \qquad u = 0,1,\ldots,N-1 \quad v = 0,1,\ldots,M-1$$

הגדלים של u, v קובעים את התדר

היחס u/v קובע את הכיוון

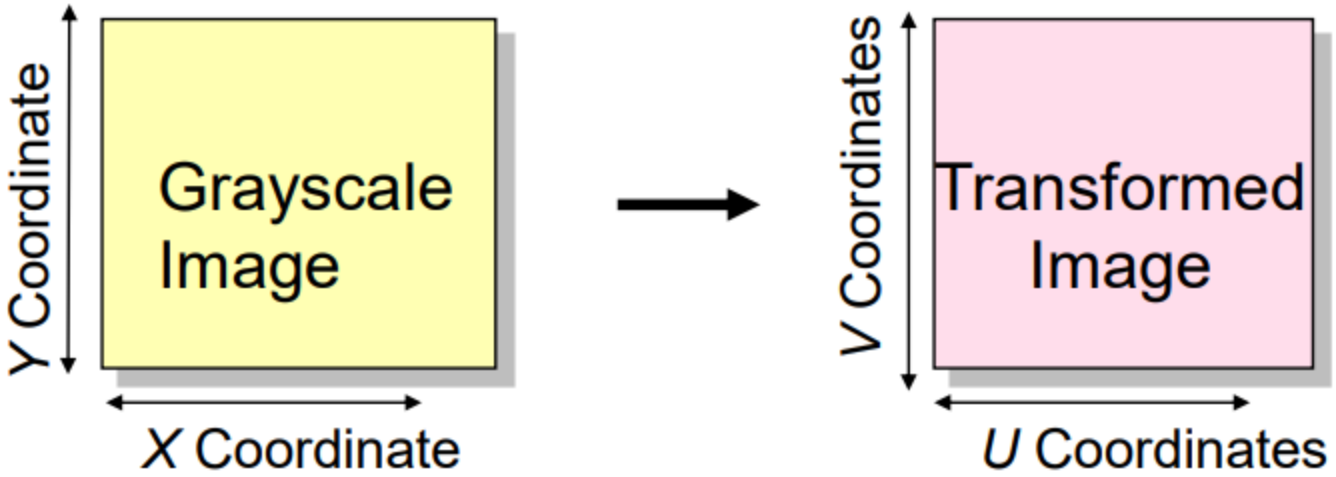
- טרנספורם פורייה דו ממדי:

$$f(x,y) = \sum_{u=0}^{N-1}\sum_{v=0}^{M-1} F(u,v)e^{2\pi i(\frac{ux}{N}+\frac{vy}{M})} \qquad x = 0,1,\ldots,N-1 \quad y = 0,1,\ldots,M-1$$

$$F(u,v) = \frac{1}{NM}\sum_{x=0}^{N-1}\sum_{y=0}^{M-1} f(x,y)e^{-2\pi i(\frac{ux}{N}+\frac{vy}{M})} \qquad u = 0,1,\ldots,N-1 \quad v = 0,1,\ldots,M-1$$

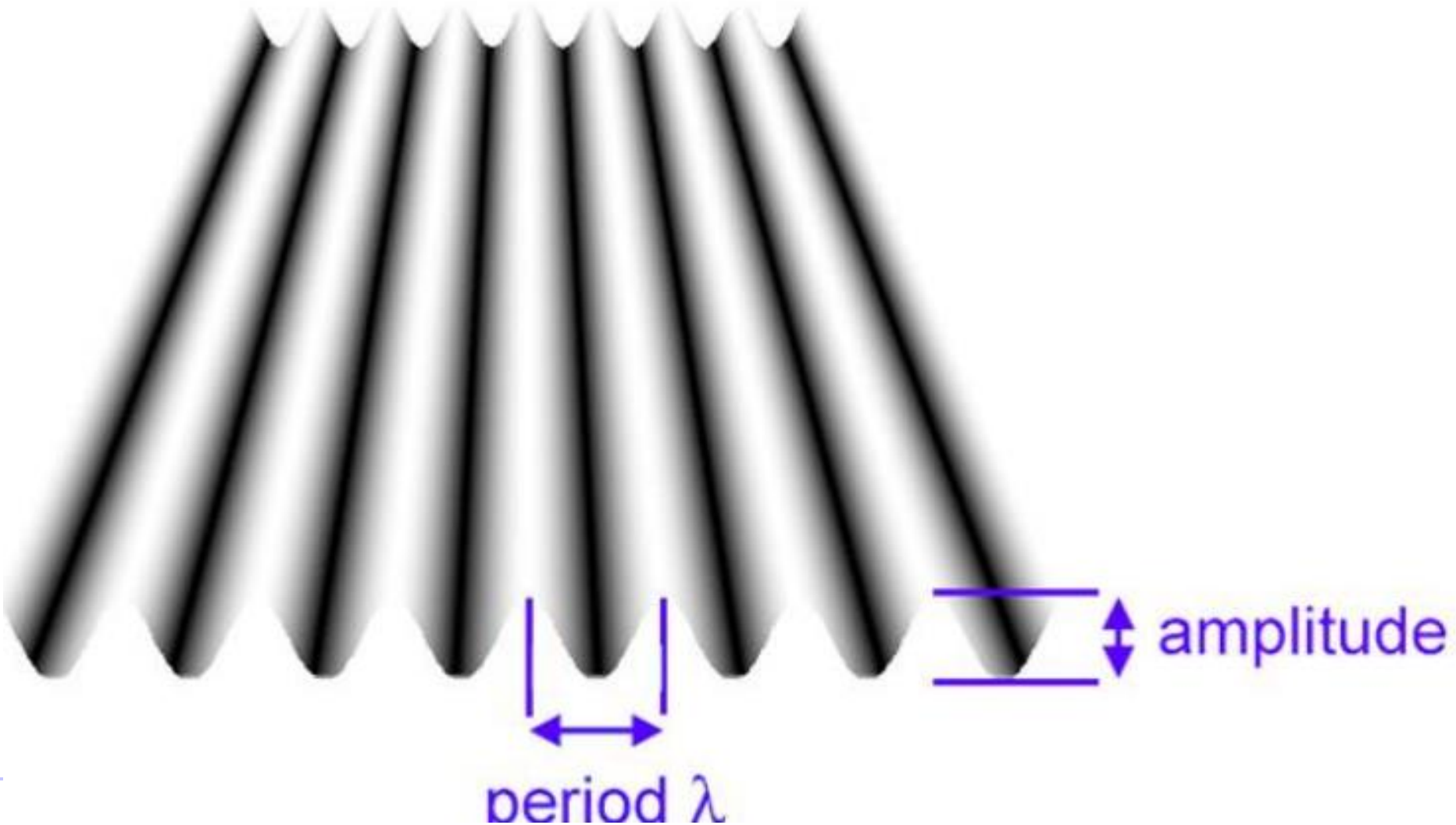
75

הגדרת טרנספורם פורייה דו ממד



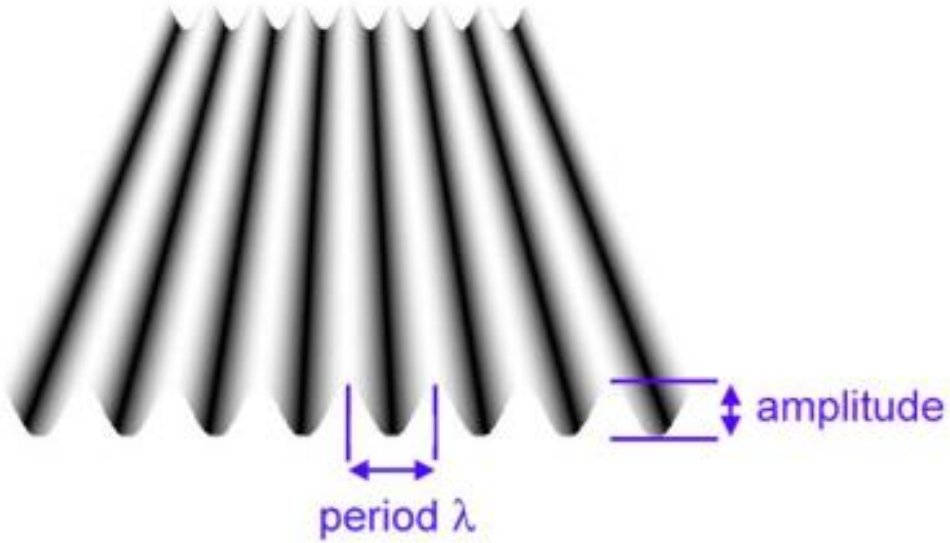
The transform coefficients are arranged in a 2D array.

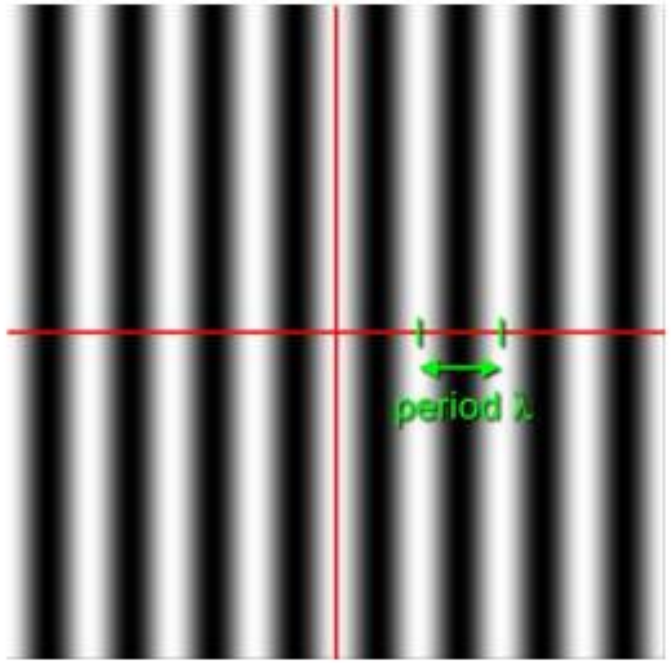
הגדרת טרנספורם פורייה דו ממד



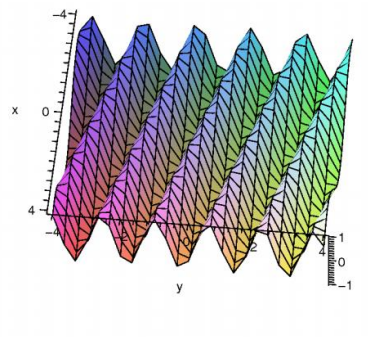
הגדרת טרנספורם פורייה דו ממד

ניתן לדמיין סינוסואידים דו-ממדיים כגלים מישוריים
עם המשרעת המוצגת כעוצמה (רמת אפור).



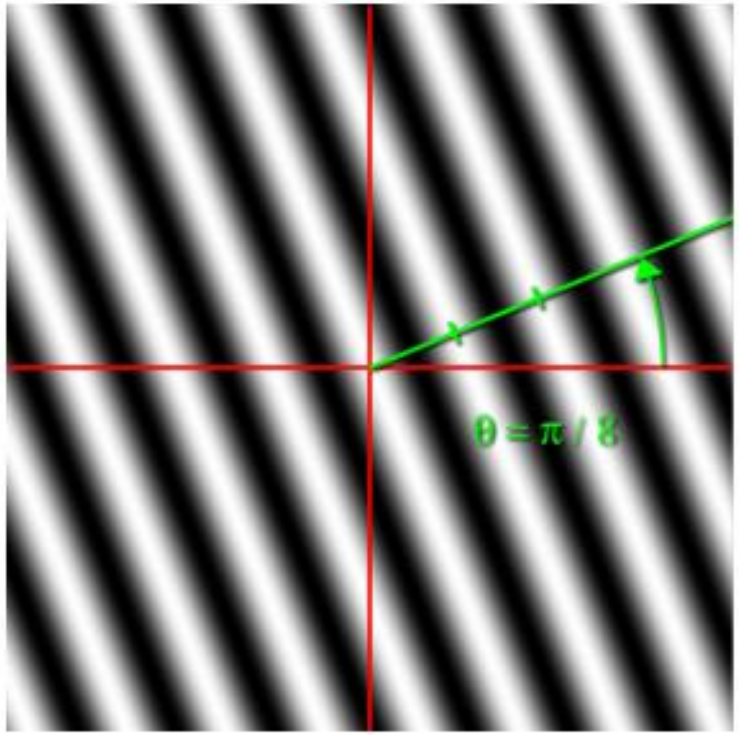


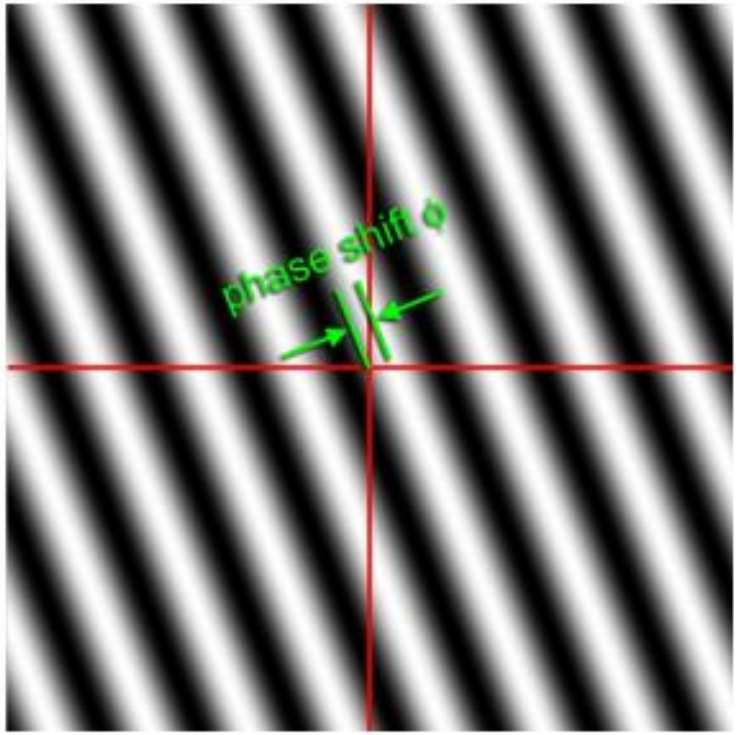
הגדרת טרנספורם פורייה דו ממד



ניתן לדמיין סינוסואידים דו-ממדיים כגלים מישוריים עם המשרעת המוצגת כעוצמה (רמת אפור).

אפשר גם לסובב.... (פאזה)





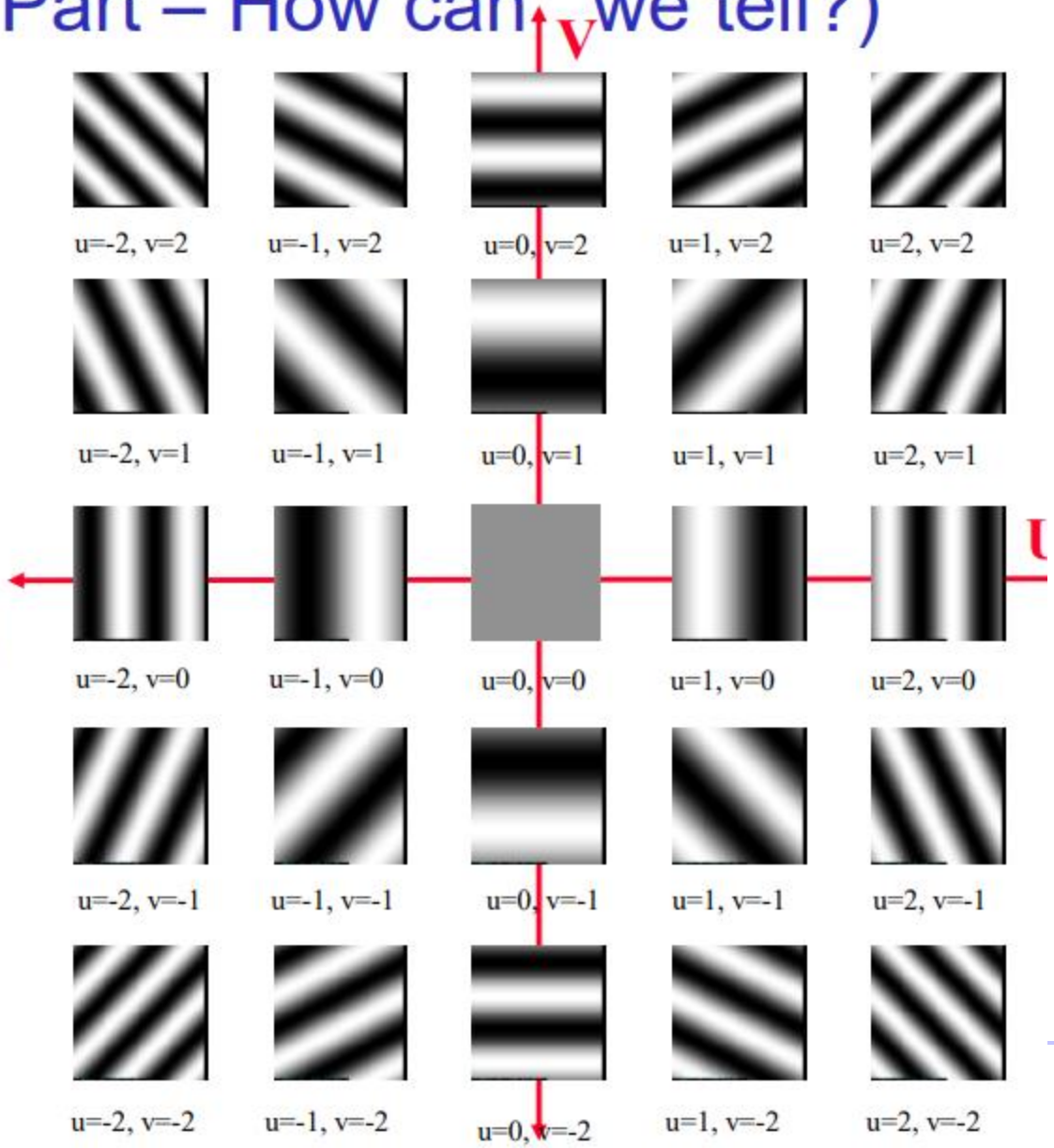
2D Fourier Basis Functions

(Imaginary Part – How can we tell?)

$$e^{\frac{2\pi i(ux+vy)}{N}}$$

Every basis function with frequency *(u,v)* is multiplied by *F(u,v)* specifying

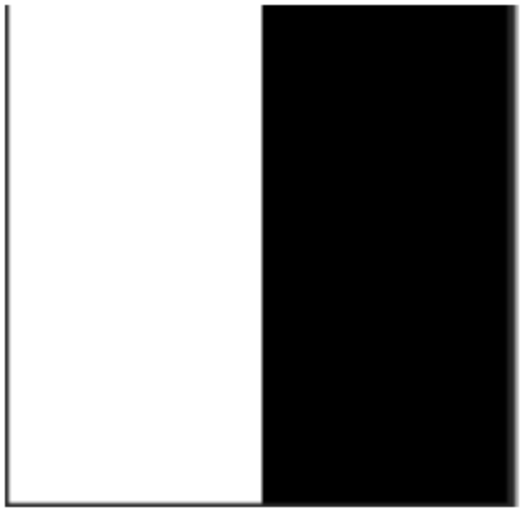
- Phase (Shift)
- Amplitude

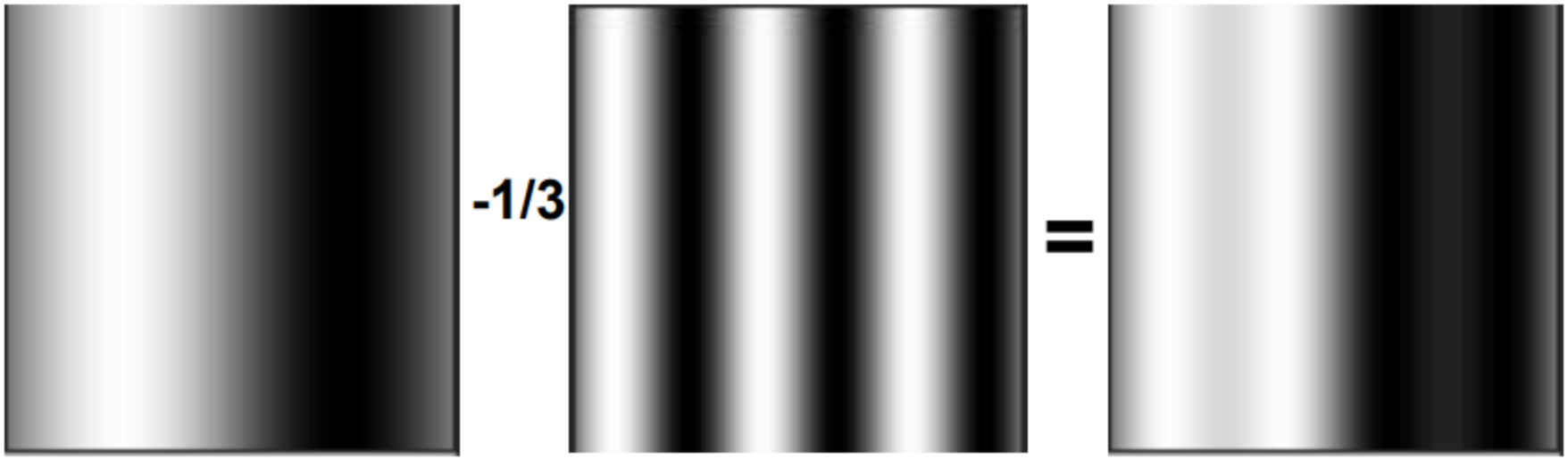


עוד מעט נפתח מעט יותר \ נראה איך הגענו לבסיס הזה...

לפני זה נראה אילוסטרציה לשימוש....

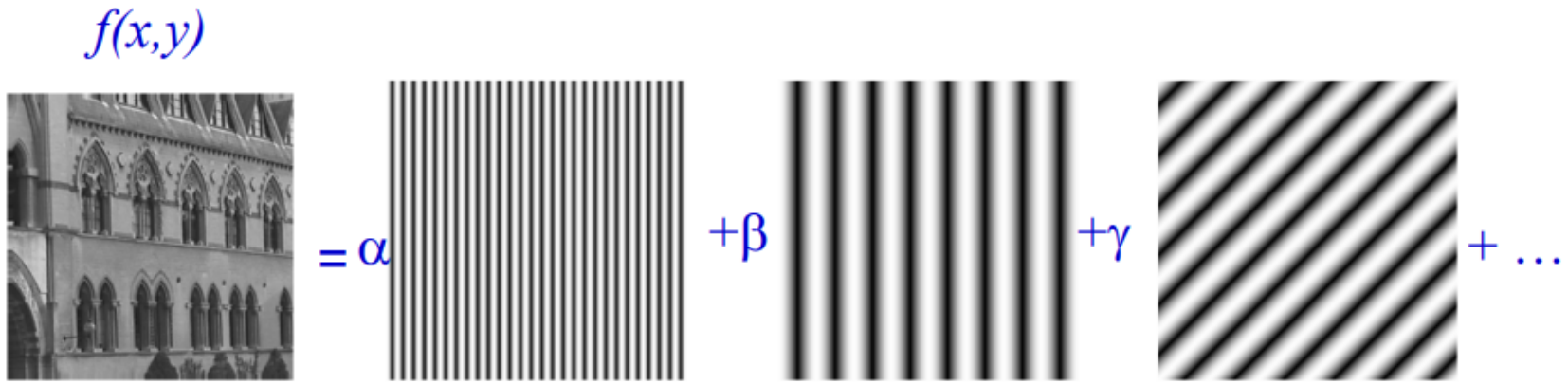
2D Pictorial Example





הגדרת טרנספורם פורייה דו ממד

is decomposed into a weighted sum of 2D orthogonal basis functions in a similar manner to decomposing a vector onto a basis using scalar products.



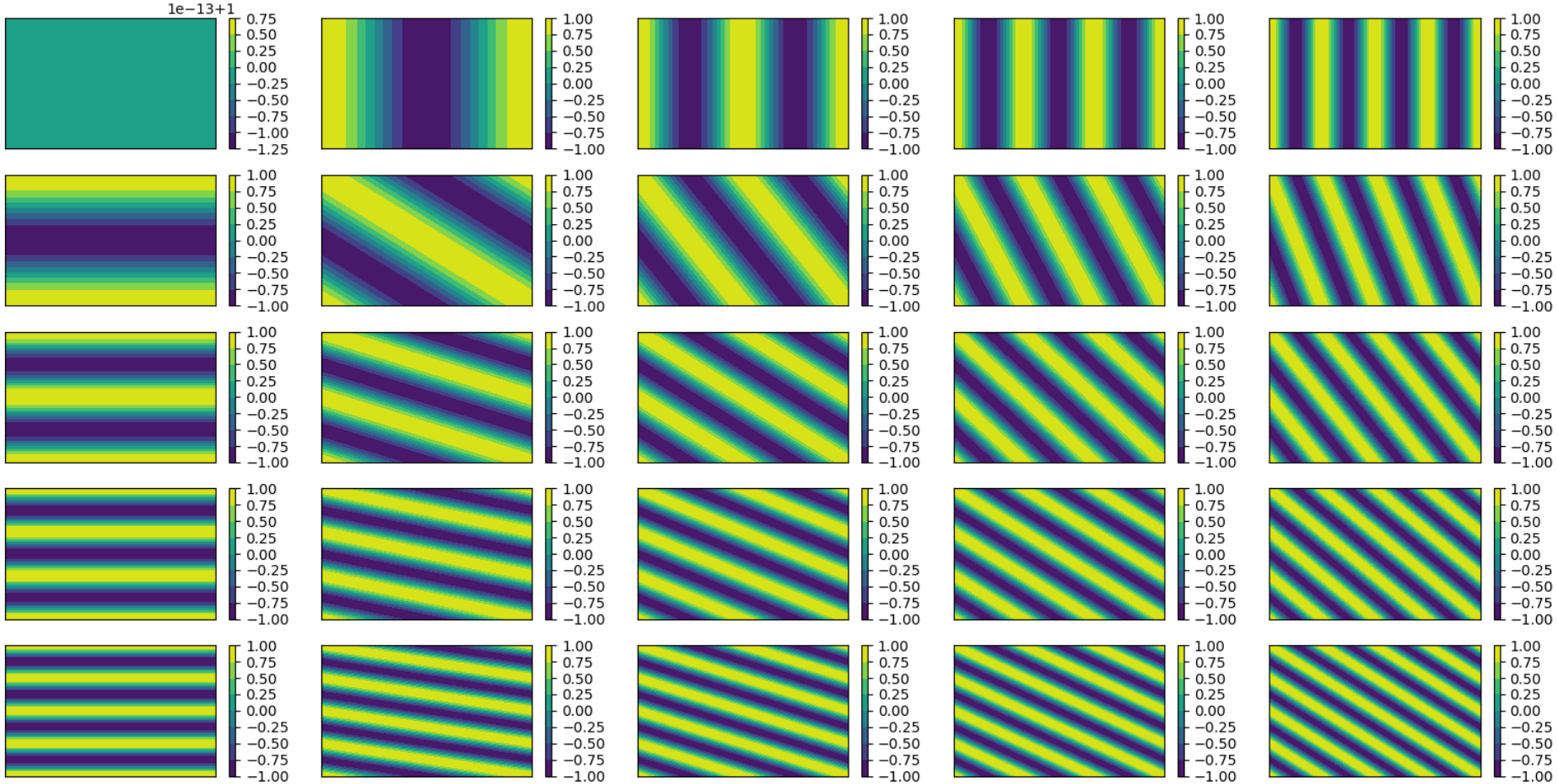
https://www.youtube.com/watch?v=D9ziTuJ3OCw

בסיס פוריה דו-ממדי

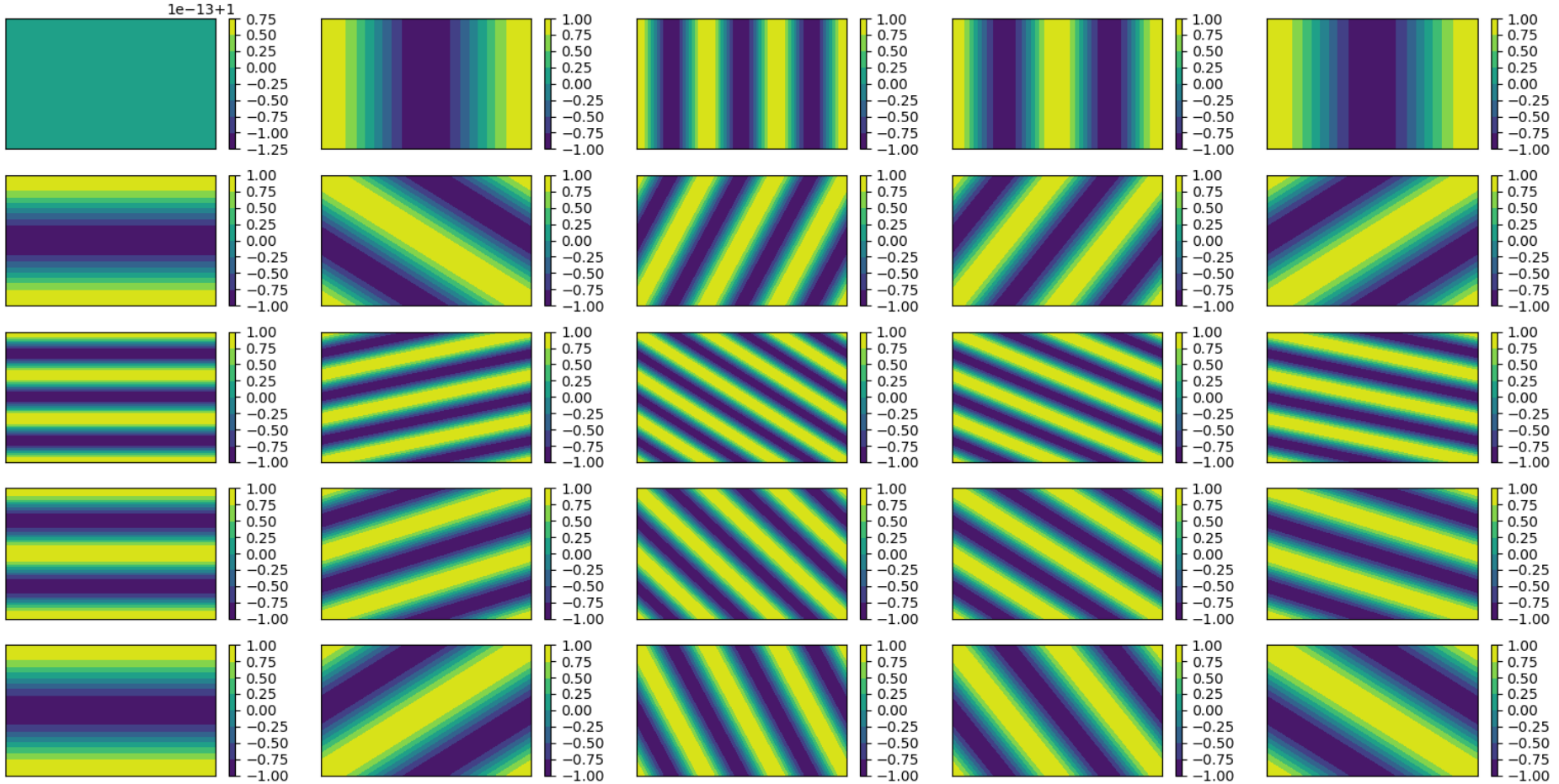
נפתח את הבסיס...

```
N = 5; M = 5;
fig, axes = plt.subplots(N, M, figsize=(N+1, M+1),
                         subplot_kw={'xticks': [], 'yticks': []})

for ii in range(N):
    for jj in range(M):

        cax = axes[jj][ii]
        fftBaseFunctions(N,M,ii,jj , cax )

plt.show()
```

```
def fftBaseFunctions(N, M, u, v, cax):
    x = np.linspace(0, N, 100*N)
    y = np.linspace(0, M, 100*M)
    X, Y = np.meshgrid(x, y)

#     if u >= (N - 1) / 2:
#         u = u - N;
#     if v >= (M - 1) / 2:
#         v = v - M;

    z = np.real(np.exp(2 * np.pi * 1j * (u * X / N + v * Y / M)));
    cf = cax.contourf(x, y , z)
    fig.colorbar(cf, ax=cax)
```


1e−13+1
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

```
def fftBaseFunctions(N, M, u, v, cax):
    x = np.linspace(0, N, 100*N)
    y = np.linspace(0, M, 100*M)
    X, Y = np.meshgrid(x, y)

    if u >= (N - 1) / 2:
        u = u - N;
    if v >= (M - 1) / 2:
        v = v - M;
    z = np.real(np.exp(2 * np.pi * 1j * (u * X / N + v * Y / M)));
    cf = cax.contourf(x, y , z)
    fig.colorbar(cf, ax=cax)
```


1e−13+1
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

fftShift (1D)

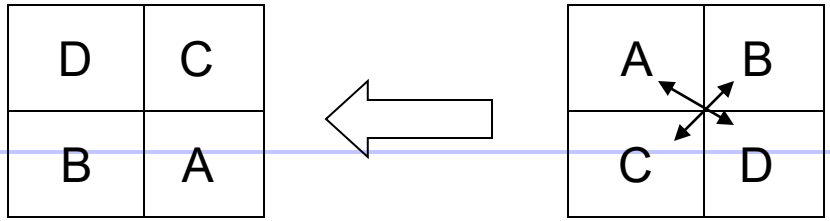
```
Xodd = [0 1 2 3 4 ];
fftshift(Xodd)
```

2 1 0 4 3

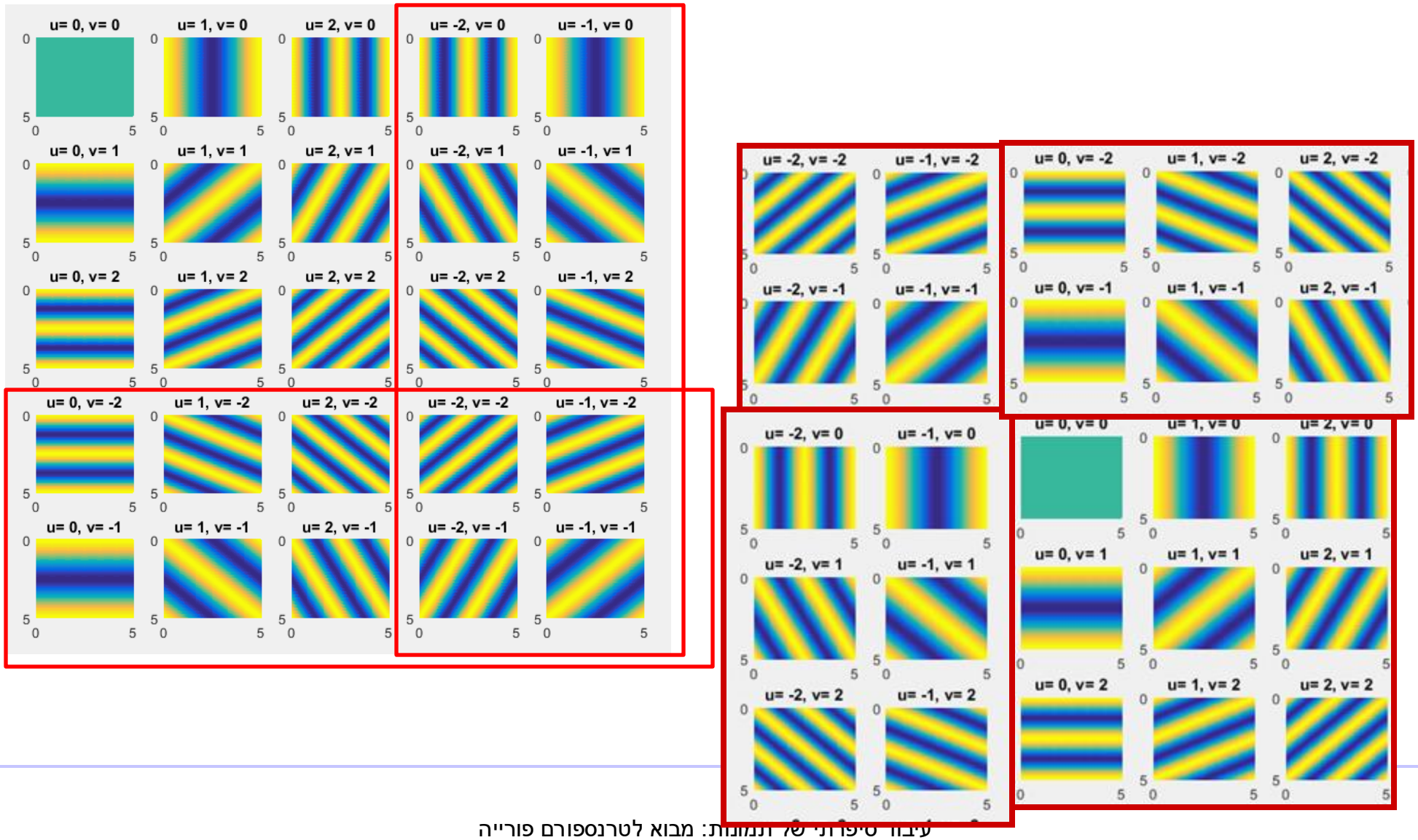
fftShift (2D)

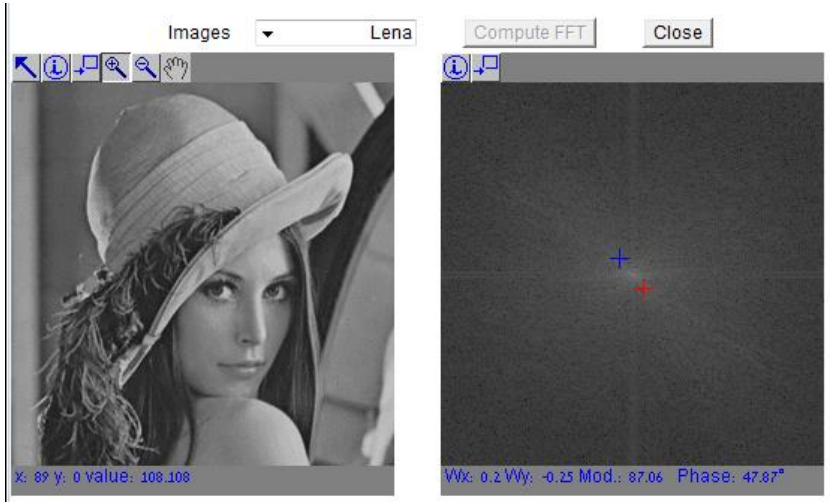
```
Xodd = [[11 12 13 14 15 ]; ...
        [21 22 23 24 25 ]; ...
        [31 32 33 34 35 ]; ...
        [41 42 43 44 45 ]; ...
        [51 52 53 54 55 ]];
fftshift(Xodd)
```

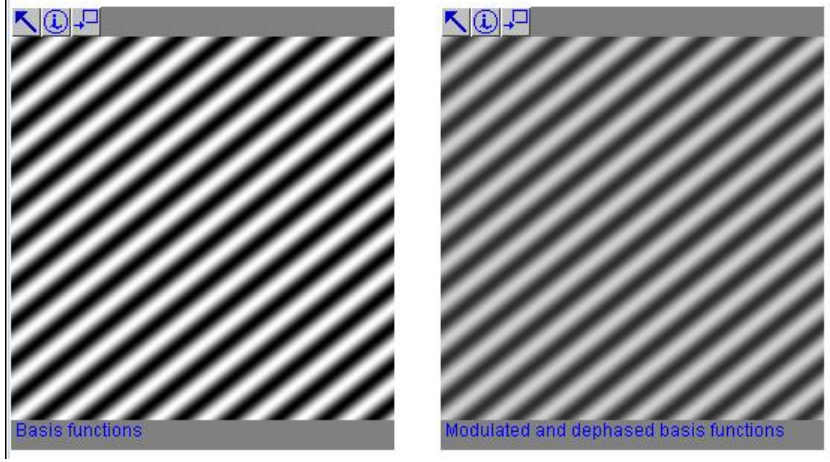
```
ans =

    44    45    41    42    43
    54    55    51    52    53
    14    15    11    12    13
    24    25    21    22    23
    34    35    31    32    33
```

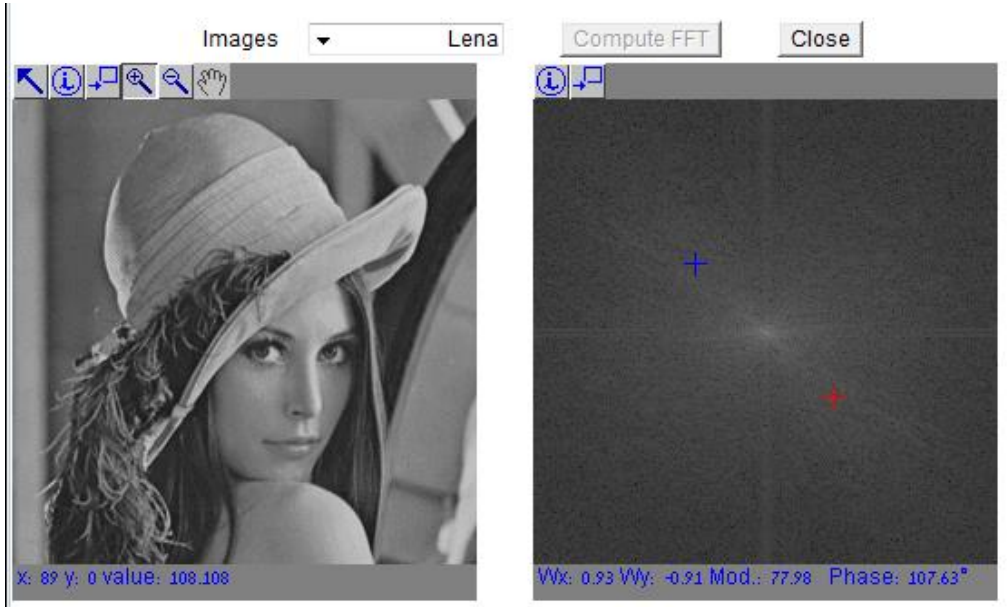


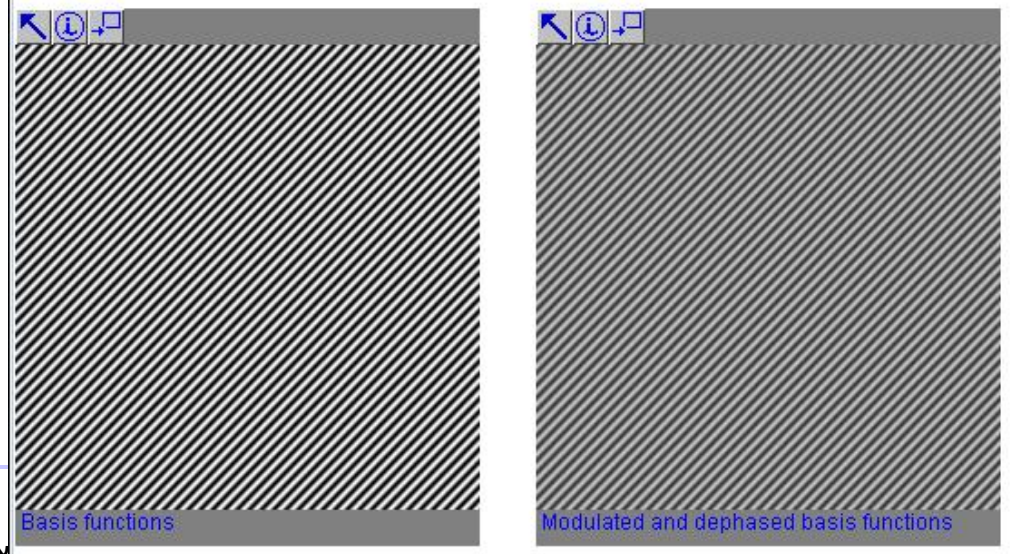
בסיס פורייה אחרי fftshift



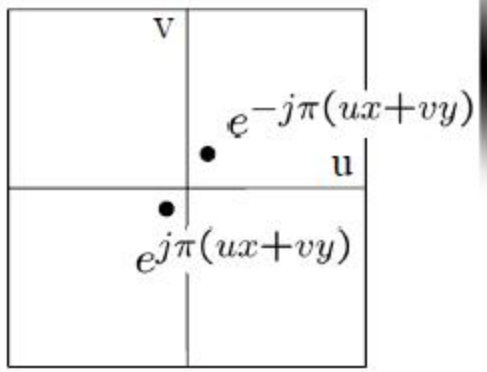

Images
Lena
Compute FFT
Close
x: 89 y: 0 value: 108.108
Wx: 0.2 Wy: -0.25 Mod.: 87.06 Phase: 47.87°

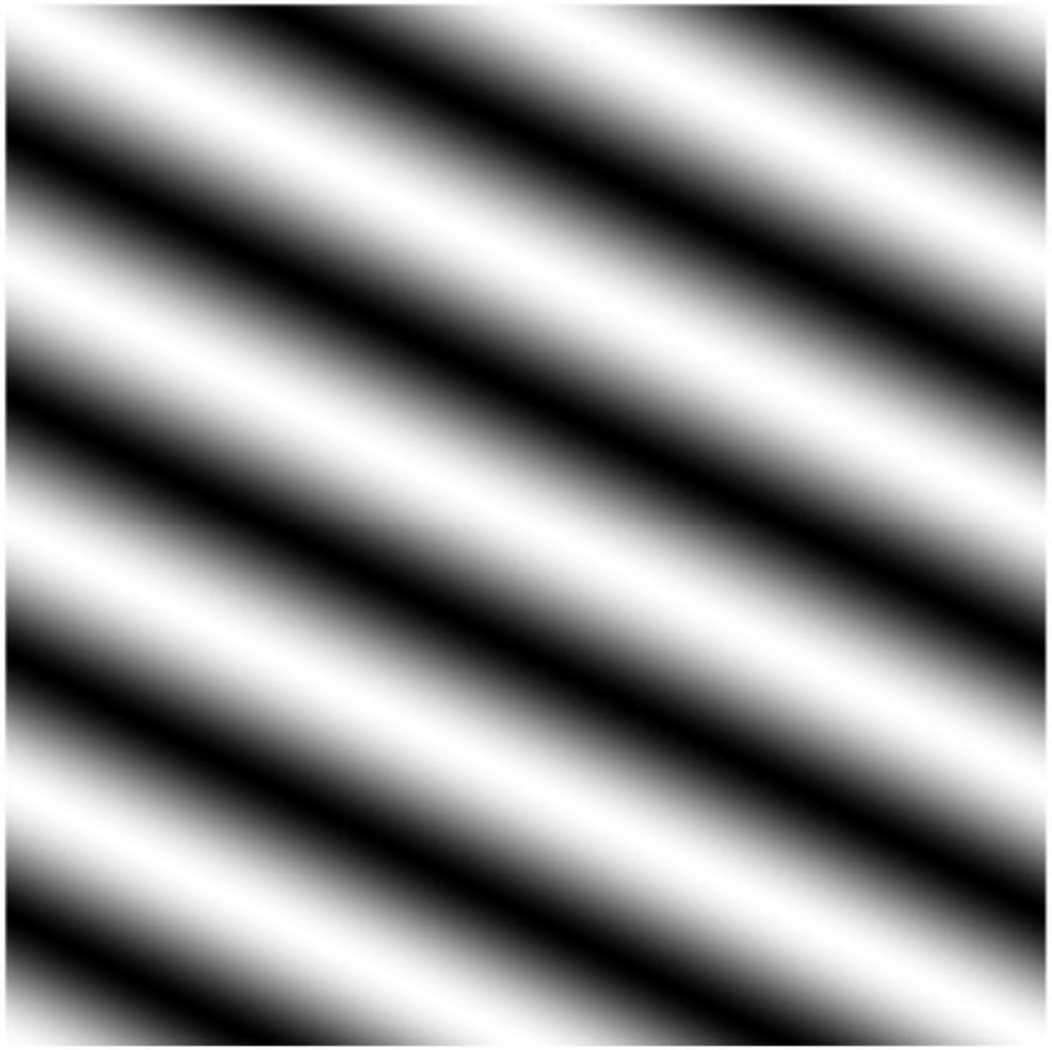

Basis functions
Modulated and dephased basis functions


Images
Lena
Compute FFT
Close
x: 89 y: 0 value: 108.108
Wx: 0.93 Wy: -0.91 Mod.: 77.98 Phase: 107.63°


Basis functions
Modulated and dephased basis functions

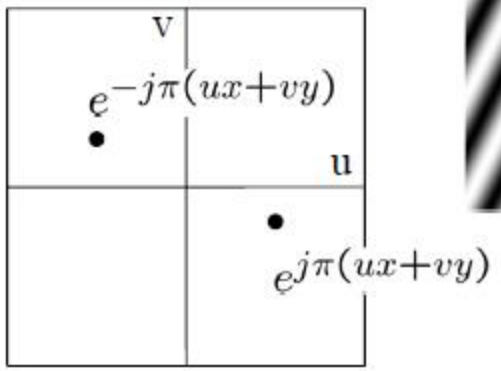
To get some sense of what basis elements look like, we plot a basis element --- or rather, its real part --- as a function of x,y for some fixed u, v. We get a function that is constant when (ux+vy) is constant. The magnitude of the vector (u, v) gives a frequency, and its direction gives an orientation. The function is a sinusoid with this frequency along the direction, and constant perpendicular to the direction.



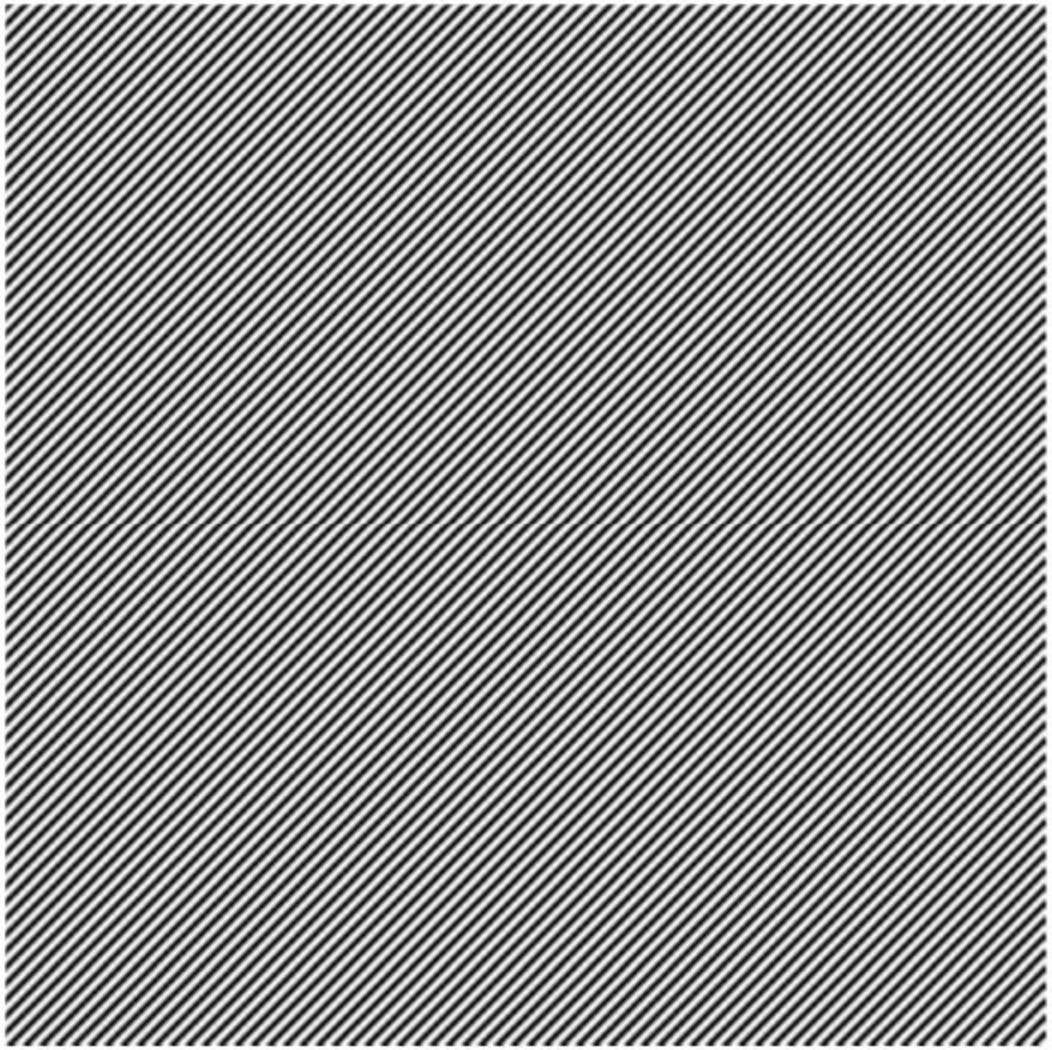


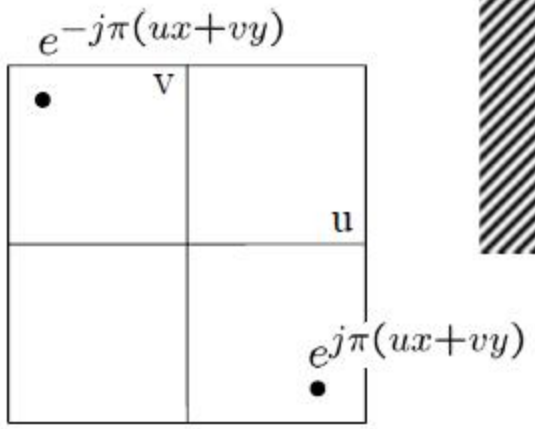
slide: B. Freeman

Here u and v are larger than in the previous slide.



And larger still...

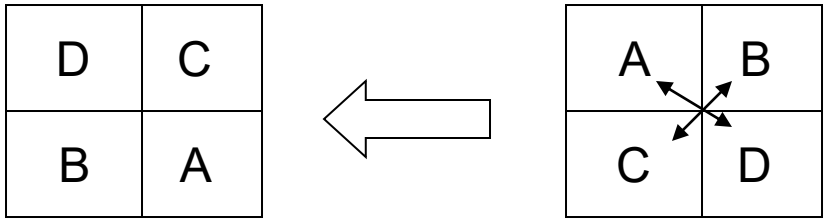




Power spectrum of an image

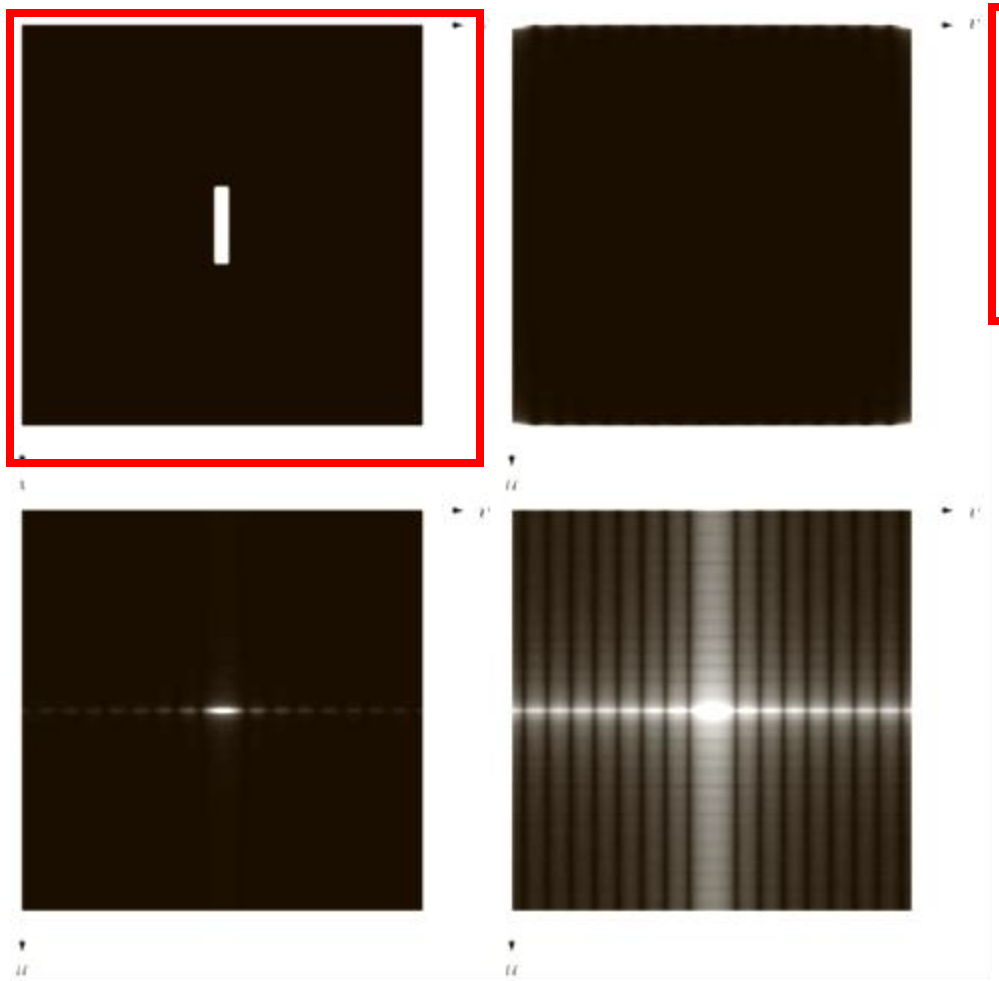
הצגת מקדמי פורייה כתמונה

- מרכוז מטריצת המקדמים:



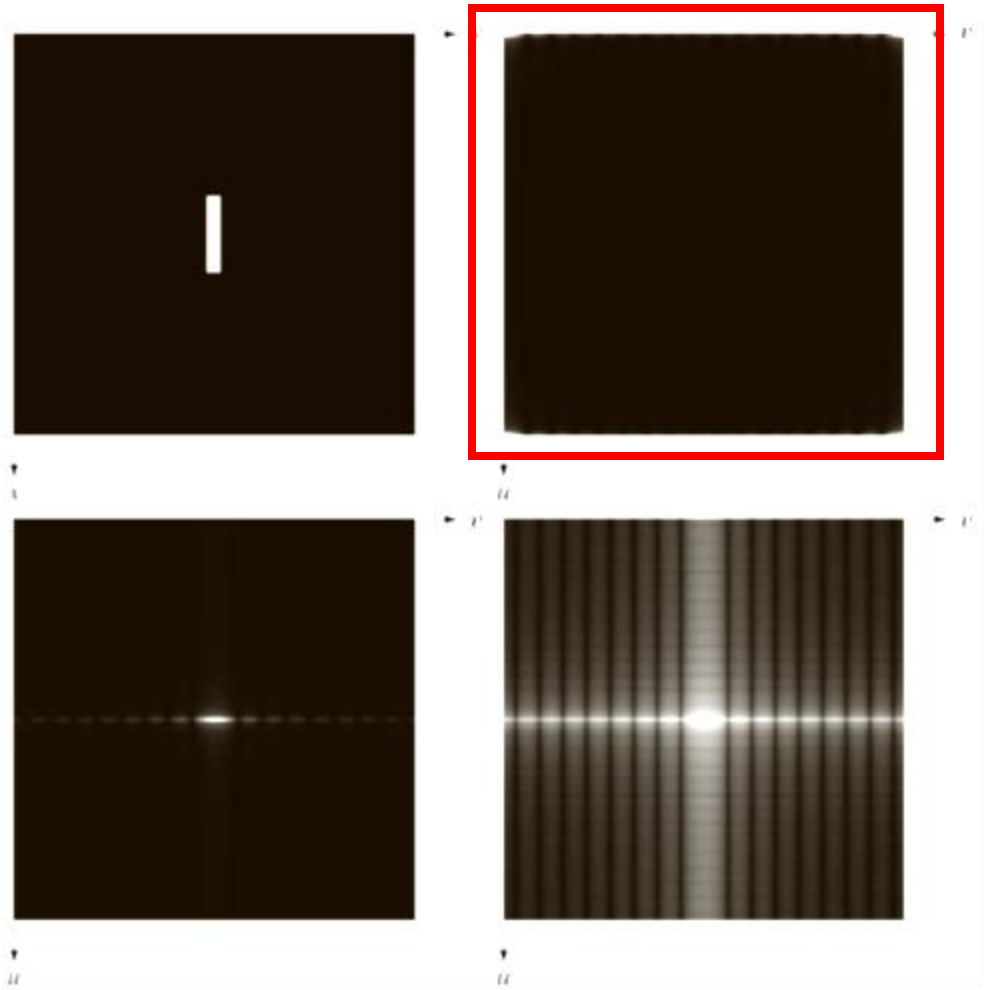
- התגברות על טווח דינאמי רחב: $\log(1+|F(u,v)|)$
- התאמה לתחום דרגות האפור

הצגת מקדמי פורייה כתמונה



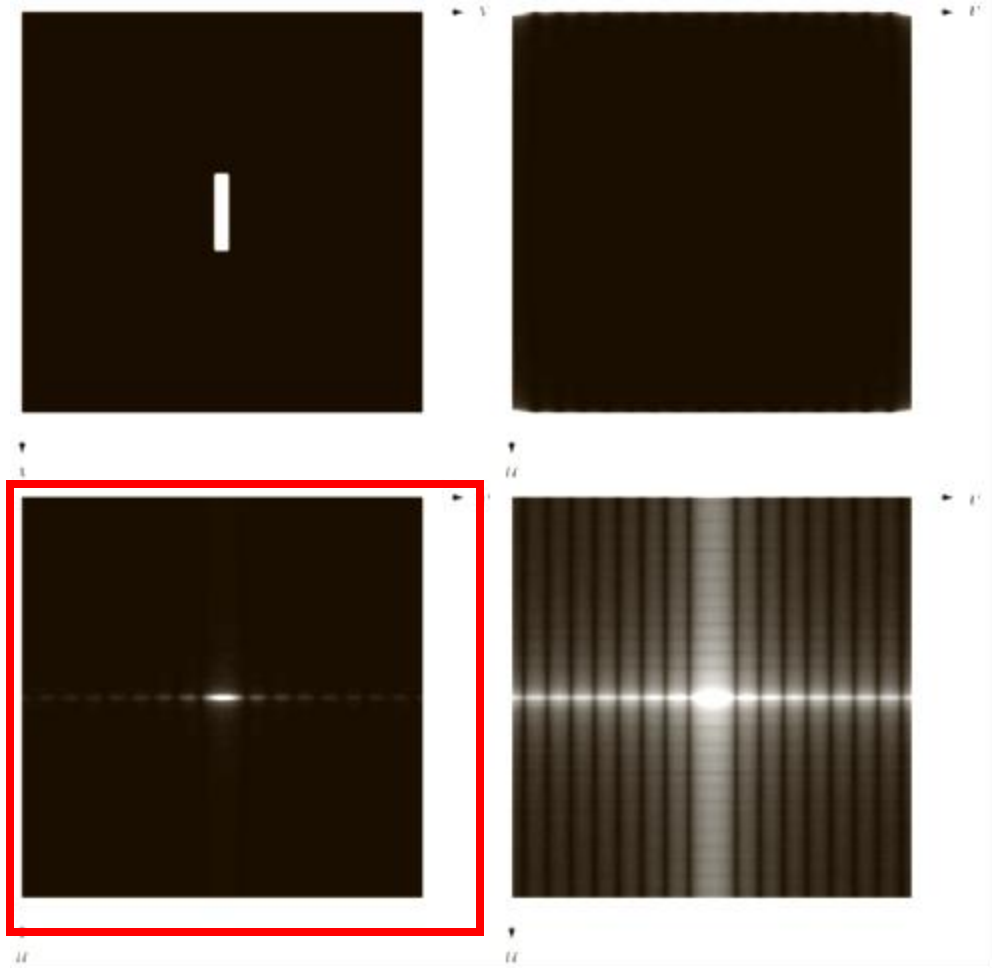
```
# Read the image
f = imread('FFT_SQUARE_IMAGE.tiff', as_gray=Tr
as_gray=True for grayscale

# Display the original image
plt.figure()
plt.title("Original Image")
plt.imshow(f, cmap='gray')
plt.axis('off')

# Perform FFT and display its magnitude spectr
F = np.fft.fft2(f)
S = np.abs(F)

plt.figure()
plt.title("Magnitude Spectrum (FFT)")
plt.imshow(S, cmap='gray')
plt.axis('off')

# Apply fftshift and display shifted spectrum
Fc = np.fft.fftshift(F)
S_shifted = np.abs(Fc)

plt.figure()
plt.title("Shifted Magnitude Spectrum")
plt.imshow(S_shifted, cmap='gray')
```

הצגת מקדמי פורייה כתמונה



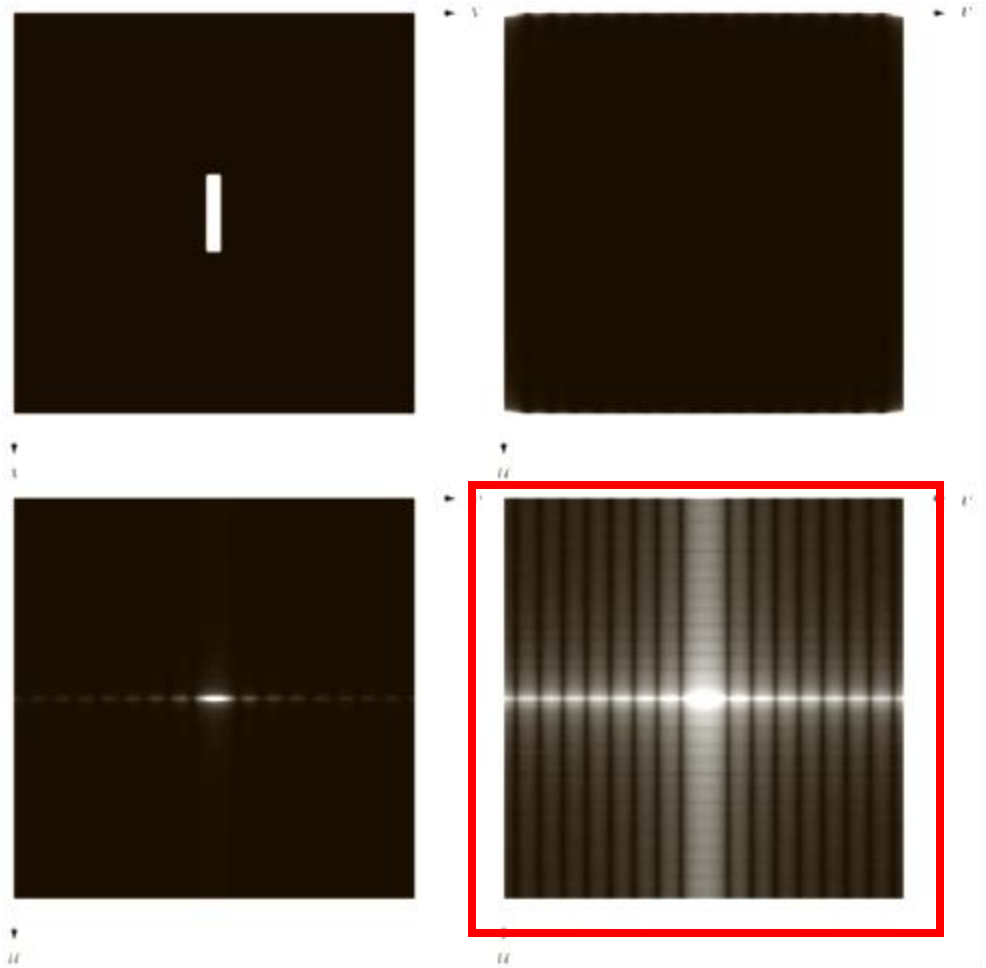
```
# Read the image
f = imread('FFT_SQUARE_IMAGE.tiff', as_gray=Tr
as_gray=True for grayscale

# Display the original image
plt.figure()
plt.title("Original Image")
plt.imshow(f, cmap='gray')
plt.axis('off')

# Perform FFT and display its magnitude spectr
F = np.fft.fft2(f)
S = np.abs(F)

plt.figure()
plt.title("Magnitude Spectrum (FFT)")
plt.imshow(S, cmap='gray')
plt.axis('off')

# Apply fftshift and display shifted spectrum
Fc = np.fft.fftshift(F)
S_shifted = np.abs(Fc)

plt.figure()
plt.title("Shifted Magnitude Spectrum")
plt.imshow(S_shifted, cmap='gray')
```

הצגת מקדמי פורייה כתמונה



```
# Read the image
f = imread('FFT_SQUARE_IMAGE.tiff', as_gray=Tr
as_gray=True for grayscale

# Display the original image
plt.figure()
plt.title("Original Image")
plt.imshow(f, cmap='gray')
plt.axis('off')

# Perform FFT and display its magnitude spectr
F = np.fft.fft2(f)
S = np.abs(F)

plt.figure()
plt.title("Magnitude Spectrum (FFT)")
plt.imshow(S, cmap='gray')
plt.axis('off')

# Apply fftshift and display shifted spectrum
Fc = np.fft.fftshift(F)
S_shifted = np.abs(Fc)

plt.figure()
plt.title("Shifted Magnitude Spectrum")
plt.imshow(S_shifted, cmap='gray')
```

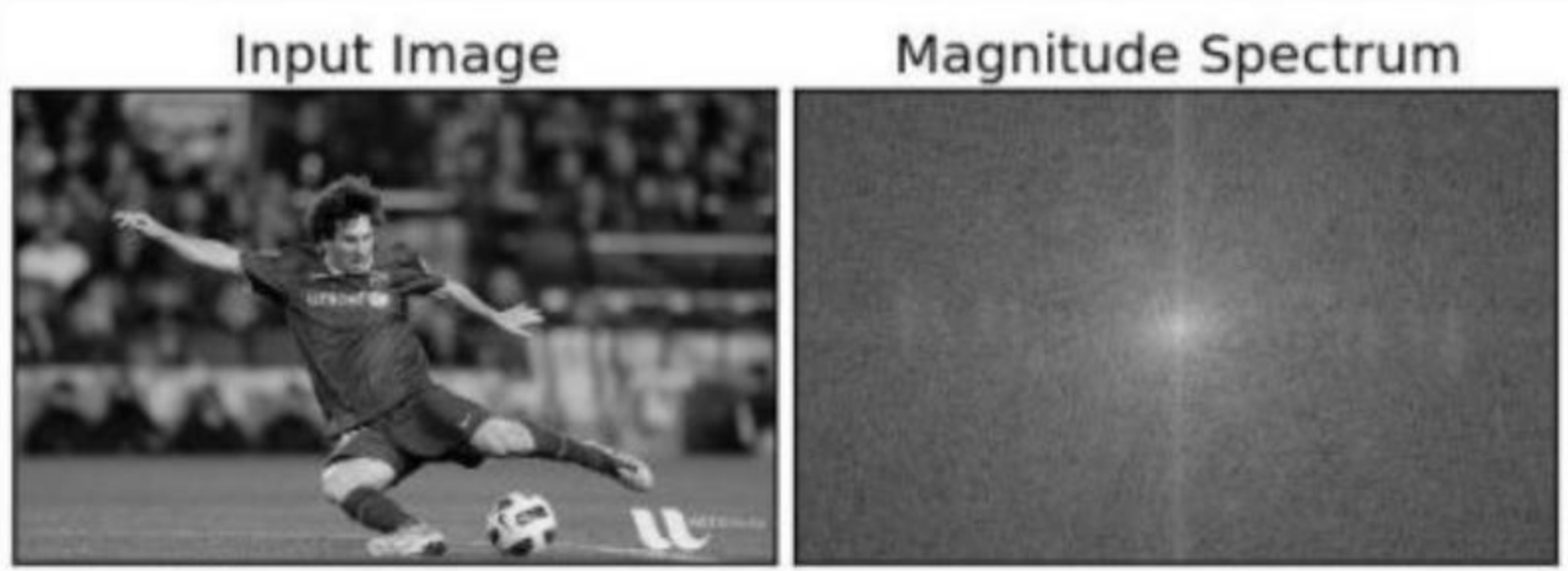
הצגת מקדמי פורייה כתמונה



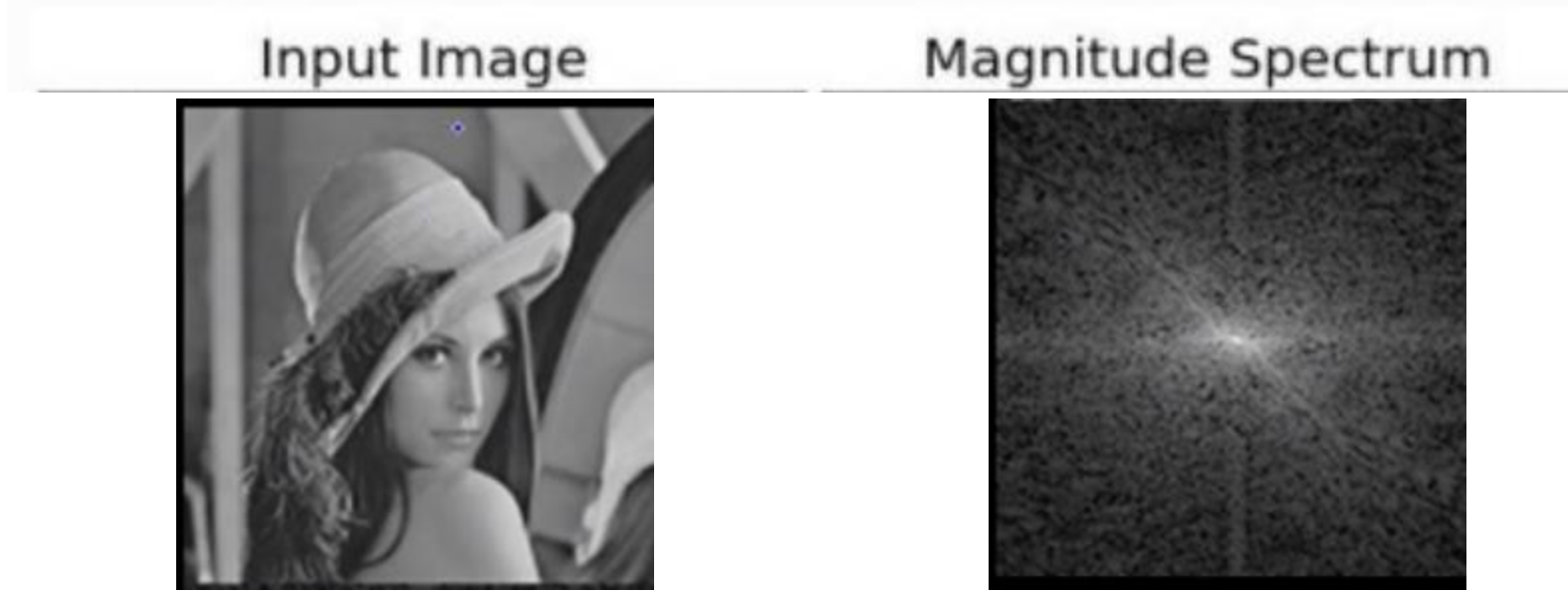
```
# Perform FFT and display its magnitude spectr
F = np.fft.fft2(f)
S = np.abs(F)

plt.figure()
plt.title("Magnitude Spectrum (FFT)")
plt.imshow(S, cmap='gray')
plt.axis('off')

# Apply fftshift and display shifted spectrum
Fc = np.fft.fftshift(F)
S_shifted = np.abs(Fc)

plt.figure()
plt.title("Shifted Magnitude Spectrum")
plt.imshow(S_shifted, cmap='gray')
plt.axis('off')

# Log-transformed magnitude spectrum
S_log = np.log(1 + S_shifted)

plt.figure()
plt.title("Log-transformed Spectrum")
plt.imshow(S_log, cmap='gray')
plt.axis('off')

plt.show()
```

עכשיו נראה בPython

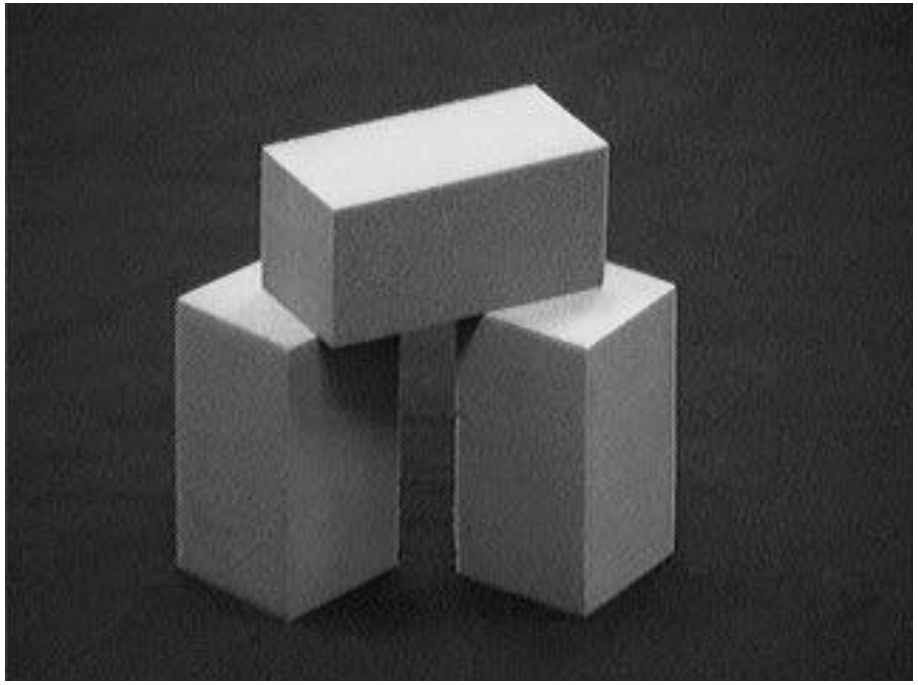
```
import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('messi5.jpg',0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
magnitude_spectrum = 20*np.log(np.abs(fshift))

plt.subplot(121),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(magnitude_spectrum, cmap = 'gray')
plt.title('Magnitude Spectrum'), plt.xticks([]), plt.yticks([])
plt.show()
```

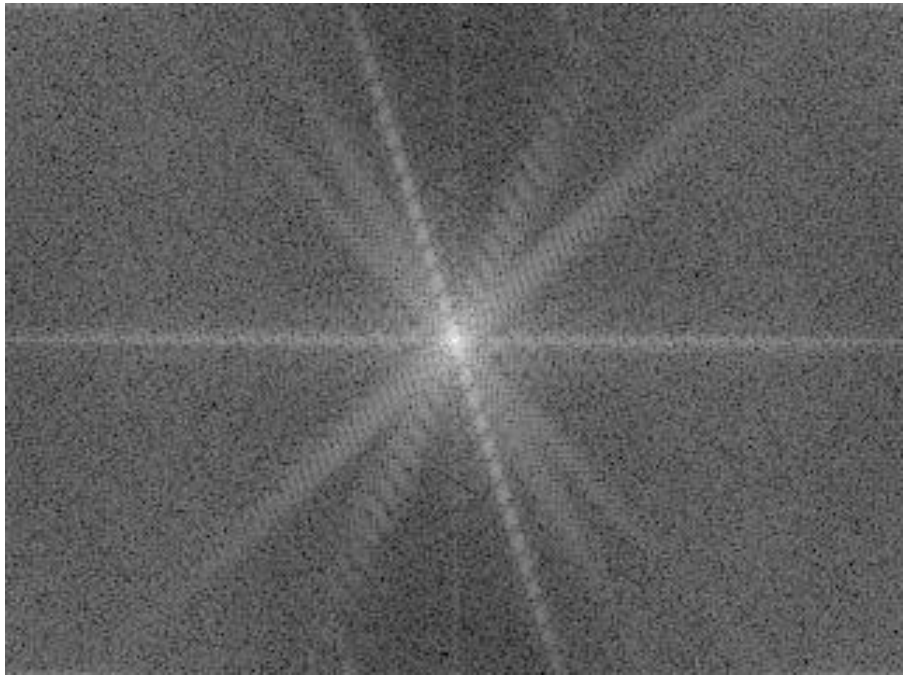


```
import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('messi5.jpg',0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
magnitude_spectrum = 20*np.log(np.abs(fshift))

plt.subplot(121),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(magnitude_spectrum, cmap = 'gray')
plt.title('Magnitude Spectrum'), plt.xticks([]), plt.yticks([])
plt.show()
```



תכונות טרנספורם פורייה –
תכונת סימטריות העוצמה





תכונות טרנספורם פורייה

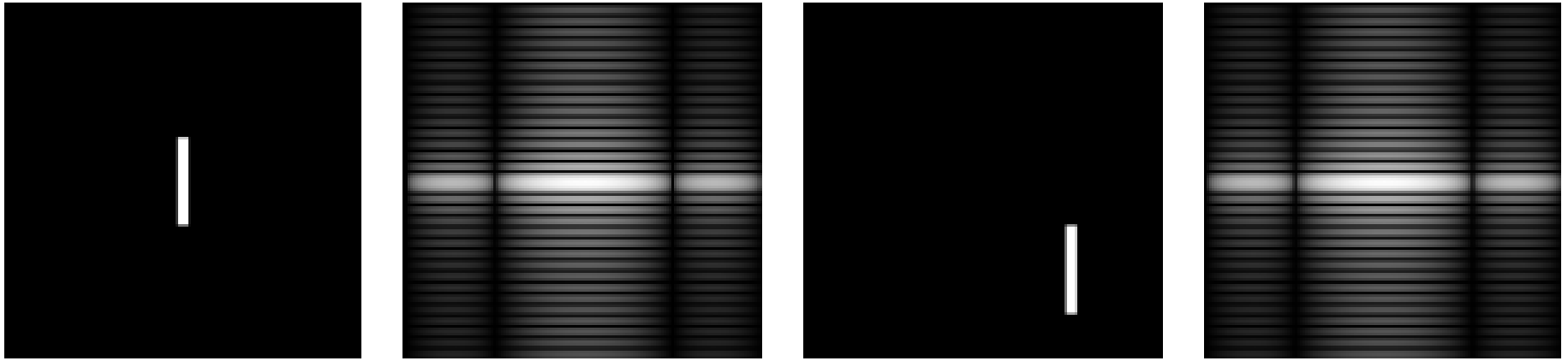
תכונות טרנספורם פורייה

Name	DFT Pairs
1) Symmetry properties	See Table 4.1
2) Linearity	$af_1(x, y) + bf_2(x, y) \Leftrightarrow aF_1(u, v) + bF_2(u, v)$
3) Translation (general)	$f(x, y)\, e^{j2\pi(u_0 x/M + v_0 y/N)} \Leftrightarrow F(u - u_0, v - v_0)$ $f(x - x_0, y - y_0) \Leftrightarrow F(u, v) e^{-j2\pi(u x_0/M + v y_0/N)}$
4) Translation to center of the frequency rectangle, $(M/2, N/2)$	$f(x, y)(-1)^{x+y} \Leftrightarrow F(u - M/2, v - N/2)$ $f(x - M/2, y - N/2) \Leftrightarrow F(u, v)(-1)^{u+v}$
5) Rotation	$f(r, \theta + \theta_0) \Leftrightarrow F(\omega, \varphi + \theta_0)$ $x = r\cos\theta \quad y = r\sin\theta \quad u = \omega\cos\varphi \quad v = \omega\sin\varphi$
6) Convolution theorem†	$f(x, y) \star h(x, y) \Leftrightarrow F(u, v)H(u, v)$ $f(x, y)h(x, y) \Leftrightarrow F(u, v) \star H(u, v)$

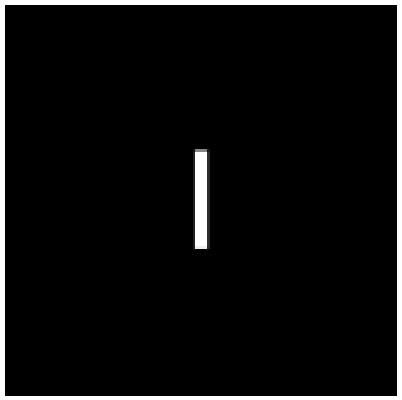
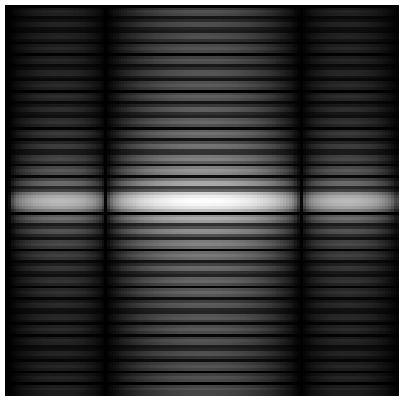
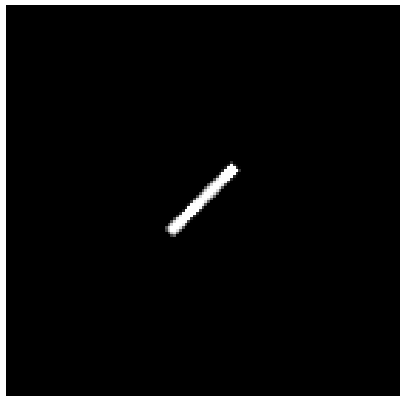
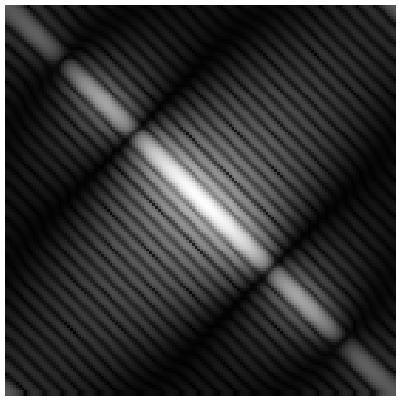
(Continued)

Property	Spatial Domain	Frequency Domain
Linearity	$\alpha f_1(x) + \beta f_2(x)$	$\alpha F_1(u) + \beta F_2(u)$
Scaling	$f(ax)$	$\frac{1}{\lvert a \rvert} F\left(\frac{u}{a}\right)$
Shifting	$f(x-a)$	$e^{-i2\pi ua} F(u)$
Differentiation	$\frac{d^n}{dx^n}(f(x))$	$(i2\pi u)^n F(u)$

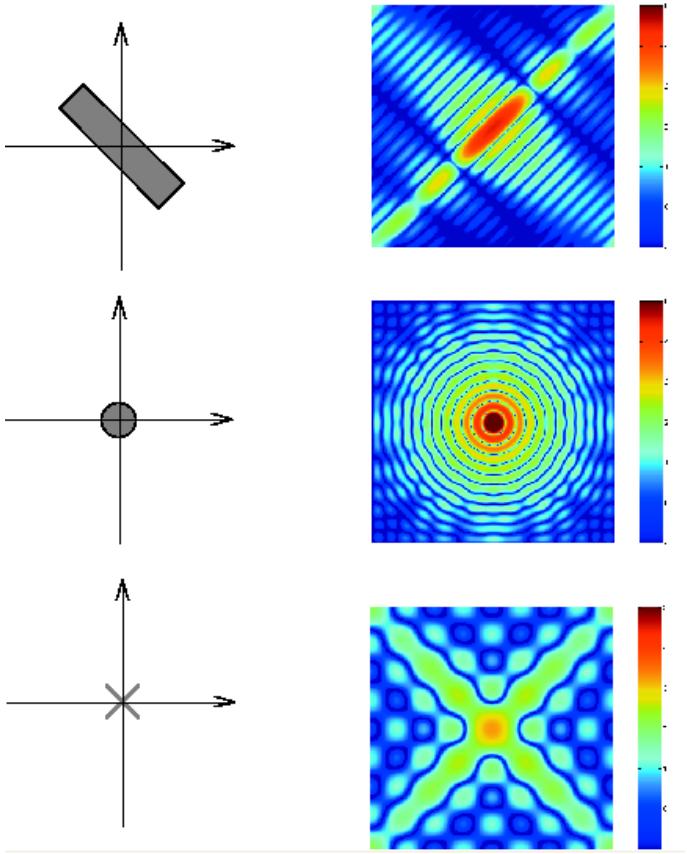
תכונות טרנספורם פורייה – תכונת ההזזה



תכונות טרנספורם פורייה – תכונת הסיבוב

דוגמאות לטרנספורם פורייה



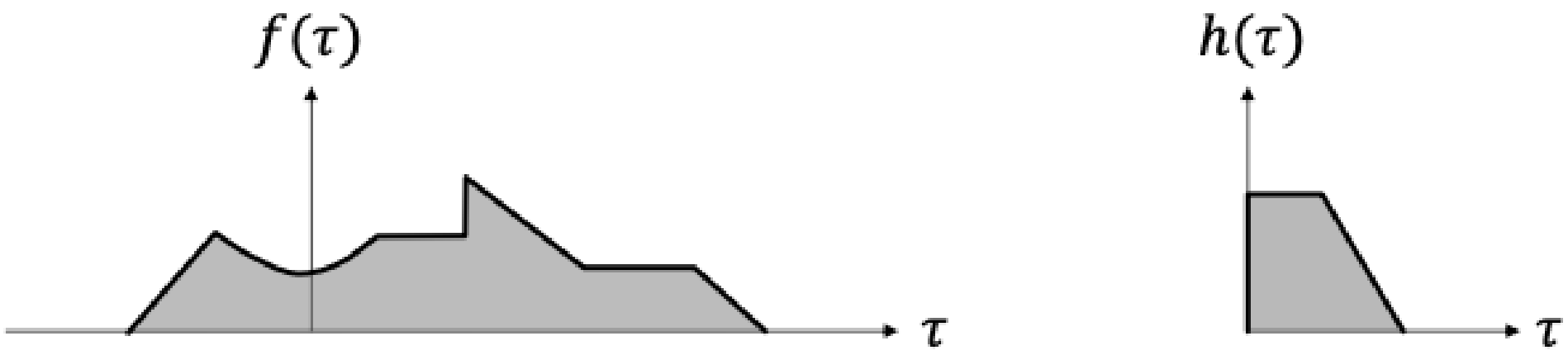
Mathworks

משפט הקונבולוציה

Convolution

Convolution of two functions $f(x)$ and $h(x)$

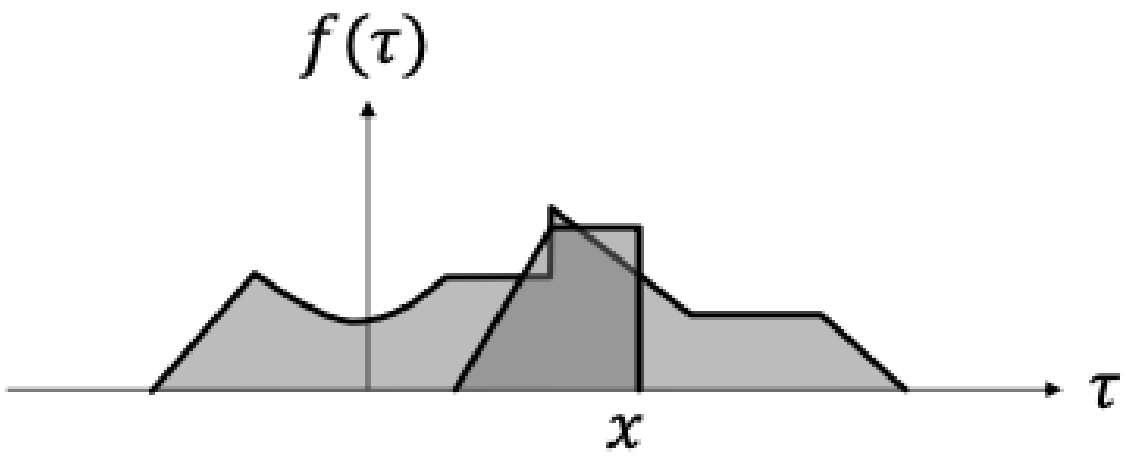
$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)\,d\tau$$

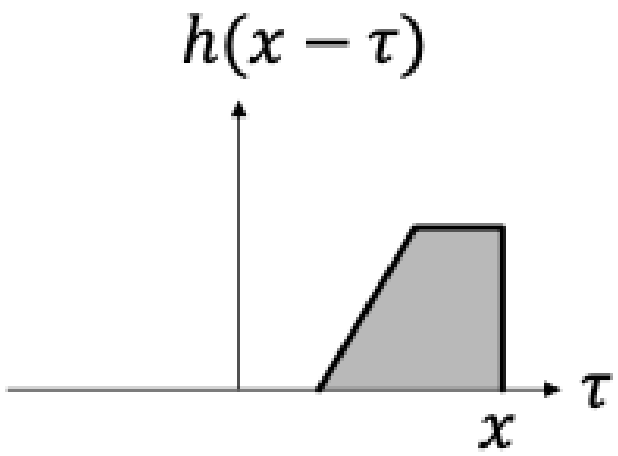


Convolution

Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)\,d\tau$$

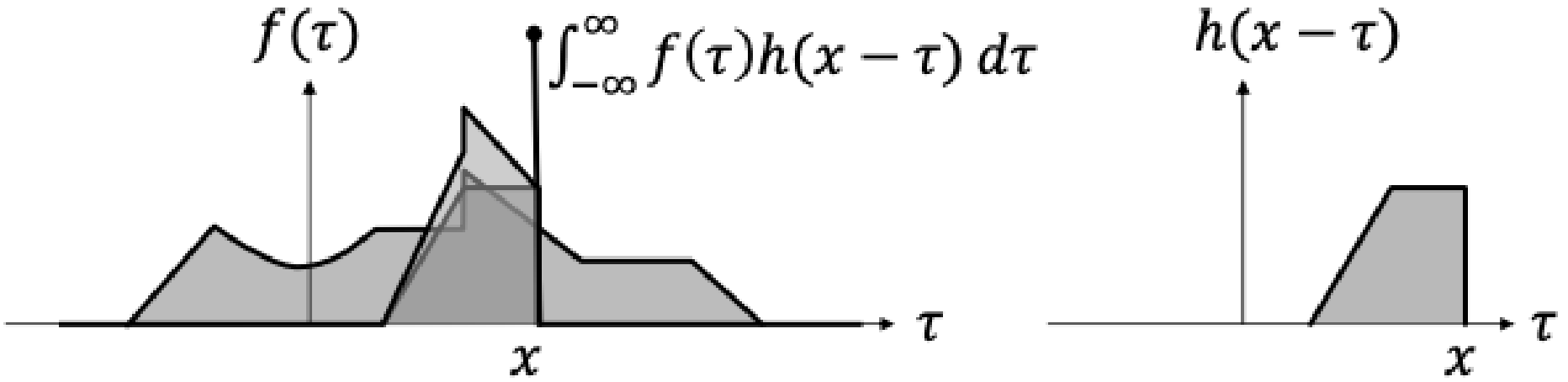




Convolution

Convolution of two functions $f(x)$ and $h(x)$

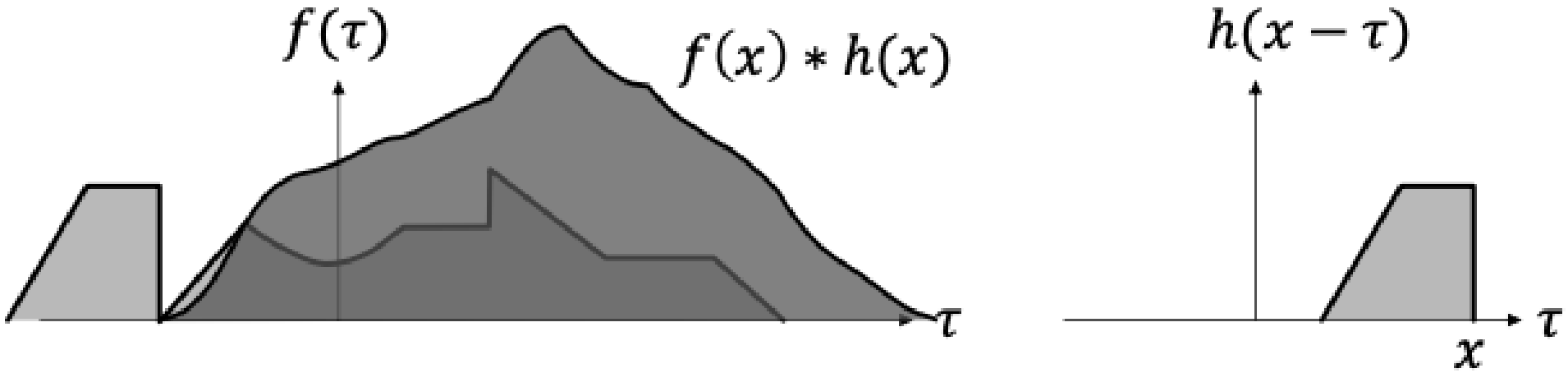
$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)\,d\tau$$



Convolution

Convolution of two functions $f(x)$ and $h(x)$

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)\,d\tau$$



Convolution and Fourier Transform

Fourier Transform of $g(x)$:

$$G(u) = \int_{-\infty}^{\infty} g(x) e^{-i2\pi ux} dx$$

$$G(u) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(\tau) h(x-\tau) e^{-i2\pi ux} d\tau dx$$

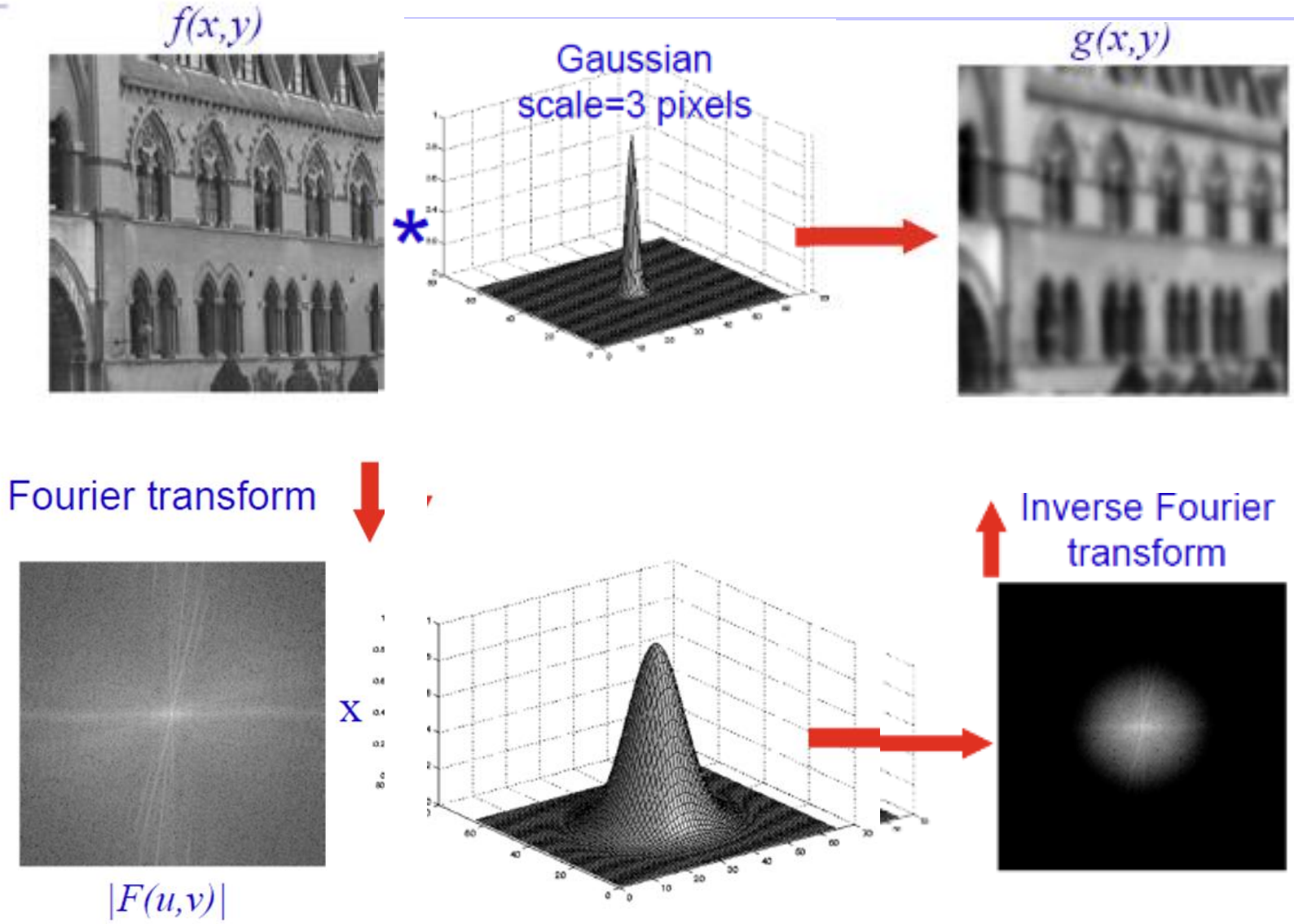
$$G(u) = \underbrace{\int_{-\infty}^{\infty} f(\tau) e^{-i2\pi u\tau} d\tau}_{F(u)} \; \underbrace{\int_{-\infty}^{\infty} h(x-\tau) e^{-i2\pi u(x-\tau)} dx}_{H(u)}$$

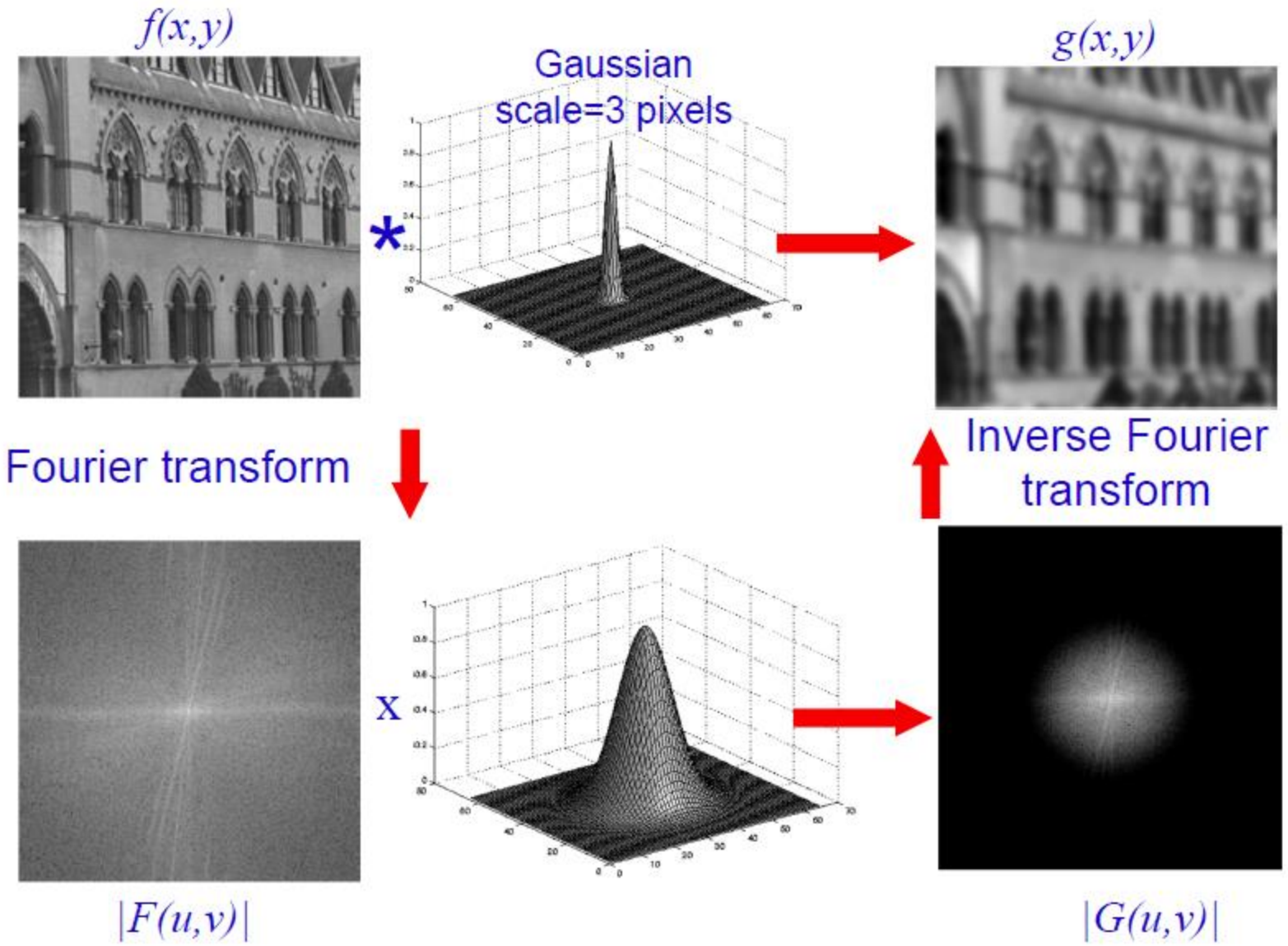
The convolution theorem

$$f(x,y) * h(x,y) \quad \Leftrightarrow \quad F(u,v)H(u,v)$$

Space convolution = frequency multiplication

In words: the Fourier transform of the convolution of two functions is the product of their individual Fourier transforms


f(x,y)
Gaussian
scale=3 pixels
g(x,y)
Fourier transform
Inverse Fourier
transform
|F(u,v)|


f(x,y)
Gaussian
scale=3 pixels
g(x,y)
*
Fourier transform
Inverse Fourier
transform
x
|F(u,v)|
|G(u,v)|

תמונה
טרנספורם
פורייה
עיבוד
טרנספורם
פורייה
הפוך
תמונה
אחרי
עיבוד

שיפור תמונה במרחב התדר

Blur / החלקה

High pass / Low pass

Low pass

mask

Low pass

mask

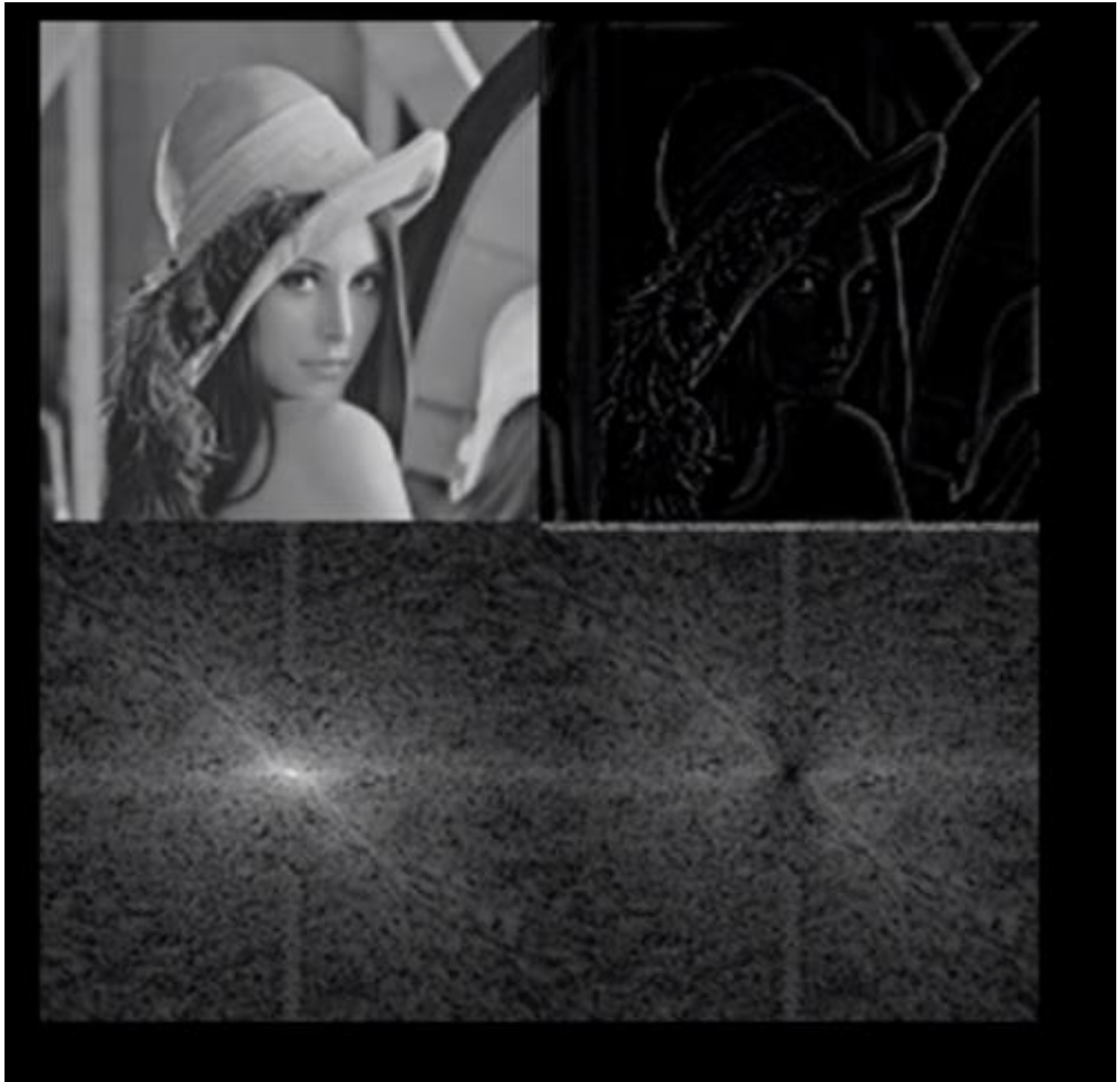
Low pass

mask

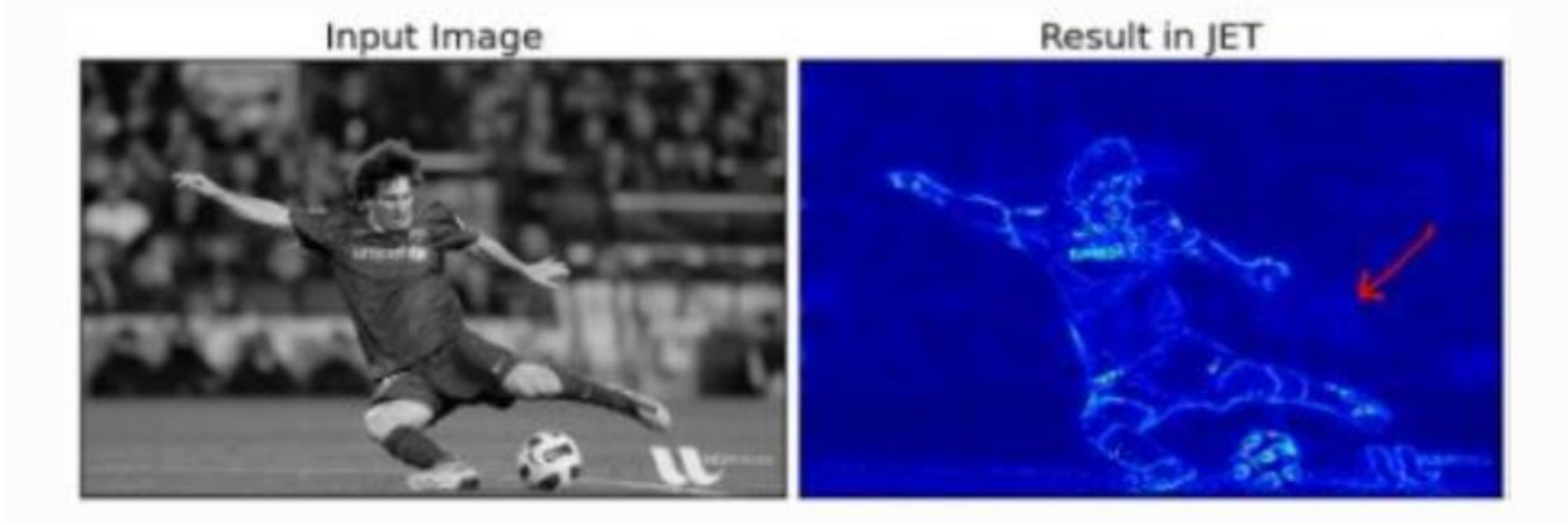
Low pass

mask

High pass



```
rows, cols = img.shape
crow,ccol = rows/2 , cols/2
fshift[crow-30:crow+30, ccol-30:ccol+30] = 0
f_ishift = np.fft.ifftshift(fshift)
img_back = np.fft.ifft2(f_ishift)
img_back = np.abs(img_back)

plt.subplot(131),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(132),plt.imshow(img_back, cmap = 'gray')
plt.title('Image after HPF'), plt.xticks([]), plt.yticks([])
plt.subplot(133),plt.imshow(img_back)
plt.title('Result in JET'), plt.xticks([]), plt.yticks([])
plt.show()
```

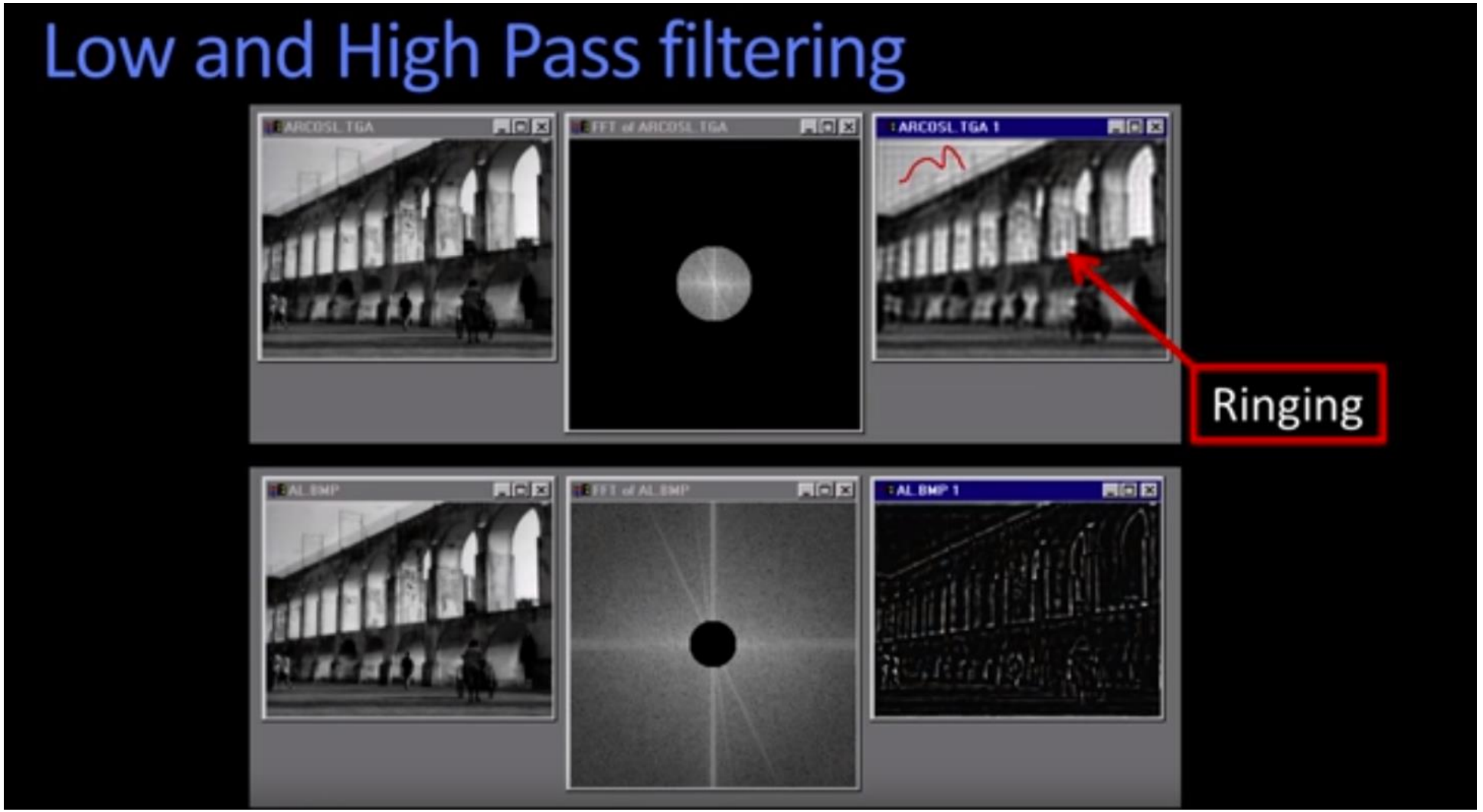


```
rows, cols = img.shape
crow,ccol = rows/2 , cols/2

# create a mask first, center square is 1, remaining all zeros
mask = np.zeros((rows,cols,2),np.uint8)
mask[crow-30:crow+30, ccol-30:ccol+30] = 1

# apply mask and inverse DFT
fshift = dft_shift*mask
f_ishift = np.fft.ifftshift(fshift)
img_back = cv2.idft(f_ishift)
img_back = cv2.magnitude(img_back[:,:,0],img_back[:,:,1])

plt.subplot(121),plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(img_back, cmap = 'gray')
plt.title('Magnitude Spectrum'), plt.xticks([]), plt.yticks([])
plt.show()
```

http://bigwww.epfl.ch/demo/ip/demos/FFT-filtering/

הדגמה

Low and High Pass filtering



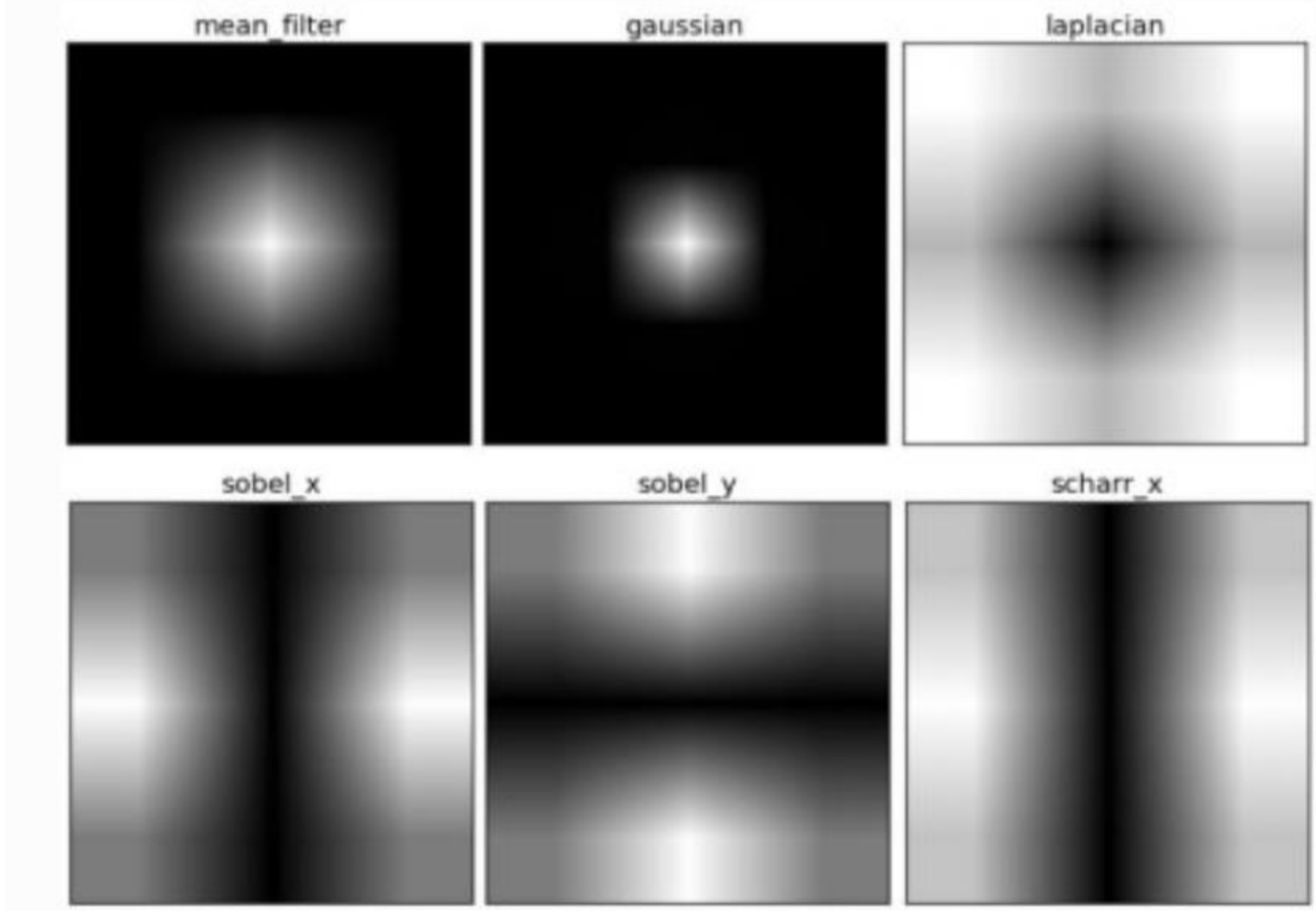
Why Laplacian is a High Pass Filter?

https://www.unioviedo.es/compnum/labs/PD.html2reiruoF_06YTHON/lab

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_transforms/py_fourier_transform/py_fourier_transform.html

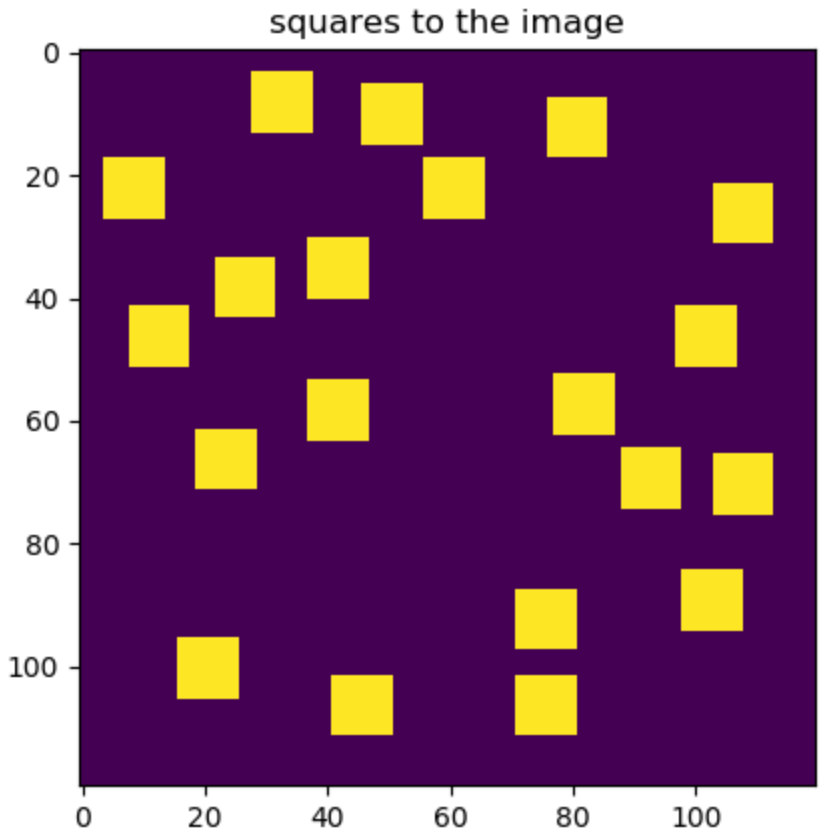
```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# simple averaging filter without scaling parameter
mean_filter = np.ones((3,3))

# creating a guassian filter
x = cv2.getGaussianKernel(5,10)
gaussian = x*x.T

# different edge detecting filters
# scharr in x-direction
scharr = np.array([[-3, 0, 3],
                   [-10,0,10],
                   [-3, 0, 3]])
# sobel in x direction
sobel_x= np.array([[-1, 0, 1],
                   [-2, 0, 2],
                   [-1, 0, 1]])
# sobel in y direction
sobel_y= np.array([[-1,-2,-1],
                   [0, 0, 0],
                   [1, 2, 1]])
# laplacian
laplacian=np.array([[0, 1, 0],
                    [1,-4, 1],
                    [0, 1, 0]])
```

```
filters = [mean_filter, gaussian, laplacian, sobel_x, sobel_y, scharr]
filter_name = ['mean_filter', 'gaussian','laplacian', 'sobel_x', \
                'sobel_y', 'scharr_x']
fft_filters = [np.fft.fft2(x) for x in filters]
fft_shift = [np.fft.fftshift(y) for y in fft_filters]
mag_spectrum = [np.log(np.abs(z)+1) for z in fft_shift]

for i in xrange(6):
    plt.subplot(2,3,i+1),plt.imshow(mag_spectrum[i],cmap = 'gray')
    plt.title(filter_name[i]), plt.xticks([]), plt.yticks([])

plt.show()
```

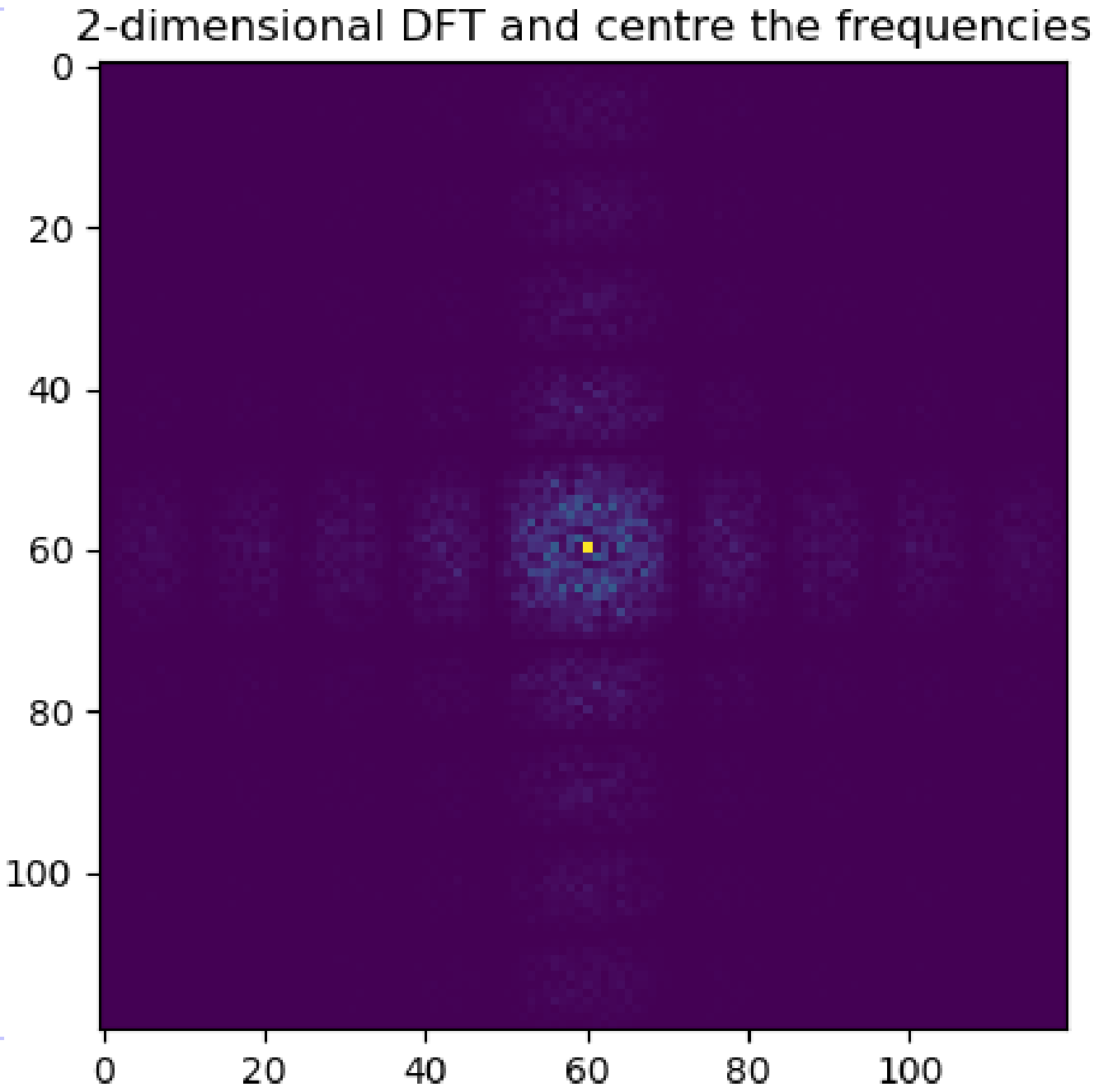

mean_filter
gaussian
laplacian
sobel_x
sobel_y
scharr_x

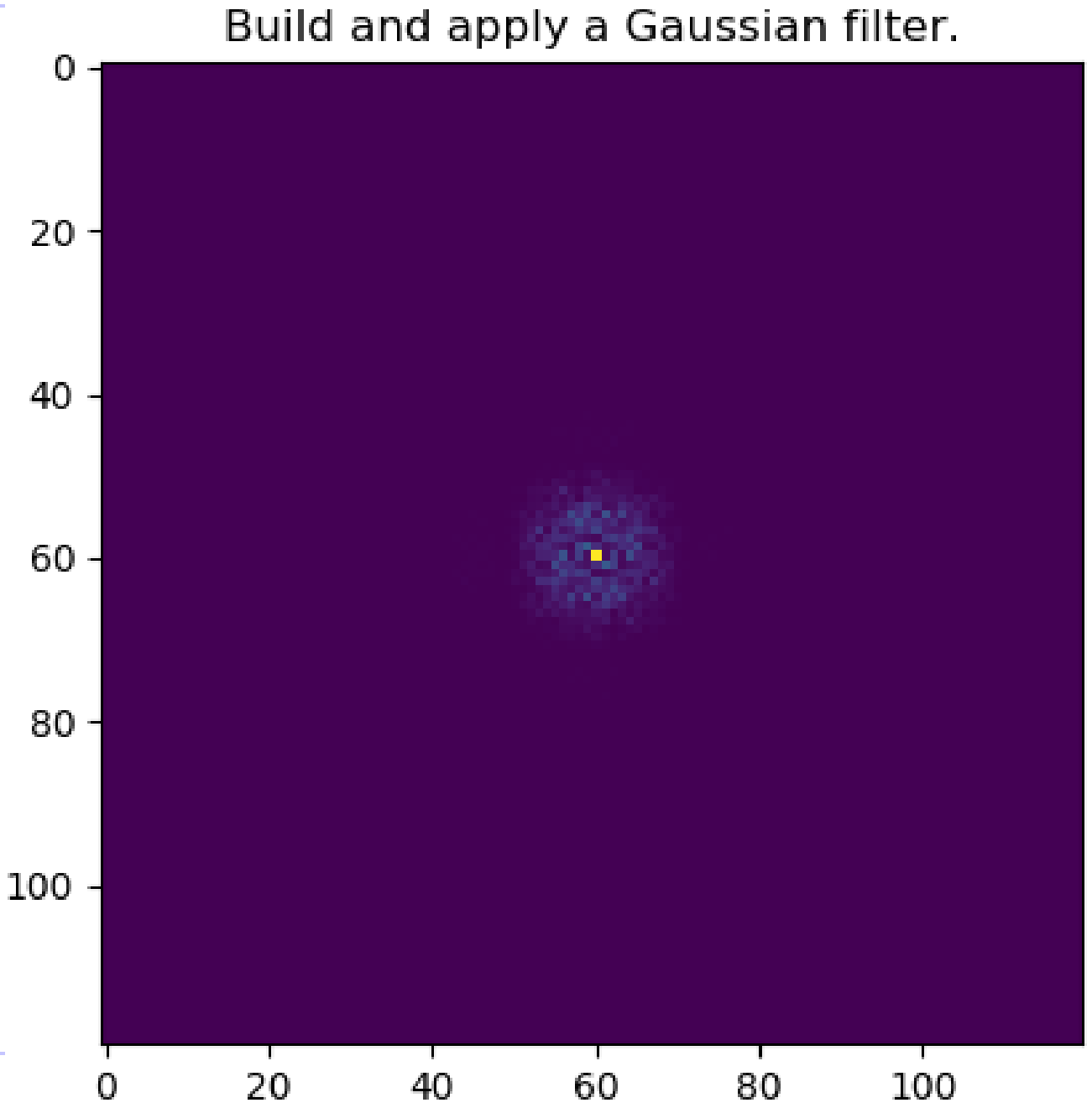
Low pass filter – code example

```
image = np.zeros((nrows, ncols))
sq_locs = np.zeros((nrows, ncols), dtype=bool)
sq_locs[1:-sq_size-1:,1:-sq_size-1] = True

def place_square():
    """ Place a square at random on the image and update sq_locs. """
    # valid_locs is an array of the indices of True entries in sq_locs
    valid_locs = np.transpose(np.nonzero(sq_locs))
    # pick one such entry at random, and add the square so its top left
    # corner is there; then update sq_locs
    i, j = valid_locs[np.random.randint(len(valid_locs))]
    image[i:i+sq_size, j:j+sq_size] = 1
    imin, jmin = max(0,i-sq_size-1), max(0, j-sq_size-1)
    sq_locs[imin:i+sq_size+1, jmin:j+sq_size+1] = False

# Add the required number of squares to the image
for i in range(nsq):
    place_square()
pylab.imshow(image)
pylab.title("squares to the image")
pylab.show()
```

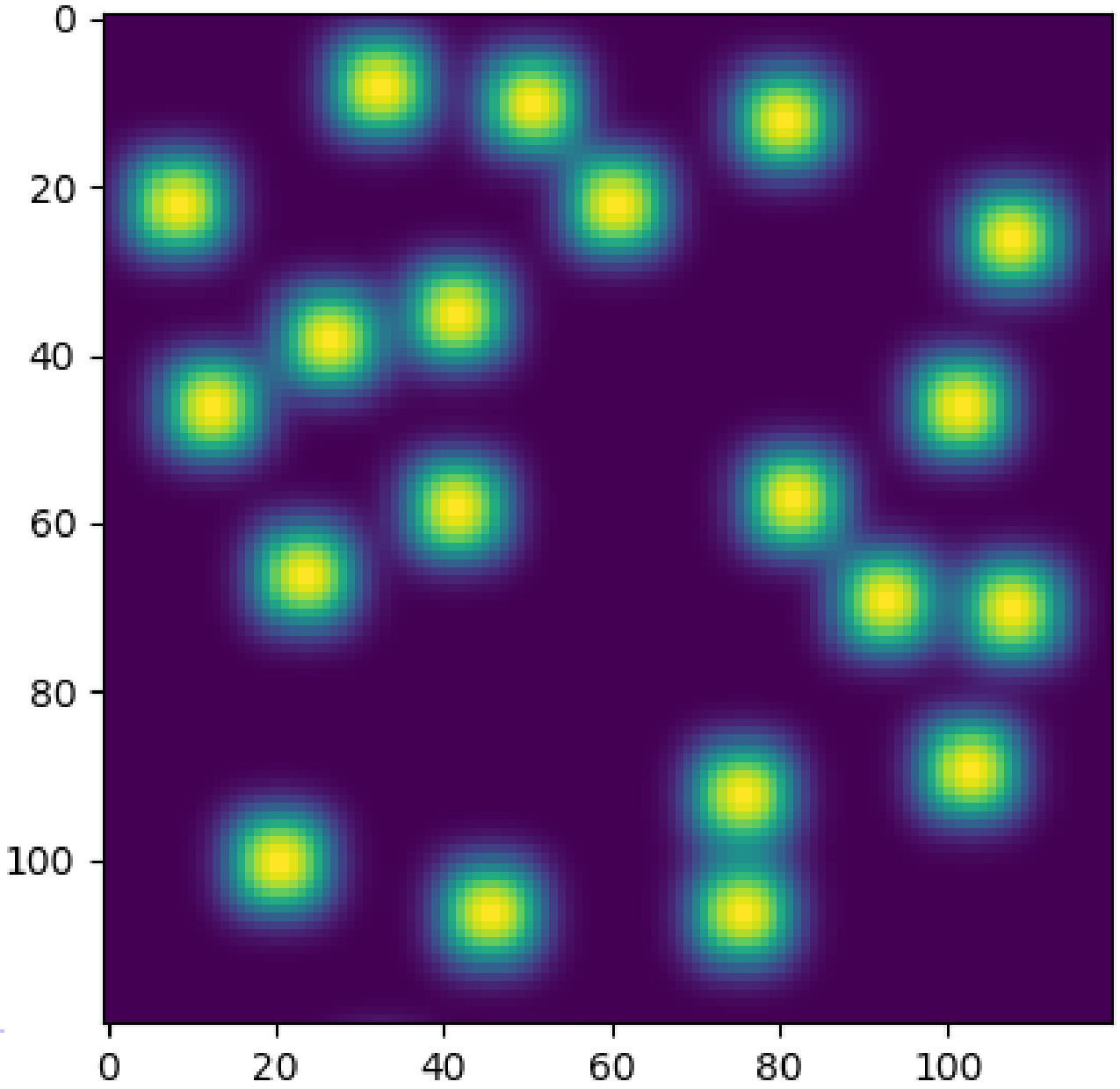

squares to the image
0
20
40
60
80
100
0
20
40
60
80
100

```
# Take the 2-dimensional DFT and centre the frequencies
ftimage = np.fft.fft2(image)
ftimage = np.fft.fftshift(ftimage)
pylab.imshow(np.abs(ftimage))
pylab.title("2-dimensional DFT and centre the frequencies")
pylab.show()


# Build and apply a Gaussian filter.
sigmax, sigmay = 10, 10
cy, cx = nrows/2, ncols/2
x = np.linspace(0, nrows, nrows)
y = np.linspace(0, ncols, ncols)
X, Y = np.meshgrid(x, y)
gmask = np.exp(-(((X-cx)/sigmax)**2 + ((Y-cy)/sigmay)**2))

ftimagep = ftimage * gmask
pylab.imshow(np.abs(ftimagep))
pylab.title("Build and apply a Gaussian filter.")
pylab.show()

# Finally, take the inverse transform and show the blurred image
imagep = np.fft.ifft2(ftimagep)
pylab.imshow(np.abs(imagep))
pylab.show()
```


2-dimensional DFT and centre the frequencies
0
20
40
60
80
100
0
20
40
60
80
100


Build and apply a Gaussian filter.
0
20
40
60
80
100
0
20
40
60
80
100

```
# Take the 2-dimensional DFT and centre the frequencies
ftimage = np.fft.fft2(image)
ftimage = np.fft.fftshift(ftimage)
pylab.imshow(np.abs(ftimage))
pylab.title("2-dimensional DFT and centre the frequencies")
pylab.show()


# Build and apply a Gaussian filter.
sigmax, sigmay = 10, 10
cy, cx = nrows/2, ncols/2
x = np.linspace(0, nrows, nrows)
y = np.linspace(0, ncols, ncols)
X, Y = np.meshgrid(x, y)
gmask = np.exp(-(((X-cx)/sigmax)**2 + ((Y-cy)/sigmay)**2))

ftimagep = ftimage * gmask
pylab.imshow(np.abs(ftimagep))
pylab.title("Build and apply a Gaussian filter.")
pylab.show()
```

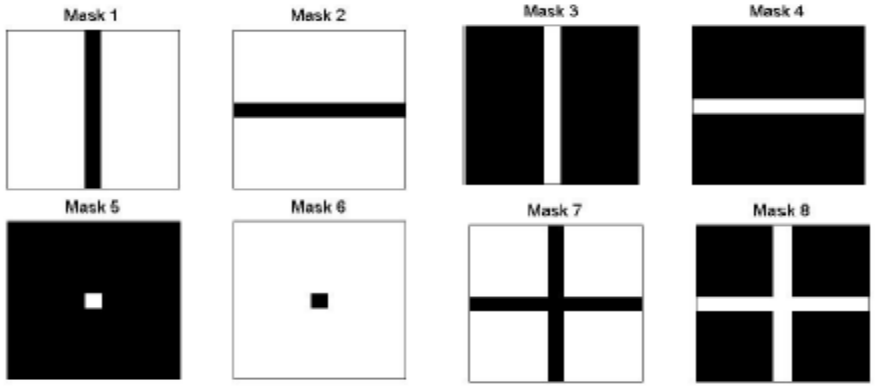

0
20
40
60
80
100
0
20
40
60
80
100

תרגיל כיתה 1

בשאלה זו נתונה תמונה (מצורף כקובץ PNG) וסט של פילטרים (אותו אתם צריכים ליצר). יש להפעיל את הפילטרים במישור התדר (רמז:fft). לשחזר את התמונה (רמז:ifft) ולנתח את התוצאות שמתקבלות.

<u>שלבי התוכנית:</u>

1. התוכנית ראשית מייצרת כל אחד מ8 הפילטרים אשר מודגמים באיור הבא:



2. במישור פוריה (התדר) התוכנית מפעילה (כלומר מכפילה) כל אחד מהפילטרים שיצרתם על התמונה של התדר.
3. ומפעילה את הטרנספורם ההפוך על התוצאה מסעיף 2.

יש לצרף הסבר ליד כל תמונה: (1) תיאור במילים של הפילטר (2) ומה הפעלתו גרמה לתמונה (3) ולמה.

JPEG

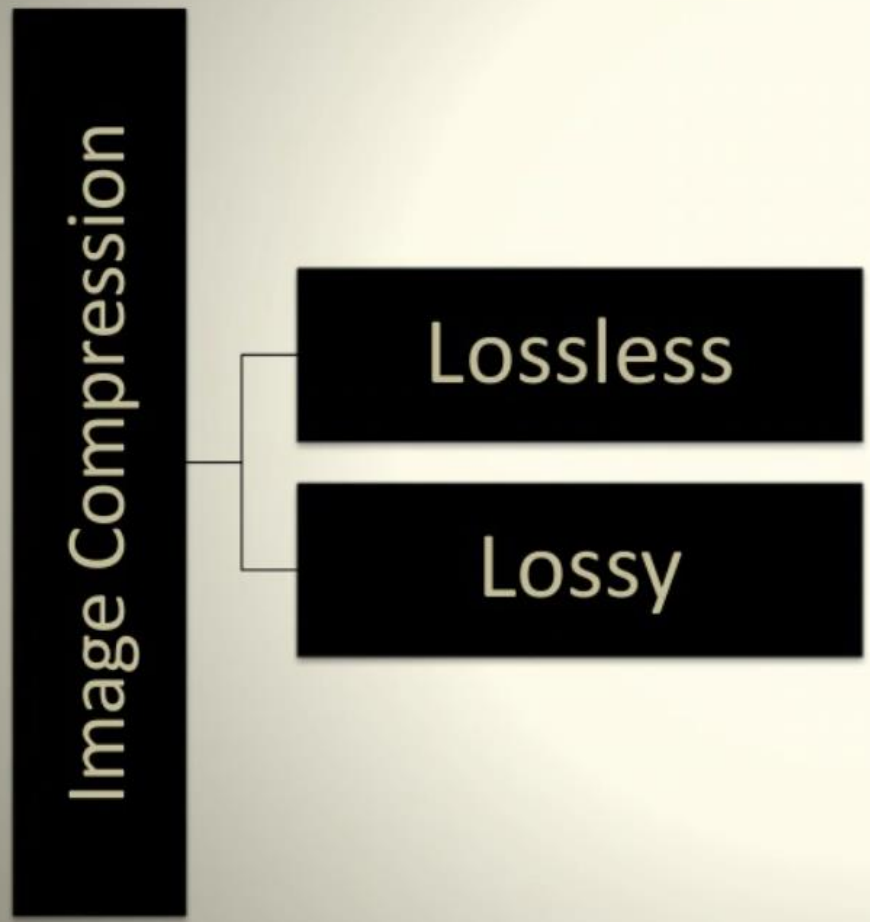
This is jpeg….

JPEGראשי תיבות של Joint Photographic Experts Group והוא אלגוריתם דחיסה מאבד שגורם לגדלים קטנים משמעותית של קבצים עם השפעה מועטה עד לא מורגשת על איכות התמונה והרזולוציה . תמונה דחוסה JPEGיכולה להיות קטנה פי עשרה מהמקורי. **JPEGפועל על ידי הסרת מידע שאינו גלוי בקלות לעין האנושית תוך שמירה על המידע שהעין האנושית מיטיבה לקלוט** .

What is Image Compression?

The objective of image compression is to reduce irrelevant and redundant image data in order to be able to store or transmit data in an efficient form.

Types



- **Lossless** image compression is a compression algorithm that allows the original image to be perfectly reconstructed from the original data.
- **Lossy** image compression is a type of compression where a certain amount of information is discarded which means that some data are lost and hence the image cannot be decompressed with 100% originality.

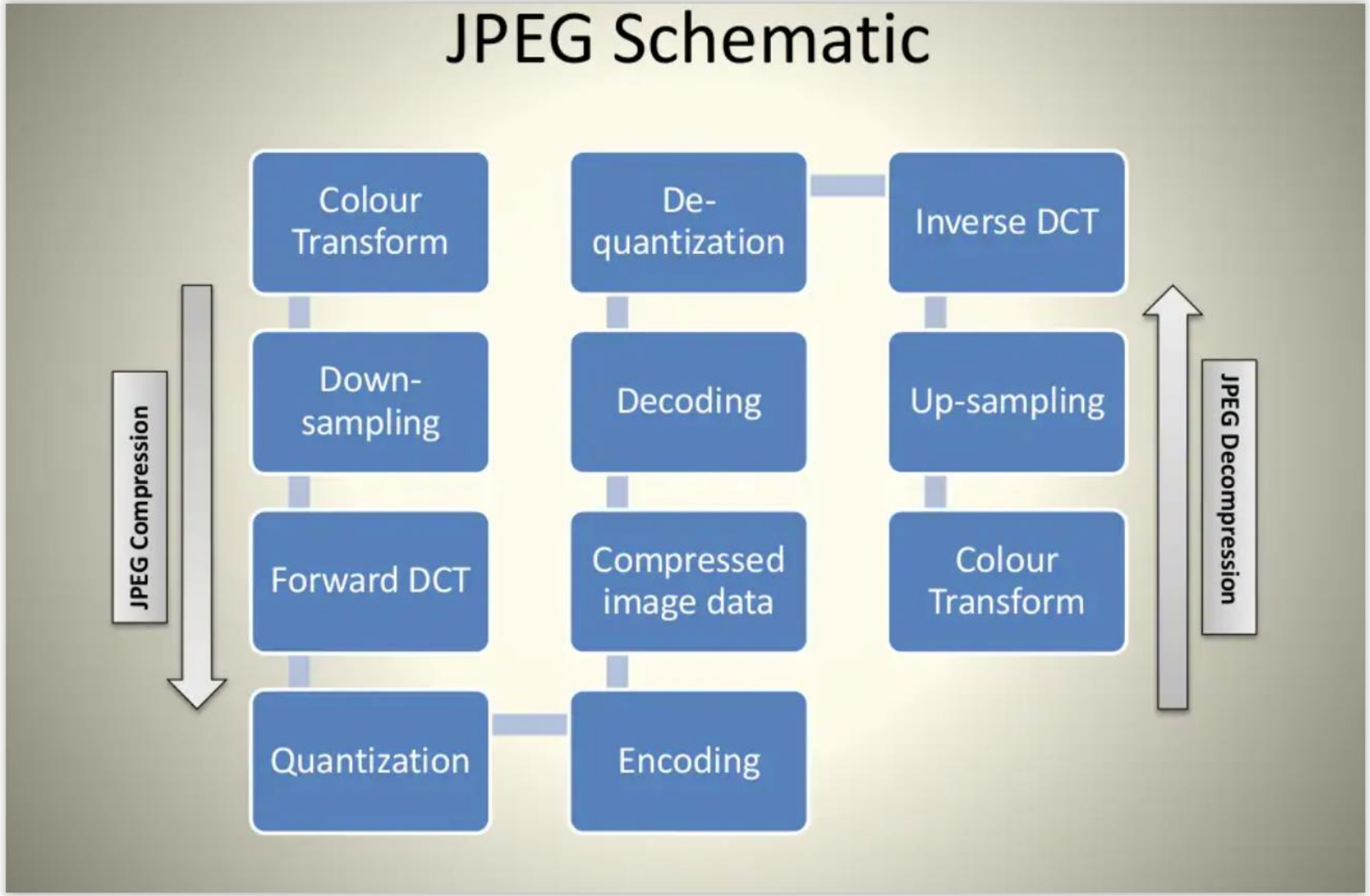
What is JPEG compression?

JPEG is a commonly used method of lossy compression for digital images. The degree of compression can be adjusted, allowing a tradeoff between storage size and image quality with a compression ratio 10:1; but with little perceptible loss in image quality.

Why JPEG?

JPEG uses ***transform coding***, it is largely based on the following observations:

- A large majority of useful image contents change relatively slowly across images, i.e., it is unusual for intensity values to alter up and down several times in a small area, for example, within an 8 x 8 image block. A translation of this fact into the spatial frequency domain, implies, generally, lower spatial frequency components contain **more information** than the high frequency components which often correspond to less useful details and noises.
- Experiments suggest that humans are more immune to loss of higher spatial frequency components than loss of lower frequency components. Human vision is insensitive to high frequency components.


JPEG Schematic
Colour Transform
Down-sampling
Forward DCT
Quantization
Encoding
Compressed image data
Decoding
De-quantization
Inverse DCT
Up-sampling
Colour Transform
JPEG Compression
JPEG Decompression

Algorithm

- **Splitting**: Split the image into 8 x 8 non-overlapping pixel blocks. If the image cannot be divided into 8-by-8 blocks, then you can add in empty pixels around the edges, essentially zero-padding the image.
- **Colour Space Transform** from [R,G,B] to [Y,Cb,Cr] & Subsampling.
- **DCT**: Take the Discrete Cosine Transform (DCT) of each 8-by-8 block.
- **Quantization**: quantize the DCT coefficients according to psycho-visually tuned quantization tables.
- **Serialization**: by zigzag scanning pattern to exploit redundancy.
- **Vectoring**: Differential Pulse Code Modulation (**DPCM**) on DC components
- Run Length Encoding (**RLE**) on AC components
- **Entropy Coding**:
  - Run Length Coding
  - Huffman Coding or Arithmetic Coding

Step I - Splitting

The input image is divided into smaller blocks having 8 x 8 dimensions, summing up to 64 units in total. Each of these units is called a ***pixel***, which is the smallest unit of any image.

Original image

Image split into multiple 8x8 pixel blocks

Step II - RGB to YCbCr conversion

- JPEG makes use of [Y,Cb,Cr] model instead of [R,G,B] model.
- The precision of colors suffer less (for a human eye) than the precision of contours (based on luminance)

Simple color space model: [R,G,B] per pixel

JPEG uses [Y, Cb, Cr] Model

Y = Brightness

Cb = Color blueness

Cr = Color redness

המרת מרחב צבע

- מרחב צבע הוא ארגון ספציפי של צבעים . RGB אדום, ירוק וכחול ו- CMYK ציאן, מגנטה, צהוב ומפתח/שחור) הם דוגמאות למרחבי צבע. יתרה מכך, לכל פיקסל בתמונה יש את ערכי מרחב הצבע שלו.
- YCbCr מורכב מ-3 ערוצים שונים: Y הוא ערוץ הבהירות המכיל מידע בהירות, ו- Cb ו- Cr הם ערוצי כחול ואדום המכילים מידע צבע.
- במרחב צבע זה, JPEG יכול לבצע שינויים בערוץ Cb ו- Cr כדי לשנות את מידע הצבע של התמונה מבלי להשפיע על מידע הבהירות שנמצא בערוץ הבהירות.
- לאחר ההמרה, לכל פיקסל יש שלושה ערכים חדשים המייצגים את הבהירות, הכרומיננטיות הכחול והמידע האדום הכרומיננטי
- העין האנושית רגישה יותר לבהירות מאשר לצבע. לכן, האלגוריתם יכול לנצל זאת על ידי ביצוע שינויים משמעותיים במידע הצבע של התמונה מבלי להשפיע על האיכות הנתפסת של התמונה .

Colour conversion

8x8 pixel
1 pixel = 3 components

MCU with
sampling factor
(1, 1, 1)

$$Y = 0.299R + 0.587G + 0.114B$$
$$Cb = -0.1687R - 0.3313G + 0.5B + 128$$
$$Cr = 0.5R - 0.4187G - 0.0813B + 128$$

הורדת דגימה

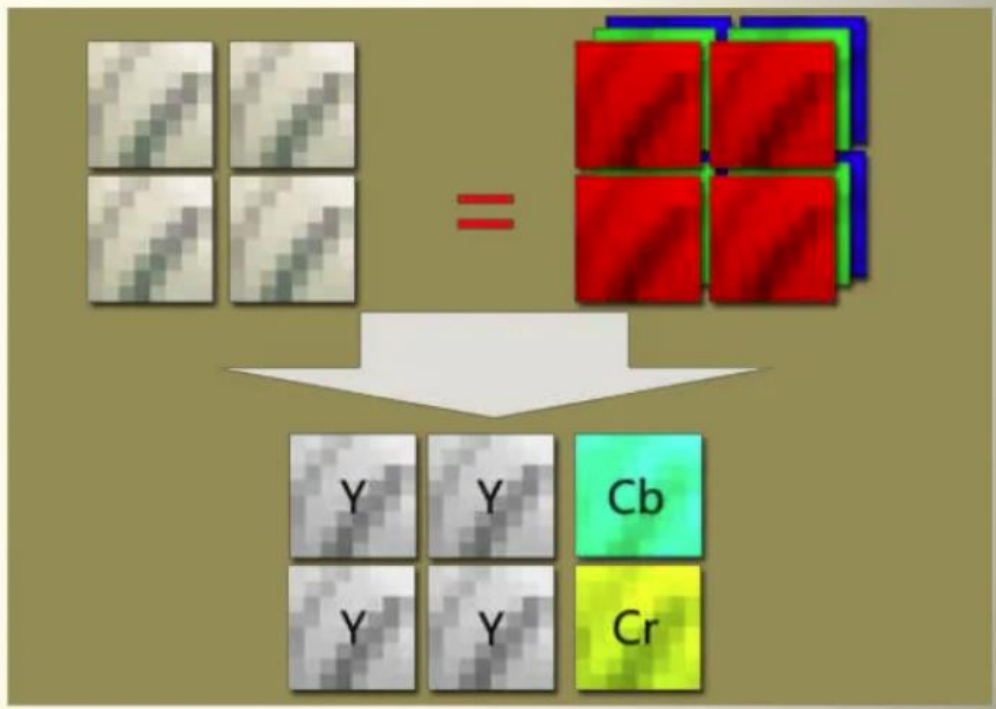
- לאחר שנפריד את מידע הצבע ממידע הבהירות, JPEG מוריד את דגימת ערוצי ה Cb ו-Cr לרבע מהגודל המקורי שלהם. כל בלוק של 4 פיקסלים נמדד לערך צבע יחיד עבור כל 4 הפיקסלים.
- כתוצאה מכך, חלק מהמידע הולך לאיבוד, וגודל התמונה מצטמצם בחצי, אך מכיוון שהעין האנושית רגישה פחות לצבע, לא ניתן להבחין בקלות בין השינויים.
- חשוב לציין שדגימה **הורדת מופעלת רק על ערוצי ה** Cb ו-Cr **, וערוץ הבהירות שומר על גודלו המקורי** .

Down-sampling

Y is taken for every pixel, and Cb, Cr are taken for a block of 2x2 pixels.

4 blocks
16 x16 pixel

MCU with sampling factor (2, 1, 1)



MCU = Minimum Coded Unit (smallest unit that can be coded)
Data size reduces to half.

Step III - Forward DCT

- The DCT uses the cosine function, therefore not interacting with complex numbers at all.
- DCT converts the information contained in a block(8x8) of pixels from ***spatial*** domain to the ***frequency*** domain.

Why DCT?

- Human vision is insensitive to high frequency components, due to which it is possible to treat the data corresponding to high frequencies as redundant. To segregate the raw image data on the basis of frequency, it needs to be converted into frequency domain, which is the primary function of DCT.

שלב III - DCT

- ה- DCT משתמש בפונקציית הקוסינוס ולכן אינו פועל עם מספרים מרוכבים כלל.
- ה- DCT ממיר את המידע הכלול בבלוק (8x8 פיקסלים) מהתחום המרחבי spatial domain לתחום התדרים frequency domain
- **מדוע DCT?**
  - הראייה האנושית אינה רגישה לרכיבי תדר גבוהים, ולכן ניתן להתייחס לנתונים המתאימים לתדרים גבוהים כעודפים. על מנת להפריד את נתוני התמונה הגולמיים לפי תדרים, יש להמיר אותם לתחום התדרים, וזהו התפקיד העיקרי של ה-DCT.

DCT Formula

- 1-D DCT –

$$F(w) = \frac{a(u)}{2} \sum_{n=0}^{N-1} f(n) cos \frac{(2n+1)w\pi}{16}$$

- But the image matrix is a 2-D matrix. So we can either apply 1-D DCT to the image matrix twice. Once row-wise, then column wise, to get the DCT coefficients. Or we can apply the standard 2-D DCT formula for JPEG compression. If the *input matrix* is *P(x,y)* and the *transformed matrix* is *F(u,v) or G(u,v)* then the DCT for the 8 x 8 block is computed using the expression:-

- 2-D DCT –

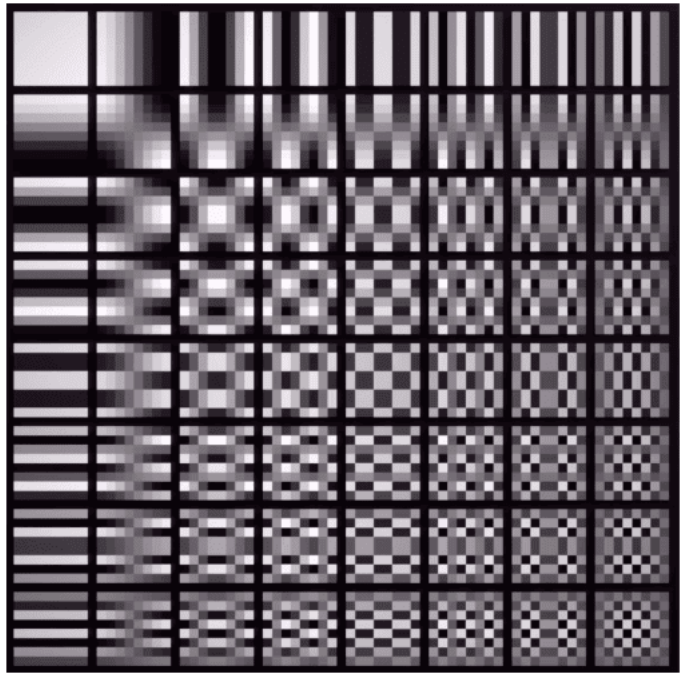
$$G_{u,v} = \frac{1}{4}\alpha(u)\alpha(v) \sum_{x=0}^{7} \sum_{y=0}^{7} g_{x,y} cos \left[\frac{(2x+1)u\pi}{16}\right] cos \left[\frac{(2y+1)v\pi}{16}\right]$$

חלוקה ל-8×8 פיקסל בלוקים

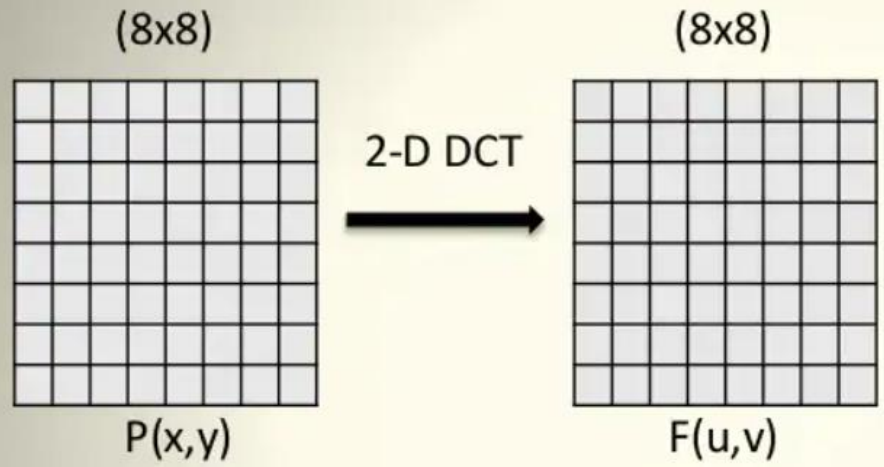
- לאחר דגימה מטה, נתוני הפיקסלים של כל ערוץ מחולקים ל-8×8 בלוקים של 64 פיקסלים. מעתה, האלגוריתם מעבד כל גוש פיקסלים באופן עצמאי.
-

Forward DCT (Discrete Cosine Transform)

- **באמצעות DCT עבור כל ערוץ, ניתן לשחזר כל בלוק של 64 פיקסלים על ידי הכפלת קבוצה קבועה של תמונות בסיס בערכי המשקל המתאימים שלהן ולאחר מכן לסכם אותן יחד** . כך נראות תמונות הבסיס:



DC and AC components



F(0,0) is called the ***DC component*** and the rest of P(x,y) are called ***AC components***.

- For *u = v = 0* the two cosine terms are 0 and hence the value in the location *F[0,0]* of the transformed matrix is simply a function of the summation of all the values in the input matrix.
- This is ***the mean*** of all 64 values in the matrix and is known as the **DC coefficient.**
- Since the values in all the other locations of the transformed matrix have a frequency coefficient associated with them they are known as **AC coefficients.**

Quantization

Standard Formula: $\widehat{F}(\mathrm{u}, v) = round\left(\frac{\mathrm{F(u,v)}}{\mathrm{Q(u,v)}}\right)$

- F(u,v) represents a *DCT coefficient*, Q(u,v) is *quantization matrix*, and $\hat{\mathrm{F}}(\mathrm{u}, \mathrm{v})$ represents the *quantized DCT coefficients* to be applied for successive entropy coding.
- The quantization step is the major information losing step in JPEG compression.
- For non-uniform quantization, there are 2 psycho-visually tuned quantization tables each for luminance (Y) and chrominance (Cb,Cr) components defined by jpeg standards.

Quantization

- **העין האנושית אינה טובה במיוחד בתפיסת אלמנטים בתדר גבוה בתמונה. בשלב זה, JPEG מסיר חלק ממידע זה בתדירות גבוהה מבלי להשפיע על האיכות הנתפסת של התמונה .**
- לשם כך, מטריצת המשקל מחולקת בטבלת כימות מחושבת מראש, והתוצאות מעוגלות למספר השלם הקרוב ביותר. כך נראית טבלת קוונטיזציה עבור ערוצי כרומיננס:

Quantization

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

The Luminance Quantization Table

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

The Chrominance Quantization Table

- The entries of Q(u,v) tend to have larger values towards the lower right corner. This aims to introduce more loss at the higher spatial frequencies.
- The tables above show the default Q(u,v) values obtained from psychophysical studies with the goal of maximizing the compression ratio while minimizing perceptual losses in JPEG images.

Quantization - Example

160	80	47	39	5	3	0	0
65	53	48	8	5	2	0	0
58	34	6	4	2	0	0	0
40	7	6	1	1	0	0	0
8	4	0	0	0	0	0	0
5	2	0	0	0	0	0	0
3	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

DCT Coefficients F(u,v)

Quantizer

16	7	4	2	0	0	0	0
5	4	3	0	0	0	0	0
4	2	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Quantized Coefficients $\hat{F}(u,v)$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Quantization Table Q(u,v)

Each element of F(u,v) is divided by the corresponding element of Q(u,v) and then rounded off to the nearest integer to get the $\hat{F}(u,v)$ matrix.

Huffman Coding

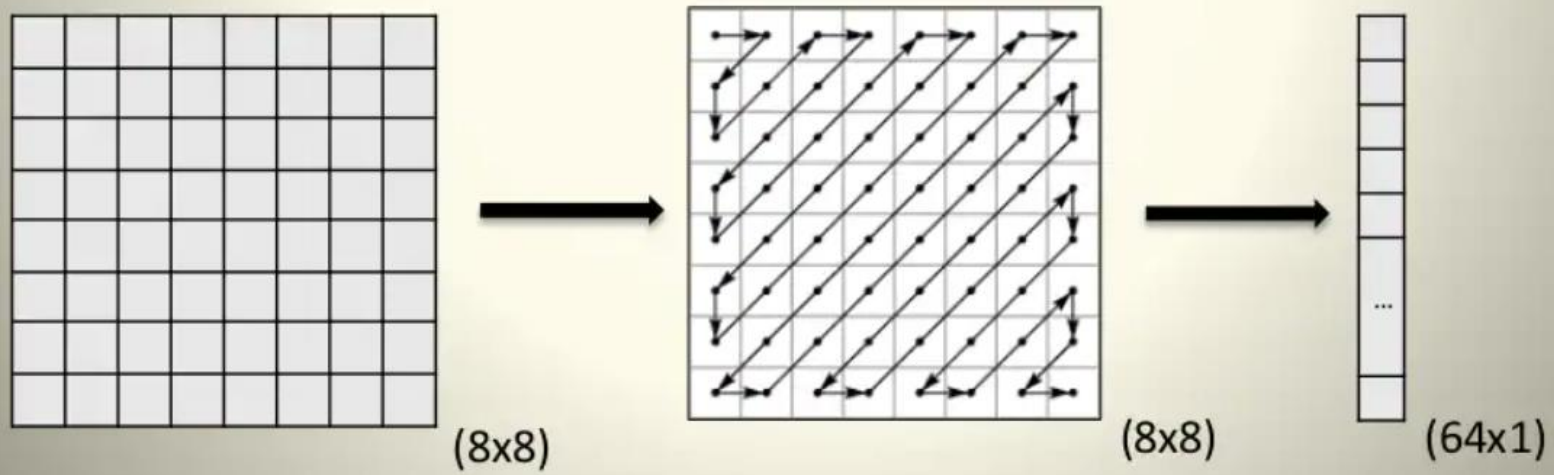
אנו יכולים לראות שלטבלאות הקוונטיזציה יש מספרים גבוהים יותר בפינה הימנית התחתונה, היכן שנמצאים נתונים בתדר גבוה, ויש להם מספרים נמוכים יותר בפינה השמאלית העליונה, היכן שנמצאים נתונים בתדר נמוך. כתוצאה מכך, לאחר חלוקת כל ערך משקל, נתוני תדר גבוה מעוגלים ל-0:

כעת כל מה שנותר לעשות הוא לרשום את הערכים שנמצאים בתוך כל מטריצה, להריץ (Run Length Encoding) RLE מכיוון שיש לנו הרבה אפסים, ולאחר מכן להפעיל את אלגוריתם Huffman Coding לפני אחסון הנתונים.

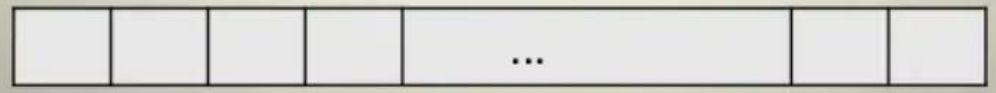
RLE- ו Huffman Coding הם אלגוריתמי דחיסה ללא אובדן שמצמצמים את המקום הדרוש לאחסון מידע מבלי להסיר נתונים כלשהם .

Step V – Zig-Zag Scan

- Maps 8 x 8 matrix to a 1 x 64 vector.
- Why zigzag scanning?
  - To group low frequency coefficients at the top of the vector and high frequency coefficients at the bottom.
- In order to exploit the presence of the large number of zeros in the quantized matrix, a zigzag of the matrix is used.



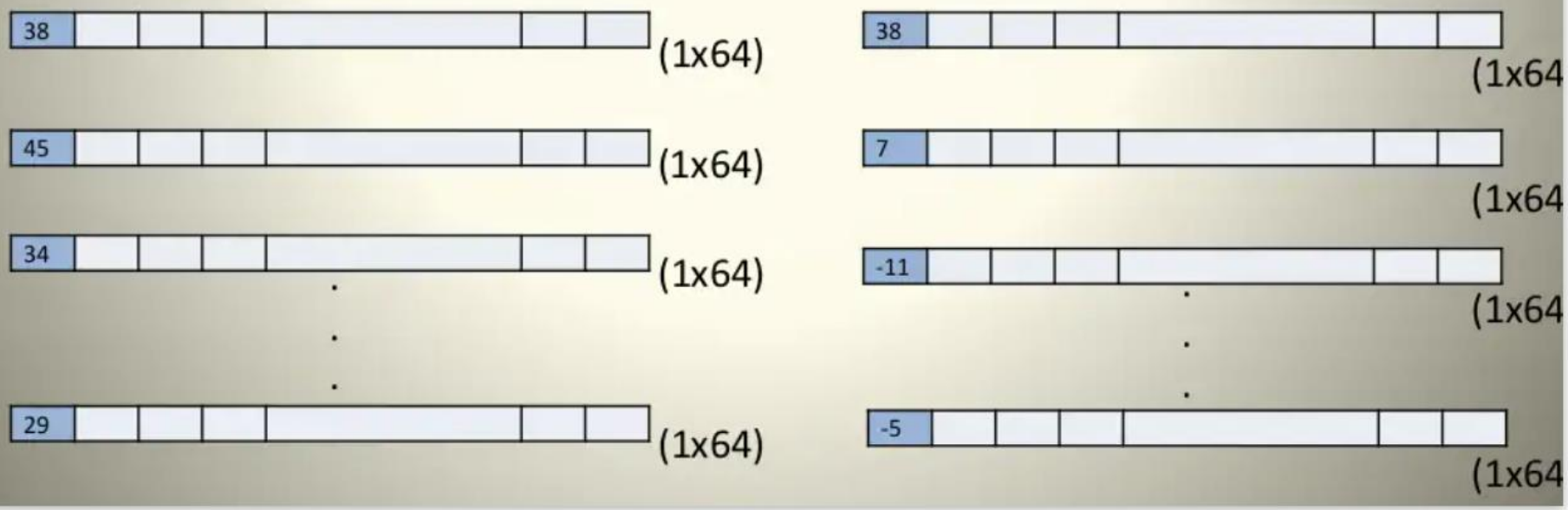
End Product:



(1x64)

Step VI – Vectoring: DPCM on DC component

- Differential Pulse Code Modulation (DPCM) is applied to the DC component.
- DC component is large and varies, but often close to previous value.
- DPCM encodes the difference between the current and previous 8x8 block.
- The entropy coding operates on a 1-dimensional string of values (vector). However the output of the quantization matrix is a 2-D matrix and hence this has to be represented in 1-D form. This is known as ***Vectoring***.
- Example:-



קידוד הופמן

- **ייצוג רכיבי DC ו- DC:**
- DC
- במקום לקודד את הערך המלא של רכיב ה- DC, קודדים את **ההפרש** בין רכיב ה- DC הנוכחי לבין זה של הבלוק הקודם בעזרת DPCM
  - ההפרש מקודד בעזרת קידוד הופמן:
    - נרשם גודל הביטים הנדרש לייצוג ההפרש **Size**
    - לאחר מכן נרשמים הביטים עצמם **Amplitude**
- **AC**
- לאחר הכימות, רבים מהרכיבי AC הם אפסים.
  - רכיבי ה- AC מקודדים בזוגות (skip, value)
    - **skip** מספר האפסים שנמצאים לפני הערך הלא-אפס הבא.
    - **value** הערך הלא-אפס הבא.
  - גם זוגות אלו עוברים קידוד הופמן כדי להקטין את מספר הביטים.

Huffman Coding: DC Components

- DC Components are coded as (*Size,Value*). The look-up table for generating code words for Value is as given below:-

SIZE	Value	Code
0	0	---
1	-1,1	0,1
2	-3, -2, 2,3	00,01,10,11
3	-7,…, -4, 4,…, 7	000,…, 011, 100,…111
4	-15,…, -8, 8,…, 15	0000,…, 0111, 1000,…, 1111
.		.
.		.
11	-2047,…, -1024, 1024,… 2047	…

Table 1: Huffman Table for DC components *Value* field

Huffman Coding – DC Components

- The look-up table for generating code words for *Size* is as given below:-

SIZE	Code Length	Code
0	2	00
1	3	010
2	3	011
3	3	100
4	3	101
5	3	110
6	4	1110
7	5	11110
8	6	111110
9	7	1111110
10	8	11111110
11	9	111111110

Table 2: Huffman Table for DC components *Size* field

Huffman DC Coding - Example

- If the DC component is 48, and the previous DC component is 52. Then the difference between the 2 components is (48-52) = -4.
- Therefore it is Huffman coded as: 100011
- 011: The codes for representing -4 (Table 1.)
- 100: The size of the value code word is 3. From Table 2, code corresponding to 3 is 100.

Huffman Coding: AC Components

- AC components are coded in two parts:
  - **Part1: (RunLength/SIZE)**
    - **RunLength:** The length of the consecutive zero values [0..15]
    - **SIZE:** The number of bits needed to code the *next* nonzero AC component's value. [0-A]
    - (0,0) is the End Of Block (EOB) for the 8x8 block.
    - **Part1** is Huffman coded (see Table 3)
  - **Part2: (Value)**
    - **Value:** Is the actual value of the AC component.(Table 1)

Run/ SIZE	Code Length	Code
0/0	4	1010
0/1	2	00
0/2	2	01
0/3	3	100
0/4	4	1011
0/5	5	11010
0/6	7	1111000
0/7	8	11111000
0/8	10	1111110110
0/9	16	1111111110000010
0/A	16	1111111110000011

Table 3(a)

Run/ SIZE	Code Length	Code
1/1	4	1100
1/2	5	11011
1/3	7	1111001
1/4	9	111110110
1/5	11	11111110110
1/6	16	1111111110000100
1/7	16	1111111110000101
1/8	16	1111111110000110
1/9	16	1111111110000111
1/A	16	1111111110001000
… 15/A	More	Such rows

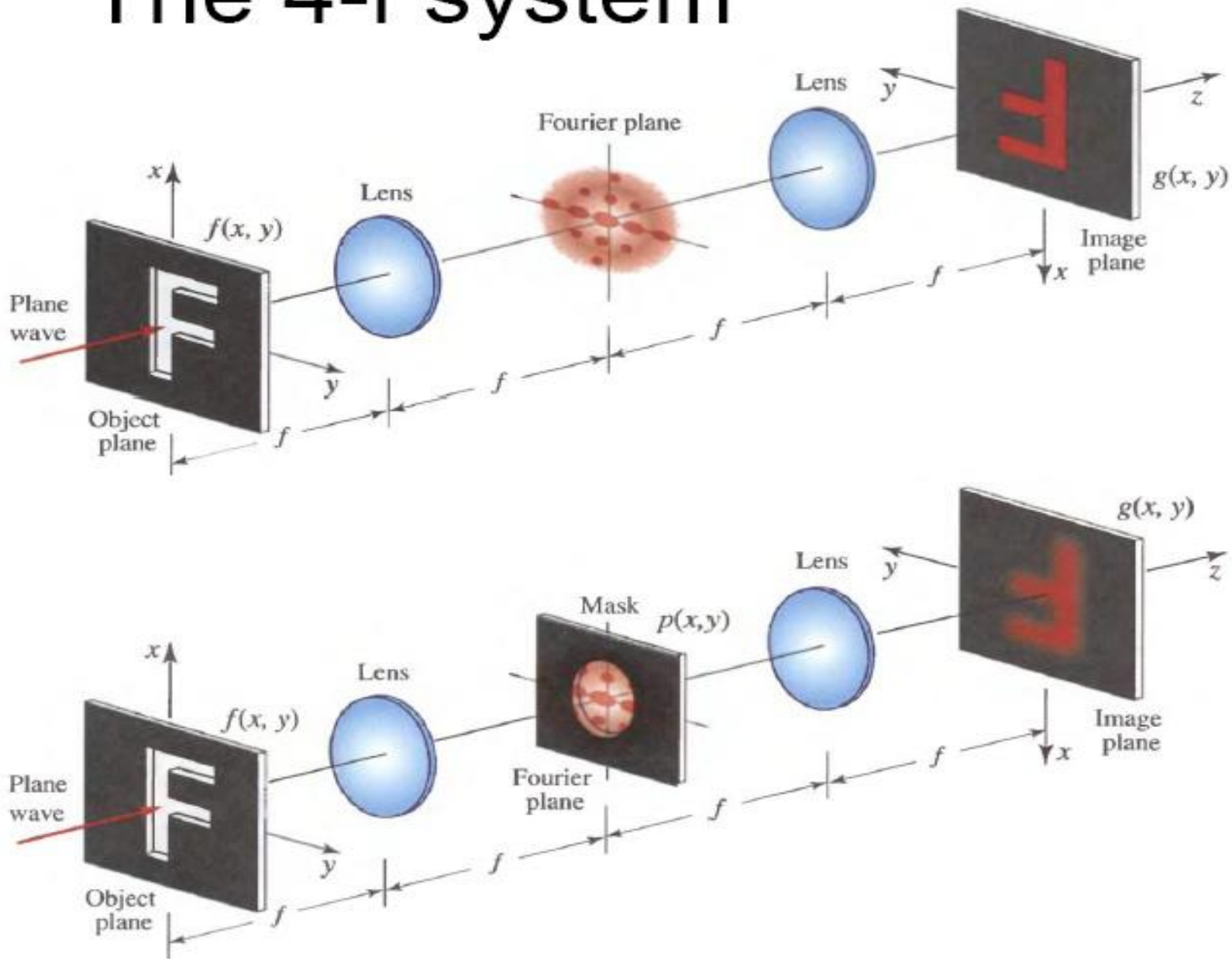
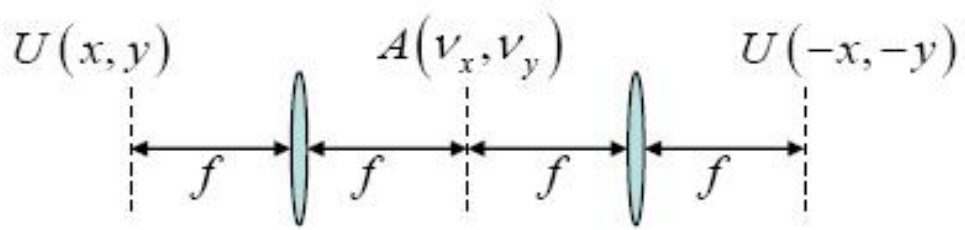
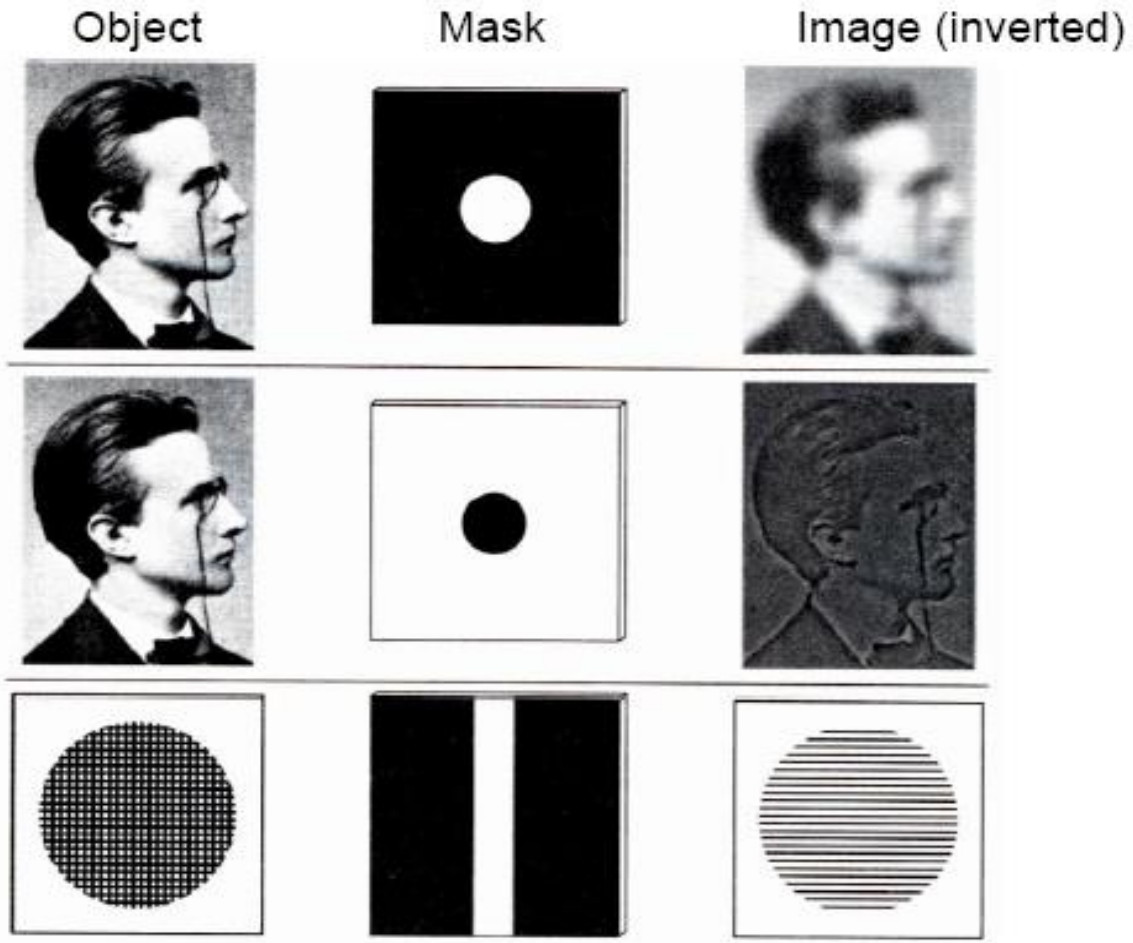
Table 3(b)

Merits of JPEG Compression

- Works with colour and grayscale images, but not with binary images.
- Up to 24 bit colour images (Unlike GIF)
- Target photographic quality images (Unlike GIF)
- Suitable for many applications e.g., satellite, medical, general photography, etc.

בונוס..

The 4-f system



Imaging examples